AI-POWERED PROACTIVE YOUTUBE TITLE OPTIMIZER

A Multi-Agent System for Psychologically Optimised, Faithfulness-Constrained Video Title Generation

P.A.D.B. PATHIRAJA

Student ID: 29071

BSc (Hons) Software Engineering

Department of to be confirmed

Faculty of Computing

NSBM Green University

Sri Lanka

September 2026

AI-POWERED PROACTIVE YOUTUBE TITLE OPTIMIZER

A Multi-Agent System for Psychologically Optimised, Faithfulness-Constrained Video Title Generation

A thesis submitted to NSBM Green University for the degree of BSc (Hons) Software Engineering

By

P.A.D.B. PATHIRAJA

Student ID: 29071

Department of to be confirmed

Faculty of Computing

NSBM Green University

Sri Lanka

September 2026

DECLARATION

I declare that the content of this undergraduate thesis titled "AI-Powered Proactive YouTube Title Optimizer" is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for any other degree in any university or institution of higher learning.

Signature: ……

Name: P.A.D.B. Pathiraja (Student ID: 29071)

Date: ……

Signature of the Supervisors

……

Ms. Nimesha surname

Principal Supervisor

Senior Lecturer / Lecturer

Department of to be confirmed,

NSBM Green University

Date: ……

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Ms. Nimesha, for her guidance, patience and encouragement throughout this research. Her constructive feedback and support at every stage shaped both the system and this thesis, and I am truly grateful for the time and care she invested in my work.

I also thank the lecturers and staff of the Faculty of Computing at NSBM Green University for the knowledge and support I received during my degree, and the participants who took the time to complete the evaluation survey.

Finally, I thank my family and friends for their constant support, understanding and motivation, which carried me through the demanding months of this project.

ABSTRACT

Video titles strongly influence whether a viewer clicks, yet writing titles that are both engaging and faithful to the video content is a skill most creators lack, and generic large language model (LLM) prompting rarely provides it. This research aimed to design, develop and evaluate a multi-agent AI system, implemented as a Facade-pattern API wrapper, that generates psychologically optimised YouTube titles while preserving semantic faithfulness to the source video. Following the Design Science Research Methodology of Peffers et al., the system was built as an Analyzer, Generator and Critic agent pipeline orchestrated with LangGraph and served through FastAPI. The Generator uses Few-Shot, Chain-of-Thought and Persona prompting, and the Critic combines a predicted click-through rate (pCTR) model with VADER, BERTScore and natural language inference (NLI) checks. The system was evaluated through functional and non-functional testing, offline model metrics, a system-level evaluation on 20 videos, an LLM-as-a-Judge evaluation (60 evaluations) and a human survey (n = 29). All eleven functional tests passed. The pCTR classifier reached an F1 score of 0.6901 and a ROC-AUC of 0.7613, below its targets of 0.75 and 0.80, and four of six system-level targets were met. The system significantly outperformed a purely descriptive baseline and held a significant advantage over a zero-shot GPT-4o baseline on relevance. The human survey was directionally consistent with these results but not statistically significant, and agreement between human and LLM ratings was weak (r = -0.14). The work shows that generative AI can be systematically constrained to produce faithful, engagement-oriented titles, while the moderate predictive performance and pilot-scale human evidence mark clear limits to the current validation.

TABLE OF CONTENTS

Declaration ii
Acknowledgement iii
Abstract iv
Table of Contents v
List of Figures ix
List of Tables x
List of Abbreviations xi
1 Introduction 1
1.1 Chapter overview 1
1.2 Problem background 1
1.2.1 The video-first internet 1
1.2.2 The algorithmic gatekeeper: CTR and the cold-start loop 2
1.2.3 The expertise bottleneck 3
1.2.4 The rise of LLMs and their generic application gap 3
1.3 Problem statement 4
1.3.1 General problem 4
1.3.2 Specific problem 5
1.4 Research question 6
1.4.1 Primary research question 6
1.4.2 Supporting sub-questions 6
1.5 Research motivation 7
1.5.1 Economic and social motivation 7
1.5.2 Technical motivation 8
1.5.3 Ethical motivation 8
1.6 Research aim 9
1.7 Rich picture of the proposed solution 9
1.8 Resource requirements 11
1.8.1 Hardware 11
1.8.2 Software 12
1.9 Project scope 13
1.9.1 In scope 13
1.9.2 Out of scope 13
1.10 Chapter summary 14
2 Objectives 16
2.1 To identify 16
2.2 To analyse 16
2.3 To design and develop 16

2.4 To evaluate 16
3 Literature Review 18
3.1 Chapter overview 18
3.2 Conceptual map of the literature 18
3.3 Domain overview 19
3.3.1 The video-first internet and the creator economy 19
3.3.2 Click-through rate as the algorithmic gatekeeper 20
3.3.3 Historical evolution: from manual copywriting to AI-assisted generation 21
3.4 Existing systems, frameworks, and designs 21
3.4.1 System 1: manual expert copywriting 21
3.4.2 System 2: YouTube analytics platforms (TubeBuddy, VidIQ) 22
3.4.3 System 3: generic LLM prompting (zero-shot) 23
3.4.4 System 4: clickbait detection systems 24
3.4.5 Comparative summary 25
3.5 Technological analysis 26
3.5.1 Algorithmic analysis 26
3.5.1.1 A1: Large language models, the transformer architecture 26
3.5.1.2 A2: Prompt engineering techniques 26
3.5.1.3 A3: Sentiment analysis, VADER 28
3.5.1.4 A4: Semantic faithfulness, BERTScore 29
3.5.1.5 A5: Natural language inference for faithfulness verification 30
3.5.1.6 A6: Predicted CTR (pCTR), machine learning model 31
3.5.2 Design analysis 31
3.5.2.1 D1: Multi-agent architecture, supervisor pattern 31
3.5.2.2 D2: API wrapper with facade pattern 32
3.5.2.3 D3: Psychological trigger operationalisation 33
3.5.3 Workflow analysis 34
3.5.3.1 W1: Dataset pipeline 34
3.5.3.2 W2: Evaluation workflow 35
3.6 Reflection, research gap justification 36
3.7 Chapter summary 37
4 Methodology 39
4.1 Research paradigm 39
4.1.1 Paradigm: pragmatism 39
4.2 Research approach 40
4.2.1 Approach: abductive 40
4.3 Research strategy 41
4.3.1 Strategy: design science research (DSR) 41

4.3.2 Alternative strategies considered and rejected....41
4.4 Fact collection mechanisms....42
4.4.1 Data ethical considerations....43
4.5 Research methodology....43
4.5.1 Methodology: Peffers' design science research methodology (DSRM)..43
4.6 Execution workflow....44
4.6.1 Entry point....45
4.7 Project management methodology....46
4.7.1 Methodology: Agile (Scrum-inspired)....46
4.7.2 Alternative considered: PRINCE2....46
4.8 Project timeline....47
4.9 Ethical considerations....49
4.9.1 Deception versus persuasion....49
4.9.2 Data privacy....50
4.9.3 Bias and toxicity in training data....50
4.9.4 Transparency and human oversight....50
4.9.5 Academic integrity....51
4.9.6 Platform policy compliance....51
4.10 Chapter summary....51
5 System Requirements Specification....53
5.1 Chapter overview....53
5.2 Stakeholder analysis....54
5.3 Operationalisation process....55
5.4 Use case diagrams....56
5.5 Class diagram....57
5.6 Activity diagram....58
5.7 Sequence diagrams....60
5.8 Proposed system architecture diagram....62
5.9 Functional requirements....63
5.10 Non-functional requirements....65
5.11 Chapter summary....66
6 Implementation....67
6.1 Chapter overview....67
6.2 Technology stack and justification....67
6.3 Repository and module structure....71
6.4 Analyzer agent....73
6.5 Prompt factory and dynamic prompt database....75
6.6 Generator agent....77
6.7 Predicted-CTR (pCTR) machine learning model....78

6.8 Critic agent....81
6.9 LangGraph orchestration and FastAPI facade....84
6.10 Demonstration frontend....86
6.11 Sample end-to-end system output....89
6.12 Implementation deviations from the plan....90
6.13 Chapter summary....91
7 Results....93
7.1 Chapter overview....93
7.2 Functional test plan and results....94
7.3 Offline evaluation of the pCTR model....96
7.4 System-level evaluation....99
7.5 Non-functional testing....102
7.6 Human evaluation survey: design and results....103
7.7 LLM-as-a-judge evaluation....106
7.8 Comparison against baselines A and B....107
7.9 Review of test strategies and their suitability....109
7.10 Chapter summary....110
8 Discussion and Conclusions....112
8.1 Chapter overview....112
8.2 Accomplishment of the research objectives....112
8.2.1 Objective 2.1: to identify....112
8.2.2 Objective 2.2: to analyse....113
8.2.3 Objective 2.3: to design and develop....113
8.2.4 Objective 2.4: to evaluate....114
8.3 Problems encountered during research and development....115
8.4 Self-reflection on the research process....116
8.5 Real-world application possibilities....117
8.6 Future recommendations....118
8.7 Chapter summary....119
References....121

LIST OF FIGURES

Page

Figure 1.1 Rich Picture: YouTube Title Optimizer (YTO) System Architecture.....10
Figure 5.1 Use Case Diagram.....56
Figure 5.2 Class Diagram.....58
Figure 5.3 Activity Diagram.....59
Figure 5.4 Sequence Diagram: Successful Generation Path.....60
Figure 5.5 Sequence Diagram: Critic Agent Scoring and Rejection Logic.....61
Figure 5.6 Proposed System Architecture.....62
Figure 6.1 Module Dependency Structure.....72
Figure 6.2 Critic Agent Parallel Scoring.....83
Figure 6.3 Deployment and Component View.....88
Figure 7.1 ROC Curve, pCTR Classifier.....97
Figure 7.2 Confusion Matrix, pCTR Classifier.....98
Figure 7.3 Feature Importance, pCTR Classifier.....99
Figure 7.4 System-Level Evaluation, All Six Metrics vs. Targets.....101

LIST OF TABLES

Page

Table 1.1 Hardware resource requirements....11
Table 1.2 Software resource requirements....12
Table 1.3 In-scope items....13
Table 1.4 Out-of-scope items....13
Table 3.1 Conceptual Map of the Literature....18
Table 3.2 Comparative Analysis of Existing Approaches....25
Table 3.3 Psychological Trigger Taxonomy and Linguistic Operationalisation....33
Table 4.1 Alternative research strategies considered and rejected....41
Table 4.2 Data Collection Mechanisms....42
Table 4.3 DSRM Execution Workflow....44
Table 4.4 Project Timeline....47
Table 5.1 Operationalisation of Research Objectives....55
Table 5.2 Functional Requirements....63
Table 5.3 Non-Functional Requirements....65
Table 6.1 Technology Stack and Justification....67
Table 6.2 Module-to-Architecture Mapping....72
Table 6.3 Sample End-to-End Output (Science & Technology category)....89
Table 6.4 Consolidated Implementation Deviations....90
Table 7.1 Functional Test Cases....94
Table 7.2 pCTR Model, Held-Out Test Set Results....96
Table 7.3 System-Level Evaluation, Aggregate Results (n = 20 runs)....99
Table 7.4 NLI Rejection Reasons (Sample from system_eval_results.json)....101
Table 7.5 Human Evaluation: Mean Scores by Condition (n = 29 respondents)....104
Table 7.6 Human Evaluation: Wilcoxon Signed-Rank Significance (System vs. Each Baseline)....105
Table 7.7 LLM-as-a-Judge, Mean Scores by Condition (n = 20 videos)....106
Table 7.8 LLM-as-a-Judge, Wilcoxon Signed-Rank Significance (System vs. Each Baseline)....106
Table 7.9 Summary: System vs. Baselines....107
Table 8.1 Objective Accomplishment Summary....114

LIST OF ABBREVIATIONS

AAAI	Association for the Advancement of Artificial Intelligence
ACL	Association for Computational Linguistics
AI	Artificial Intelligence
ANEW	Affective Norms for English Words
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
AUC	Area Under the Curve
BERT	Bidirectional Encoder Representations from Transformers
BLEU	Bilingual Evaluation Understudy
CI/CD	Continuous Integration and Continuous Delivery
CLI	Command-Line Interface
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
CTR	Click-Through Rate
DSR	Design Science Research
DSRM	Design Science Research Methodology
FAISS	Facebook AI Similarity Search
FOMO	Fear of Missing Out
GB	Gigabyte
GIL	Global Interpreter Lock
GPT	Generative Pre-trained Transformer
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
ID	Identifier
IEEE	Institute of Electrical and Electronics Engineers
IS	Information Systems
JSON	JavaScript Object Notation
LIWC	Linguistic Inquiry and Word Count
LLM	Large Language Model
LSTM	Long Short-Term Memory
METEOR	Metric for Evaluation of Translation with Explicit Ordering
ML	Machine Learning
NLI	Natural Language Inference

NLP	Natural Language Processing
NLTK	Natural Language Toolkit
ORM	Object-Relational Mapping
OS	Operating System
P50 / P95	50th / 95th Percentile (latency)
pCTR	Predicted Click-Through Rate
POS	Part of Speech
PRINCE2	Projects IN Controlled Environments
RAM	Random Access Memory
REST	Representational State Transfer
RMSE	Root Mean Square Error
RNN	Recurrent Neural Network
ROC	Receiver Operating Characteristic
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
RQ	Research Question
SDK	Software Development Kit
SE	Software Engineering
SEO	Search Engine Optimisation
SOT	Source of Truth
SRS	System Requirements Specification
SSD	Solid-State Drive
TER	Translation Edit Rate
UI	User Interface
UML	Unified Modelling Language
VADER	Valence Aware Dictionary and sEntiment Reasoner
VRAM	Video Random Access Memory
XSS	Cross-Site Scripting
YTO	YouTube Title Optimizer

1 INTRODUCTION

1.1 Chapter overview

The internet has undergone a fundamental structural shift over the past decade. Where once the primary mode of online information exchange was text (articles, blog posts, search results), video has emerged as the dominant format through which billions of people learn, discover, and decide. YouTube alone processes over 500 hours of new video content every single minute [1], creating an environment in which the challenge of digital content is no longer *production* but *discovery*. In this landscape, the metadata attached to a video, primarily its title, is the interface between a creator's work and the algorithmic systems that determine whether that work will be seen.

This chapter establishes the foundation for the research project. Section 1.2 provides the problem background, tracing the evolution of the video-first internet and the mechanics of the algorithmic discovery ecosystem that has made metadata optimization a first-order problem. Section 1.3 presents the problem statement at both a general and domain-specific level, culminating in the identification of the research gap. Section 1.4 formalises the research question. Sections 1.5 and 1.6 present the research motivation and aim. Chapter 2 states four specific research objectives. Section 1.7 provides a rich picture of the proposed solution. Sections 1.8 and 1.9 specify the resource requirements and project scope. The chapter closes with a summary in Section 1.10.

1.2 Problem background

1.2.1 The video-first internet

The global media ecosystem has experienced a decisive shift toward video-based communication. Platforms including YouTube, TikTok, and Instagram Reels have collectively transformed how information is consumed, making short-to-medium form video the primary medium for education, entertainment, news, and product discovery. YouTube, launched in 2005, has grown to become the world's second-largest search engine and processes more than 500 hours of uploaded video content every single minute [1]. That volume, on its own, is a structural problem of supply overwhelming demand.

In a text-based information environment, search algorithms could rely on body text, links, and explicit keyword signals to rank and surface content. In a video-first environment, the *content itself* (the audio, the video stream) is largely opaque to algorithmic indexing systems at the moment of upload. The video's textual metadata (its title, description, and tags) becomes the primary signal by which a recommender system categorises, understands, and distributes new content before any engagement data exists.

1.2.2 The algorithmic gatekeeper: CTR and the cold-start loop

YouTube's recommendation system, as documented by Covington, Adams, and Sargin (2016) in the foundational paper *Deep Neural Networks for YouTube Recommendations* [2], operates as a two-stage pipeline: a candidate generation stage that narrows billions of videos to hundreds of candidates, followed by a ranking stage that orders those candidates for individual users. The ranking stage weighs multiple signals, and click-through rate (CTR), the proportion of users who click a video upon seeing its impression, is one of the most immediate and influential of them.

The system operates through an implicit exploration-exploitation loop for new videos. When a creator uploads a new video, the algorithm serves an impression of that video to a small test cohort of users. If the CTR recorded during this test window is strong, the system expands distribution to progressively larger audiences. If CTR is weak, distribution is suppressed. This is the *cold-start problem* as it applies to YouTube content [2]: a video has no historical engagement data, so the algorithm must use the earliest available signal, the click decision made on the basis of the title and thumbnail alone, to determine whether to invest further distribution resources.

The asymmetric consequence of this mechanism is severe. A video whose title fails to generate a sufficient CTR during the cold-start window may receive permanent suppression from the recommendation engine, regardless of the actual quality of the content it contains. High-quality educational material, well-produced creative work, and valuable tutorials are all equally susceptible to this failure mode if their metadata packaging is weak. Discovery, not quality, becomes the primary determinant of a video's performance.

CTR benchmarks on YouTube illustrate the stakes: YouTube's own documentation states that half of all channels and videos have an impressions CTR between 2% and 10% [3]. On the home feed, a CTR above 7% is considered good; above 10% is a significant

outlier typically associated with strong title-and-thumbnail combinations [3]. The difference between a 3% CTR and a 7% CTR, which appears small in absolute terms, translates into dramatically different levels of algorithmic amplification over time, as higher CTR produces more impressions, more watch time data, and higher rankings in a self-reinforcing feedback loop.

1.2.3 The expertise bottleneck

Professional copywriters and experienced content creators apply a well-documented body of heuristic knowledge when crafting high-performing video titles. This knowledge encompasses psychological principles, curiosity gap construction [4], loss aversion framing [5], social proof signals [6], and emotional arousal engineering [7], as well as platform-specific rules regarding optimal length, keyword placement, numerical triggers, and audience specificity. Applying this expertise at scale runs into real limits, though.

It is *unscalable*: a professional copywriter who can craft a high-CTR title in twenty minutes cannot meaningfully serve the volume of content being produced, and at 500 hours of video uploaded per minute, no human workforce can operate at the pace required. It is also *inaccessible* in practice: most content creators (individual educators, small businesses, independent journalists, student filmmakers) have neither training in marketing psychology nor the money to retain professional copywriting services, so the expertise stays gated behind economic privilege. And even where the expertise exists, it is *inconsistent*: experienced practitioners still produce flat, uninspired titles under time pressure, fatigue, or unfamiliarity with a particular content niche.

1.2.4 The rise of LLMs and their generic application gap

The rapid development and public availability of large language models (LLMs), including GPT-4o (OpenAI), Llama 3 (Meta), and Claude (Anthropic), has opened a potential pathway toward automated, scalable title generation. These models, built on the Transformer architecture introduced by Vaswani et al. (2017) [8], demonstrate strong capabilities in creative text generation, instruction-following, and constraint satisfaction.

However, the generic application of LLMs to title generation, prompting a model with a request such as “write me a YouTube title for this video”, does not resolve the expertise bottleneck. Research by Yang (2024) in *PLOS ONE* confirmed that standard AI-generated titles produced without specialised prompting strategies lack the emotional sophistication

and cultural specificity that human experts apply [9]. The resulting output is typically descriptive and accurate, but psychologically flat: it tells the viewer what the video is about without creating the psychological tension that motivates a click.

A large-scale randomised field experiment by Zhang et al. (2024) involving over two million content producers on a leading short-video platform provided further empirical grounding for this limitation [10]. The study found that when videos already had human-generated titles, replacing those titles with pure AI-generated alternatives *decreased* valid watch counts by 37.9%, suggesting that naive AI title generation falls below the quality threshold that experienced human creators achieve intuitively. Critically, however, the same study found that when producers significantly revised AI-generated titles through human-AI collaboration, content consumption improved, and each 10% reduction in similarity between the AI draft and the final adopted title was associated with a 9.8% increase in valid watch counts [10]. This finding points toward a hybrid, engineered approach, where AI generation is systematically directed by psychological expertise, as the productive research direction.

1.3 Problem statement

1.3.1 General problem

At the macro level, the structural reality of algorithmically-driven content discovery creates a systemic misalignment between content quality and content visibility. YouTube distributes over 500 hours of new video every minute [1], and its recommendation algorithm uses CTR, formed in milliseconds by a viewer who reads a title, as the primary early signal for distribution decisions [2]. The consequence is that video metadata, specifically the title, exerts a disproportionate influence on a video's long-term performance relative to its actual content quality.

This creates a measurable economic and social cost. Educational content that fails to achieve adequate CTR during the cold-start window does not reach students who would benefit from it. Small business product demonstrations are buried below competitors who have invested in copywriting expertise. Independent creators with genuine talent produce high-quality work that is algorithmically invisible, not because it lacks merit, but because its packaging is suboptimal. The economic dimension extends to the creator economy

itself: in a system where visibility determines monetisation, advertising revenue, and brand partnership opportunities, the ability to consistently generate effective metadata is a critical business capability that most creators lack [10].

The problem is further compounded by the reality that, as Zhang et al. (2024) documented, only 60.7% of uploaded videos on a major platform had any title at all during their study's pre-treatment period [10]. Metadata sparsity, the absence of the very signals that recommendation systems rely on to understand and distribute content, is not a marginal failure mode but a prevalent condition affecting the majority of uploaded content.

1.3.2 Specific problem

At the domain-specific level, the problem narrows to the absence of a generative AI solution that occupies the intersection of three requirements that existing systems fail to satisfy simultaneously.

Existing LLM-based tools are reactive and psychologically ungrounded: they respond to explicit user prompts but do not proactively analyse video content and generate optimised titles without user direction. More critically, they do not operationalise the psychological theories (Information Gap Theory [4], Prospect Theory [5], Social Proof [6]) that the copywriting literature identifies as the mechanisms underlying high-performing titles. The result is generic, descriptive output that fails the psychological criterion for an effective click trigger. Yang (2024) confirmed this limitation empirically, demonstrating that AI-generated titles produced without specialised methodology lack the nuance human experts apply [9].

Existing analytics platforms, meanwhile, are generative-free. Tools such as TubeBuddy and VidIQ provide rich historical performance data (keyword rankings, competitor CTR benchmarks, trend analysis), but offer no capability to generate optimised titles from video content. They are purely reactive instruments that show creators what has already happened, without generating what should happen next.

And no existing tool enforces ethical faithfulness constraints alongside engagement optimisation. Clickbait detection systems (SemEval shared tasks, ClickGuard, the Webis Clickbait Corpus tools) identify deceptive headlines but provide no constructive alternative, and marketing-oriented AI tools optimise for clicks without constraining against factual misrepresentation. Unconstrained optimisation for engagement produces

misleading titles, a pattern that research has shown leads to high initial CTR but devastated watch time, algorithmic penalisation, and potential violations of YouTube's Terms of Service (Section 4H), which prohibit metadata that "sets up expectations the video doesn't deliver" [11].

Furthermore, research published in *Scientific Reports* (2024) demonstrated that when curiosity-gap headline techniques became ubiquitous on platforms such as Facebook and Upworthy circa 2014, platform algorithms were adjusted to penalise them, illustrating the long-term algorithmic risk of pure engagement optimisation without truthfulness constraints [12].

The research gap is therefore the absence of a system that is simultaneously: **(1) proactive**, generating titles from content analysis without user prompt engineering; **(2) psychologically grounded**, operationalising established psychological trigger theory as explicit algorithmic instructions; **(3) generative**, producing novel, diverse title candidates rather than analysing existing ones; and **(4) ethically constrained**, enforcing semantic faithfulness to the source video as a hard system requirement. No existing academic publication or commercial tool occupies this intersection.

1.4 Research question

1.4.1 Primary research question

> **How can a multi-agent AI system be designed and evaluated to automatically generate YouTube video titles that are psychologically optimised for high Click-Through Rates while maintaining semantic faithfulness to the source video content?**

1.4.2 Supporting sub-questions

RQ1: How can specific linguistic patterns associated with virality, including information gap structures, high-arousal sentiment, loss aversion framing, and numerical triggers, be systematically operationalised into Few-Shot and Chain-of-Thought prompt templates for Large Language Models?

RQ2: What is the optimal software architecture for an API Wrapper that integrates LLM-based title generation with a computational validation layer to ensure generated titles are both engagement-optimised and semantically faithful to the source content?

RQ3: To what extent do LLM-generated titles produced using psychologically-engineered prompt strategies outperform baseline human-generated titles and generic zero-shot AI titles, when evaluated against predicted virality metrics and human preference ratings?

RQ4: How can the LLM-as-a-Judge evaluation pattern be employed to automatically score and rank generated titles against a structured marketing rubric encompassing Curiosity, Urgency, and Relevance dimensions?

1.5 Research motivation

The motivation for this research operates across three distinct dimensions: economic and social, technical, and ethical.

1.5.1 Economic and social motivation

The creator economy has grown into a multi-billion dollar industry in which individual content creators build audiences, generate revenue, and communicate expertise through video platforms. However, the skills required to succeed in this economy, specifically, the ability to write compelling, psychologically informed video metadata, are not evenly distributed. Professional marketing expertise is expensive and inaccessible to the vast majority of independent creators, educators, and small organisations. The consequence is a structural inequality in which creators with financial resources to invest in copywriting expertise enjoy algorithmic advantages over creators who produce equally or more valuable content.

Beyond commercial creators, the problem extends to educational and social impact contexts. A university professor uploading lecture content, a non-profit publishing health information, or a language teacher creating free tutorial videos all face the same discovery bottleneck. If the title does not generate an adequate CTR during the cold-start window, the content is effectively invisible to the audience it was designed to serve. Zhang et al. (2024) demonstrated that AI-assisted metadata significantly increased valid watch

counts for low-skilled producers (+1.6% at access stage, +7.1% when adopted), confirming the democratising potential of engineered AI title generation [10].

This research is motivated by the opportunity to democratise access to high-quality title optimisation, placing capabilities previously available only to well-resourced creators within reach of any creator with a video transcript.

1.5.2 Technical motivation

From a technical standpoint, this research addresses an open and practically important question in generative AI: whether LLMs can be systematically *steered* to optimise a specific, measurable real-world metric (predicted CTR) through engineered prompt infrastructure, rather than through weight-level training.

The distinction between zero-shot and few-shot prompting (Brown et al., 2020 [13]) and the emergence of Chain-of-Thought reasoning (Wei et al., 2022 [14]) as prompting paradigms have demonstrated that LLM behaviour is substantially shaped by how they are prompted, not only by their weights. Yet the application of these techniques to a domain as specific and psychologically complex as viral title generation remains understudied. This project contributes a novel instantiation of these techniques, a Prompt Factory that constructs psychologically-engineered, trigger-specific prompts dynamically, as a technical artefact whose performance can be measured and compared.

Furthermore, the multi-agent architecture, Analyzer -> Generator -> Critic, addresses a known limitation of single-shot generation: the conflation of content analysis, creative generation, and quality evaluation in a single model call. By separating these concerns across isolated agents, the system enforces architectural boundaries that constrain hallucination by design: the Generator reads only the Analyzer's structured brief, never the raw transcript, preventing the model from fabricating content that is not grounded in the video's actual material.

1.5.3 Ethical motivation

The proliferation of clickbait, titles that systematically misrepresent or exaggerate video content to maximise initial clicks at the expense of viewer satisfaction, is a documented harm to the information ecosystem. Research has shown that curiosity-gap techniques, when overused and deployed deceptively, prompted algorithm-level countermeasures from major platforms and eroded user trust [12]. YouTube's own Terms of Service

(Section 4H) explicitly prohibit metadata that creates undelivered expectations [11], and violations can result in demonetisation and channel strikes.

This research is motivated by the conviction that engaging and honest content packaging are not mutually exclusive objectives. The NLI-based faithfulness layer in this system puts that conviction into code: every generated title is automatically evaluated against the video's core promise, and any title that factually contradicts the source content is hard-rejected before it can be surfaced to the creator. This ethical constraint is a first-class architectural component, designed in from the start rather than bolted on afterward.

1.6 Research aim

> To design, develop, and evaluate a multi-agent AI system implemented as an API Wrapper that automatically generates psychologically optimised YouTube video titles, demonstrating that generative AI can be systematically constrained to maximise predicted Click-Through Rate while preserving semantic faithfulness to the source video content.

1.7 Rich picture of the proposed solution

The proposed system, the YouTube Title Optimizer (YTO), operates as a multi-agent API Wrapper. The figure below illustrates the high-level information flow from input to output.

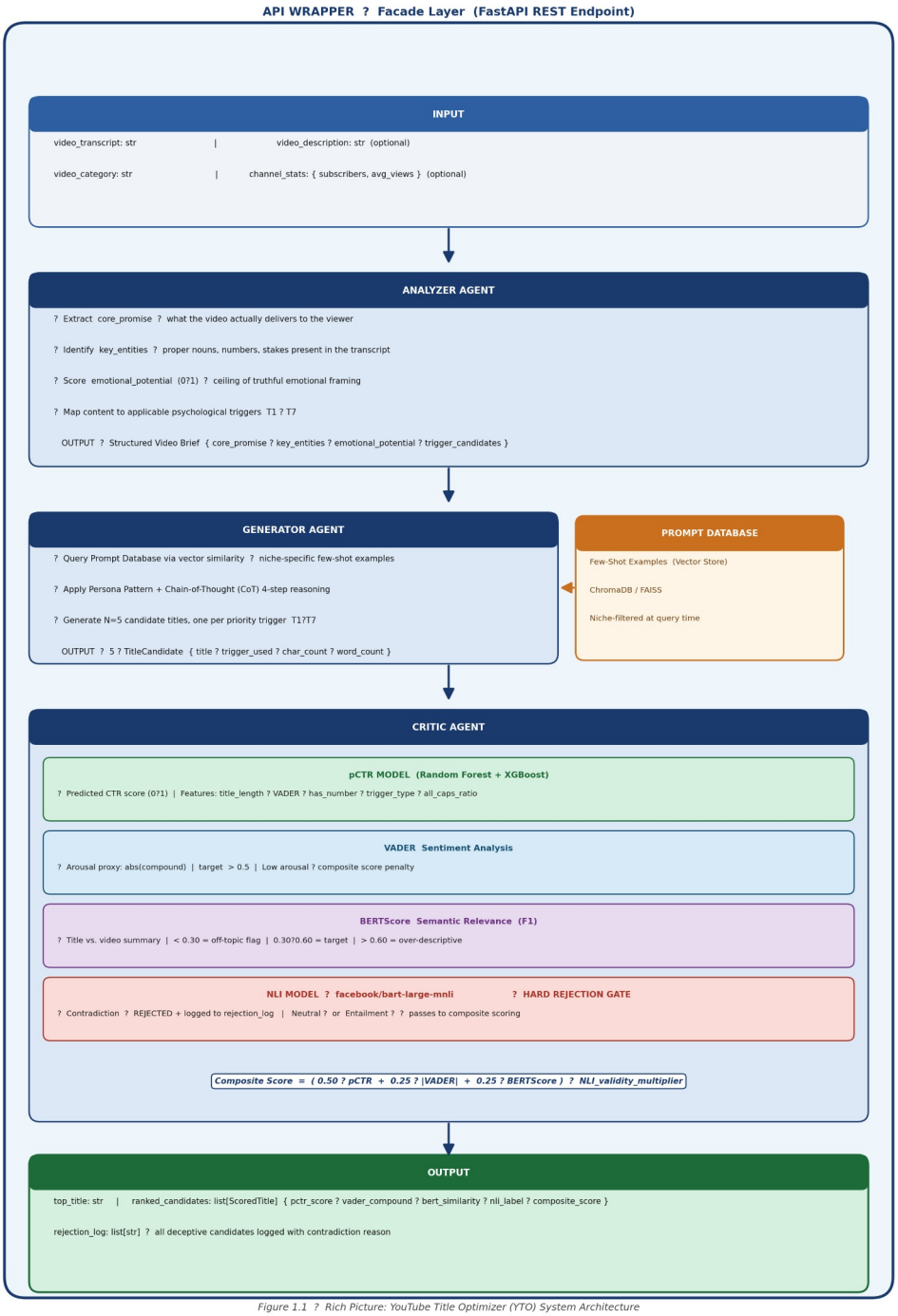


Figure 1.1 ? Rich Picture: YouTube Title Optimizer (YTO) System Architecture

Figure 1.1: Rich Picture: YouTube Title Optimizer (YTO) System Architecture

Figure 1.1: Rich Picture: YouTube Title Optimizer (YTO) System Architecture

The API Wrapper functions as a Facade: the consuming application (a creator's workflow tool, a browser extension, or a content management system) submits a simple HTTP request containing the video transcript and optional metadata, and receives a ranked list of psychologically optimised title candidates with scores. All internal complexity (agent

orchestration, prompt construction, ML model inference, NLP validation) is hidden behind this single interface.

The Analyzer Agent extracts the semantic ground truth of the video: what it actually delivers, what entities are real and usable, what emotional framing is legitimately supportable. The Generator Agent receives only this structured brief, never the raw transcript, which is the primary architectural defence against hallucination. The Critic Agent applies four independent scoring mechanisms and enforces the hard rejection gate: any title classified as "Contradiction" to the video's core promise by the NLI model is eliminated from the output, regardless of its predicted CTR score.

1.8 Resource requirements

1.8.1 Hardware

The following hardware resources are required to implement and evaluate the proposed system:

Table 1.1: Hardware resource requirements

Resource	Minimum Specification	Purpose
Development Machine	16 GB RAM, 8-core CPU, 64-bit OS	LLM API calls, ML model training, agent pipeline execution
GPU (Recommended)	NVIDIA GPU, 8 GB+ VRAM (e.g., RTX 3060)	Accelerated NLI model inference, BERTScore computation
Storage	50 GB+ SSD	Kaggle dataset storage, model weight caching, embeddings
Internet Connectivity	Stable broadband (10 Mbps+)	OpenAI API, Hugging Face model downloads, YouTube Data API

GPU access is recommended but not strictly required; CPU-based inference for the NLI and BERTScore models is functional at reduced speed, and is acceptable within the evaluation latency budget (P95 < 10 seconds, end-to-end). Where GPU access is unavailable, Hugging Face's Inference API or Google Colab can substitute for model inference during development.

1.8.2 Software

Table 1.2: Software resource requirements

Software / Library	Version	Purpose
Python	3.10+	Core implementation language
FastAPI + Uvicorn	Latest stable	ASGI REST API backend; async LLM call handling
LangChain + LangGraph	Latest stable	Multi-agent orchestration; Supervisor pattern; state management
OpenAI Python SDK	Latest stable	GPT-4o API access for Generator and Analyzer agents
Hugging Face Transformers	Latest stable	facebook/bart-large-mnli (NLI), BERTScore inference
bert-score (Python)	Latest stable	BERTScore computation library
NLTK + vaderSentiment	Latest stable	VADER compound score for Critic agent
Scikit-learn	Latest stable	Random Forest pCTR model, feature preprocessing
XGBoost	Latest stable	XGBoost pCTR model; hyperparameter tuning
Sentence-Transformers	Latest stable	Prompt Database vector embeddings for few-shot retrieval
Kaggle CLI / API	Latest stable	Dataset download automation
GitHub	Free	Version control; CI/CD
Python dotenv	Latest stable	API key and environment management

All dependencies are open-source or available via free-tier API access. The OpenAI API incurs per-token inference costs; the GPT-4o pricing as of the project's evaluation period is documented and version-pinned to ensure reproducibility [15].

1.9 Project scope

1.9.1 In scope

Table 1.3: In-scope items

Element	Specification
Primary deliverable	Multi-agent AI system implemented as a RESTful API Wrapper
Title generation target	YouTube video titles only
Language	English-language titles and input transcripts
Input modality	Text-based input: video transcript, description, or summary
Engagement metric	Predicted CTR (pCTR) as the primary optimisation objective
Faithfulness enforcement	NLI-based hard rejection gate (Contradiction -> eliminated)
ML model	pCTR classifier trained on public Kaggle YouTube datasets
Agent pipeline	Analyzer Agent -> Generator Agent -> Critic Agent
Evaluation	Quantitative (pCTR, BERTScore, VADER, Self-BLEU, NLI acceptance rate) + qualitative (human survey, LLM-as-a-Judge)
Comparison baselines	Baseline A: human-generated titles; Baseline B: zero-shot LLM titles
Backend framework	FastAPI (Python)
Agent orchestration	LangChain + LangGraph (Supervisor pattern)

1.9.2 Out of scope

Table 1.4: Out-of-scope items

Element	Rationale for Exclusion
Thumbnail generation or optimisation	Requires computer vision/multimodal generation pipeline; constitutes a separate research project
Non-English or multilingual titles	Language-specific trigger lexicons and cultural adaptation require per-language validation; excluded to maintain evaluable scope
Direct video file processing (audio/visual)	System accepts text-only input; video

Element	Rationale for Exclusion
	transcription is a pre-processing step outside system boundaries
Live CTR measurement from real channels	Requires YouTube Studio API access with creator consent and a longitudinal study design incompatible with the project timeline
Real-time YouTube Studio integration	Constitutes a production deployment feature; excluded from the research prototype
Proprietary CTR data collection	Public datasets and normalised proxy metrics (Views/Subscriber_Count) are used instead
Frontend user interface or web application	System delivers a REST API; UI is a future work item
Production-ready commercial deployment	The artefact is a research prototype evaluated in a controlled academic setting
Large-scale live A/B testing on creator channels	Outside ethical scope and timeline constraints; evaluation uses offline metrics and a human preference survey

The system is scoped as a **research prototype** whose primary contribution is the demonstration that psychologically-engineered, ethics-constrained, proactive title generation is feasible and measurably superior to baseline alternatives. Commercial productisation is acknowledged as a natural extension but is explicitly excluded from this study's claims.

1.10 Chapter summary

This chapter has established the context, problem, and research direction for the YouTube Title Optimizer project. The video-first internet has created a structural discovery problem in which the metadata attached to a video, principally its title, functions as the primary algorithmic gatekeeper, determining whether high-quality content reaches its intended audience. YouTube's recommendation system, as documented by Covington et al. (2016) [2], uses Click-Through Rate during a cold-start test window as an early and influential distribution signal, creating an asymmetric penalty for poor titles that quality content cannot overcome after the fact.

The skill gap between professional copywriting expertise and the capabilities of most individual creators, combined with the failure of generic LLM prompting to replicate that

expertise [9], defines the specific technical problem. No existing system simultaneously provides proactive title generation, psychologically-grounded prompt engineering, and semantic faithfulness enforcement, a gap confirmed by the existing systems analysis in Chapter 3.

The research aim is to design, develop, and evaluate a multi-agent AI system that bridges this gap. Four specific objectives guide the work: identifying viral linguistic features in YouTube datasets; analysing the effectiveness of Few-Shot and Chain-of-Thought prompt techniques; designing and developing the three-agent pipeline as an API Wrapper; and evaluating the system against quantitative and qualitative baselines. The scope is deliberately bounded to English-language, text-input title generation for YouTube, with real-world CTR measurement, thumbnail optimisation, and multilingual support deferred to future work.

Chapter 3 proceeds to review the relevant literature across the domains of engagement psychology, generative AI and prompt engineering, NLP evaluation techniques, and the existing tools landscape, establishing the theoretical and empirical foundations on which the system design is built.

2 OBJECTIVES

2.1 To identify

To identify the viral linguistic features, including sentiment polarity, emotional arousal level, information gap structures, numerical triggers, and psychological trigger types (T1: Curiosity Gap, T2: Loss Aversion, T3: Social Proof, T4: Urgency, T5: Extreme/Superlative, T6: Personal Stakes, T7: Specificity), that correlate with high view velocity in YouTube trending video datasets, using feature engineering on publicly available Kaggle datasets supplemented by targeted YouTube Data API v3 scraping.

2.2 To analyse

To analyse the effectiveness of advanced prompt engineering techniques, specifically Few-Shot Prompting [13], Chain-of-Thought (CoT) Reasoning [14], and the Persona Pattern, in operationalising psychological theories (Information Gap Theory [4], Prospect Theory [5], Social Proof [6]) within a Large Language Model generation pipeline, by comparing the structured prompting outputs against zero-shot baseline outputs on the same input content.

2.3 To design and develop

To design and develop a multi-agent AI system comprising an Analyzer Agent (content analysis and ground-truth extraction), a Generator Agent (psychologically-engineered title production using a dynamic Prompt Database), and a Critic Agent (scoring and validation), encapsulated as a RESTful API Wrapper using the Facade design pattern, with an integrated Predicted CTR (pCTR) machine learning model and NLP validation layer (VADER sentiment analysis, BERTScore semantic similarity, and NLI-based factual faithfulness checking).

2.4 To evaluate

To evaluate the performance of the designed system using quantitative metrics (pCTR score, BERTScore, VADER arousal magnitude, Self-BLEU diversity, and NLI acceptance rate) and qualitative human evaluation (a preference survey with a minimum of 50 participants using a four-dimension rubric scoring Curiosity, Emotional Impact, Clarity, and Trustworthiness), benchmarked against a human-generated title baseline

(Baseline A) and a zero-shot AI title baseline (Baseline B), including a secondary automated evaluation using the LLM-as-a-Judge pattern to address RQ4.

3 LITERATURE REVIEW

3.1 Chapter overview

This chapter situates the YouTube Title Optimizer (YTO) within the existing body of academic and applied knowledge across three intersecting domains: user engagement psychology, generative AI and prompt engineering, and NLP evaluation methods. It additionally surveys the landscape of existing systems (both tools and research prototypes) that address some subset of the problem, and identifies through systematic comparison the multi-dimensional gap that this research fills.

The review is organised according to a coverage structure prescribed by the project template: a domain overview (approximately 10% of coverage) establishes the macro-context; an existing systems analysis (approximately 30%) critically examines and compares prior approaches; and a technological analysis (approximately 60%) provides an algorithmic, design, and workflow-level review of the techniques, architectures, and datasets on which the proposed system is built. The chapter closes with a reflection that traces the research gap back through the literature, providing a grounded justification for the research question stated in Chapter 1.

3.2 Conceptual map of the literature

The literature that informs this research spans four distinct knowledge domains. Table 3.1 maps the primary theoretical and technical pillars to the system components they inform, and to the research questions (RQ1–RQ4) they address.

Table 3.1: Conceptual Map of the Literature

Domain	Key Works	System Component	RQ Addressed
Engagement Psychology	Loewenstein (1994) [1], Golman & Loewenstein (2016) [2], Kahneman & Tversky (1979) [3], Cialdini (1984) [4], Berger & Milkman (2012) [5]	Psychological Trigger Taxonomy (T1–T7); Generator Agent prompt templates	RQ1, RQ3
YouTube	Covington et	System motivation;	RQ3

Domain	Key Works	System Component	RQ Addressed
Algorithm & CTR	al. (2016) [6], Kawata et al. (2025) [7], Zhang et al. (2024) [8]	pCTR target variable; dataset strategy	
Generative AI & Prompt Engineering	Vaswani et al. (2017) [9], Brown et al. (2020) [10], Wei et al. (2022) [11], Yang (2024) [12]	Generator Agent; Prompt Factory; Few-Shot Prompt Database	RQ1, RQ2
NLP Evaluation Methods	Hutto & Gilbert (2014) [13], Zhang et al. (2020) [14], Laban et al. (2023) [15], Aslam et al. (2021) [16]	Critic Agent; VADER, BERTScore, NLI, pCTR model	RQ2, RQ4
Research Methodology	Peffers et al. (2007) [17]	DSR strategy; DSRM six-phase process	:

The three columns (psychological theory, AI/NLP technology, and existing systems) represent the three pillars that must be integrated. No prior work has performed this integration in the specific combination that YTO proposes. That is the gap this chapter establishes.

3.3 Domain overview

3.3.1 The video-first internet and the creator economy

The consumption of digital content has undergone a structural reorientation over the past decade. Where textual media (articles, blog posts, and web pages) once constituted the primary vehicle for online information exchange, video has emerged as the dominant format through which billions of people learn, decide, and discover. YouTube, processing over 500 hours of new video content every minute [19], is the largest video repository in existence and the world's second-largest search engine by query volume. Alongside it, short-form platforms including TikTok and Instagram Reels have further accelerated the shift toward video as the default communication medium.

This transformation has given rise to a significant creator economy. Individual content creators, educators, journalists, and organisations now depend on video platforms as primary channels for audience reach, brand building, and monetisation. In this economy, the ability to generate video content is no longer the limiting factor: production costs have collapsed with the democratisation of recording hardware and editing software. The limiting factor is *discovery*: the ability to surface content to relevant audiences in an environment of extraordinary supply.

3.3.2 Click-through rate as the algorithmic gatekeeper

YouTube's recommendation system, as documented by Covington, Adams, and Sargin (2016) in the foundational paper *Deep Neural Networks for YouTube Recommendations* [6], operates as a two-stage pipeline. The candidate generation stage narrows billions of videos to hundreds of candidates for each user session. The ranking stage orders those candidates according to predicted user satisfaction signals, of which Click-Through Rate (CTR), the proportion of users who click a video after seeing its impression, is among the most immediate and influential.

For newly uploaded videos, the system faces a cold-start problem: no historical engagement data exists, and the recommendation engine must rely on the earliest available signal to decide whether to expand distribution. That signal is the click decision made in milliseconds by a viewer who reads a title and sees a thumbnail. If CTR during the initial test cohort is insufficient, the algorithm contracts distribution regardless of the video's intrinsic quality. YouTube's own documentation confirms that half of all channels and videos have impressions CTRs between 2% and 10% [20], and industry benchmarks indicate that CTRs above 7% on the home feed are a strong outcome associated with meaningful algorithmic amplification [21].

This dynamic creates a disproportionate influence for video titles relative to every other variable in content production. A well-crafted title that achieves a 7% CTR versus a poorly crafted title achieving 3% can produce dramatically different long-term distribution outcomes, not because the content differs, but because the metadata packaging does.

3.3.3 Historical evolution: from manual copywriting to AI-assisted generation

The craft of writing compelling headlines predates the internet. In traditional print and broadcast media, professional copywriters applied empirically developed heuristics to generate click, read, or view responses. The psychological mechanisms these professionals exploited intuitively (curiosity gaps [1], urgency framing [4], social proof signals [4], emotional arousal [5]) were formalised in academic literature beginning in the 1970s and 1980s.

The digital era introduced new layers of optimisation. Search engine optimisation (SEO) research established rules for keyword placement and title length to maximise search ranking visibility. The rise of social media platforms shifted focus toward emotional triggers and shareability. YouTube-specific tools such as TubeBuddy and VidIQ emerged to provide creators with data on which titles performed historically, without providing generative capability. Most recently, the public availability of large language models (LLMs) beginning with GPT-3 (Brown et al., 2020 [10]) opened the possibility of AI-assisted title generation, though, as discussed in Section 3.4, the unstructured application of these models does not automatically produce high-quality results [12].

3.4 Existing systems, frameworks, and designs

This section critically examines four categories of existing approaches to the title optimisation problem, followed by a comparative analysis and identification of the research gap.

3.4.1 System 1: manual expert copywriting

Description: Professional copywriters with expertise in marketing psychology apply their knowledge of psychological triggers, platform-specific conventions, and audience psychology to craft titles manually. This is the highest quality baseline in the existing landscape.

Strengths: Manual expert copywriting produces titles grounded in nuanced understanding of cultural context, brand voice, and audience psychology. An experienced practitioner can apply information gap theory [1], loss aversion framing [3], and social proof construction [4] intuitively, adjusting for niche, audience, and content type. The approach has been validated by the measurable performance of channels whose creators

invest in this expertise: MrBeast's publicly documented title philosophy, for instance, explicitly prioritises emotional intensity, brevity, and comprehensibility over descriptive accuracy.

Weaknesses: The approach runs into the same three structural limitations noted in Section 1.2. It is *unscalable*: at 500 hours of video uploaded per minute [19], no human workforce can apply expert-level review at the required throughput. It is *inaccessible*, since professional copywriting services are expensive and create an economic barrier that disadvantages individual creators, non-profit educators, and small organisations. And it is *inconsistent*: even expert practitioners produce suboptimal titles under time pressure or outside their domain of familiarity.

Reflection: Manual expert copywriting establishes the quality ceiling that an automated system must aspire to approach. Its unscalability and inaccessibility are precisely the structural problems this research addresses. The question is whether systematic prompt engineering can transfer the heuristic knowledge of expert copywriters into an algorithmic pipeline that is accessible to any creator with a video transcript.

3.4.2 System 2: YouTube analytics platforms (TubeBuddy, VidIQ)

Description: Dedicated YouTube optimisation tools such as TubeBuddy and VidIQ provide creators with data dashboards offering historical CTR analysis, keyword suggestion engines, competitor benchmarking, trend identification, and tag recommendation features.

Strengths: These platforms offer genuine empirical value. Creators with access to historical performance data can identify which topics and formats have generated high CTR in their niche, and keyword tools can surface search demand signals that inform title structure. The platforms serve a large, active user base and are integrated into YouTube Studio workflows, lowering the barrier to data-informed decision-making.

Weaknesses: Both platforms are fundamentally *reactive and generative-free*. They show creators what has already happened (historical CTR trends, past keyword performance, competitor data), but they do not generate titles from video content. The creator remains responsible for synthesising analytical insights into a new title, which requires the same expert judgment that most creators lack. Neither platform operationalises psychological

trigger theory, applies sentiment analysis, or enforces faithfulness constraints. They answer the question “what worked before?” without answering “what should I write now?”

Reflection: TubeBuddy and VidIQ are valuable data tools but do not address the generative component of the problem. Their existence confirms market demand for title optimisation assistance, but their purely analytical orientation leaves the synthesis gap, the jump from insight to actual title copy, entirely to the creator.

3.4.3 System 3: generic LLM prompting (zero-shot)

Description: The public availability of capable LLMs, including GPT-4o (OpenAI), Claude (Anthropic), and Llama 3 (Meta), has made it possible for creators to prompt these models directly for title suggestions: “Write me a YouTube title for a video about cooking pasta.”

Strengths: LLM-based title generation is highly scalable, requires no technical expertise from the user, is available at low or zero marginal cost, and can produce grammatically fluent, topically accurate suggestions within seconds.

Weaknesses: The evidence that zero-shot LLM prompting fails to match expert copywriting quality is well-documented. Yang (2024), in a study published in *PLOS ONE*, evaluated AI-assisted title generation on the RED social media platform and found that standard machine-generated titles “lack the emotional sophistication and cultural proficiency” that human experts apply, producing output that is accurate but psychologically flat [12]. The study, which tested the “POP Title AI Five-Step Optimization Method,” found that human refinement of AI-generated drafts was necessary to achieve performance gains.

More directly, Zhang et al. (2024), in a large-scale randomised field experiment involving over two million content producers, found that when videos already had human-generated titles, replacing them with pure AI-generated alternatives *decreased* valid watch counts by 37.9%, a finding that strongly suggests naive AI generation falls below the quality threshold that experienced human creators achieve [8].

The root cause of this limitation is architectural: a zero-shot prompt provides the model with no psychological framework, no niche-specific exemplars, no trigger

operationalisation, and no quality evaluation mechanism. The model generates based on its training distribution, which produces descriptive competence but not engagement engineering.

Reflection: Generic LLM prompting does not solve the problem; it reframes it. The question is no longer whether LLMs *can* generate titles, but whether they can be *systematically directed* to generate titles that operationalise psychological trigger theory, are constrained against hallucination, and are ranked by a predicted virality metric. This is precisely the contribution of the proposed system.

3.4.4 System 4: clickbait detection systems

Description: A substantial body of academic research addresses the detection and classification of clickbait. The Webis Clickbait Corpus (Potthast et al., 2016) provides annotated datasets for training detection models. SemEval shared tasks have produced multiple high-performing NLP classifiers. Research published in 2021 demonstrated the application of multi-modal ML to detect clickbait specifically in YouTube videos using title, description, and thumbnail features [22].

Strengths: Clickbait detection research has produced robust, well-validated NLP methods, rich annotated datasets, and a clear theoretical framing of what constitutes deceptive metadata. The NLI-based approach to hallucination detection that has emerged from summarisation faithfulness research (Laban et al., 2023 [15]) traces its lineage directly to this body of work.

Weaknesses: Detection systems solve the inverse of the generative problem. They identify what bad titles look like without providing a pathway to what good titles look like. They are binary (clickbait / not clickbait) or multi-class (degree of clickbait), but they do not generate alternatives, do not score engagement potential, and do not assist creators in producing improved metadata. They are diagnostic instruments without therapeutic capability.

Reflection: The clickbait detection literature is directly useful to this project as the source of negative examples (the Webis Clickbait Corpus serves as negative training data for the pCTR model's validation layer), and as the origin of the NLI faithfulness methodology

adopted in the Critic Agent. However, the detection systems themselves do not occupy the generative space that YTO targets.

3.4.5 Comparative summary

Table 3.2: Comparative Analysis of Existing Approaches

Criterion	Manual Copywriting	Analytics Platforms	Generic LLM	Clickbait Detection	**YTO (Proposed)**
Proactive (auto-generates from content)	No	No	Partial	No	**Yes**
Psychologically grounded (trigger theory)	Yes	No	No	Partial	**Yes**
Generative capability	Yes	No	Yes	No	**Yes**
Faithfulness / deception enforcement	Yes (human)	N/A	No	Yes (detect only)	**Yes**
Scalable	No	Yes	Yes	Yes	**Yes**
Virality scoring (pCTR)	No	Partial	No	No	**Yes**
Accessible (no expertise required)	No	Partial	Yes	Yes	**Yes**

The comparative analysis reveals a consistent pattern: each existing approach satisfies some criteria while failing others. No existing system simultaneously provides proactive, psychologically-grounded, generative, and faithfulness-enforced title optimisation at scale. This intersection is the research gap that YTO addresses.

3.5 Technological analysis

3.5.1 Algorithmic analysis

3.5.1.1 A1: Large language models, the transformer architecture

The architectural foundation of all major LLMs is the Transformer, introduced by Vaswani et al. (2017) in *Attention Is All You Need* [9]. The Transformer replaced recurrent architectures (RNN, LSTM) with a self-attention mechanism that computes relationships between all tokens in a sequence simultaneously, enabling parallel training and capturing long-range dependencies that sequential models could not efficiently model. This architecture underpins every model considered for use in YTO: GPT-4o, Llama 3, and the encoder-based models (BERT, RoBERTa) used in the evaluation layer.

GPT-4o (OpenAI, 2024) is selected as the primary generation model on the basis of its state-of-the-art instruction-following capability, native support for structured JSON output (critical for agent pipelines where downstream components consume structured data), and strong performance on creative generation tasks with constraint satisfaction requirements [23]. It supports fine-tuning, offers a native function-calling interface, and provides a documented pricing structure that enables cost modelling within the research prototype scope.

Llama 3 (Meta AI, 2024) is included as the open-source alternative. Its deployability on local hardware without per-token API costs makes it attractive for production deployment scenarios, and its support for fine-tuning on domain-specific data (high-CTR YouTube titles) positions it as the preferred model for the planned Phase 2 fine-tuning extension. The existence of a capable open-source alternative also mitigates vendor lock-in risk.

The core limitation of raw LLMs for title generation is that their output distribution reflects their training data (broad, general-purpose text) rather than the narrow, psychologically engineered register of high-performing YouTube titles. Bridging this gap is the function of the prompt engineering techniques reviewed in A2.

3.5.1.2 A2: Prompt engineering techniques

Few-Shot Prompting. Introduced by Brown et al. (2020) in *Language Models are Few-Shot Learners* [10], few-shot prompting involves providing a small number of curated input-output examples (typically 3–8) within the prompt context to steer the model's

output toward the desired format, register, and quality standard. Crucially, this technique requires *no updates to model weights*: the steering occurs entirely through the context window. Brown et al. demonstrated that scaled LLMs can approach fine-tuned model performance on many NLP benchmarks using only a handful of in-context examples, establishing in-context learning as a first-class capability distinct from gradient-based fine-tuning.

In the YTO Generator Agent, few-shot prompting is implemented through a dynamic Prompt Database: high-performing YouTube titles are embedded as vectors and stored in a vector store. At generation time, the video brief is embedded and a nearest-neighbour similarity search retrieves the 3–5 topically closest high-performing titles in the same content niche. These titles are injected into the Generator's prompt as examples, anchoring the model's output to the specific linguistic register, emotional register, and trigger structure that has historically performed well in that niche. This niche-specific retrieval is critical: a cooking video should receive cooking-niche exemplars, not the extreme-challenge format titles that characterise MrBeast's channel.

Chain-of-Thought (CoT) Reasoning. Wei et al. (2022), in *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models* [11], demonstrated that instructing LLMs to articulate intermediate reasoning steps before producing a final answer significantly improves performance on complex multi-step tasks. The technique causes the model to decompose the problem into explicit sub-problems: identifying the core tension, selecting the optimal emotional framing, drafting candidates, and refining for constraint satisfaction, rather than mapping input directly to output in a single step.

In the YTO Generator Agent, a four-step CoT sequence is applied: (1) identify the core information gap or conflict in the video brief; (2) determine the optimal high-arousal emotional angle authentically supported by the content; (3) draft 10 candidate titles across different psychological triggers; (4) refine the three strongest candidates for brevity, mobile display constraints, and the target character count. This decomposition prevents the Generator from short-circuiting to a generic answer and enforces deliberate trigger-type selection.

Persona Pattern. Assigning the LLM an expert role prior to the generation task, "You are a viral YouTube title copywriter with 10+ years of experience optimising CTR for top creators", has been demonstrated to shift the model's prior toward domain-specific

vocabulary, heuristics, and tone, producing outputs that more closely resemble professional practitioner work than instruction-following alone. White et al. (2023) systematised this and related techniques in the Prompt Pattern Catalog, establishing the Persona Pattern as a reproducible prompting strategy that modifies the LLM's assumed role and therefore its output distribution [34]. The persona pattern is applied in the Generator Agent's system prompt and functions as a baseline distributional alignment before few-shot examples and CoT instructions are applied.

Dynamic Prompt Database via Vector Retrieval. The integration of semantic vector search with few-shot prompting enables context-aware example selection at runtime. The Prompt Database indexes high-performing titles as dense vector embeddings using Sentence-Transformers. At generation time, the incoming video brief is embedded in the same semantic space, and a nearest-neighbour search retrieves the most topically relevant examples. A content category filter (Education, Entertainment, Technology) is applied to further constrain retrieval to niche-appropriate exemplars. This architecture ensures that the few-shot context is always maximally relevant to the current generation task, a significant improvement over static, hardcoded example sets.

3.5.1.3 A3: Sentiment analysis, VADER

Hutto and Gilbert (2014) introduced VADER (Valence Aware Dictionary and sEntiment Reasoner) in *A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text* [13], published at the AAAI International Conference on Weblogs and Social Media (ICWSM). The model was designed specifically for the characteristics of short, informal social media text, the precise domain of YouTube titles.

VADER combines a manually curated sentiment lexicon (approximately 7,500 items) with five heuristic rules that capture linguistic features distinctive to informal digital text: capitalisation effects ("GREAT" > "great"), punctuation intensity ("great!!!" > "great"), negation handling, degree modifiers, and contrastive conjunction effects. It returns four scores: positive, negative, neutral (normalised proportions summing to 1.0) and compound (a single normalised summary score ranging from −1 to +1). VADER was benchmarked against eleven standard methods including LIWC, ANEW, SentiWordNet, and a Human Rater baseline, outperforming all on social media corpora.

In the YTO Critic Agent, the absolute value of the compound score (|compound|) is used as a proxy for *emotional arousal magnitude*, independent of valence direction. This

decision is grounded in Berger and Milkman's (2012) finding [5] that arousal, not valence, is the primary driver of viral spread: high-arousal positive content (awe, excitement) and high-arousal negative content (anger, anxiety) both outperform low-arousal content regardless of whether they are positive or negative. A target of $|\text{compound}| > 0.5$ is applied, penalising emotionally flat titles that may be descriptively accurate but psychologically inert.

A key limitation of VADER is that it does not model arousal *type*: a title evoking awe and a title evoking anxiety may receive identical compound scores despite their different psychological mechanisms and viral performance profiles. Berger and Milkman (2012) identified awe as the single highest-performing emotional trigger for viral spread and anger as the second-highest [5]. VADER cannot distinguish between these; it measures intensity, not category. This limitation is acknowledged and documented in Section 3.5.3 (Workflow Analysis, Evaluation) and in Chapter 8 (Limitations) as a direction for future work via a fine-tuned arousal classifier.

3.5.1.4 A4: Semantic faithfulness, BERTScore

Zhang et al. (2020) introduced BERTScore in *BERTScore: Evaluating Text Generation with BERT* [14], published at ICLR 2020. Unlike traditional text similarity metrics such as BLEU or ROUGE, which compute n-gram overlap between candidate and reference texts, BERTScore computes token-level similarity using contextual BERT embeddings. For each token in the candidate text, BERTScore finds the maximum cosine similarity against all tokens in the reference text, aggregating these scores into precision, recall, and F1 measures. The result is a metric that captures semantic similarity across paraphrase and stylistic variation, a property that n-gram metrics cannot provide and that is essential when the candidate (a creative title) is stylistically very different from the reference (a factual video summary).

Zhang et al. demonstrated that BERTScore correlates more strongly with human quality judgements than BLEU, METEOR, and TER across 363 machine translation and image captioning systems.

In the YTO Critic Agent, BERTScore (specifically, the F1 score between the generated title and the video summary extracted by the Analyzer Agent) is used as a **semantic relevance filter**. A BERTScore below 0.30 indicates that the title is likely off-topic, semantically distant from the content the video actually delivers, and is flagged for

rejection or manual review. Scores in the range 0.30–0.60 represent the expected zone for creative, psychologically optimised titles that are related to but not verbatim repetitions of the content. Scores above 0.60 indicate that the title is so closely paraphrastic of the summary that it is likely to be descriptively accurate but psychologically flat.

It is critical to note that BERTScore measures semantic *distance*, not factual *consistency*. A title can be semantically similar to a video summary while still making a factual claim that contradicts the content. This is why BERTScore is used as Layer 1 (relevance filter) rather than as the faithfulness gate, which is handled by the NLI model in Layer 2.

3.5.1.5 A5: Natural language inference for faithfulness verification

Natural Language Inference (NLI) is the task of classifying the logical relationship between two texts, a *premise* and a *hypothesis*, as one of three categories: *Entailment* (the premise logically implies the hypothesis), *Neutral* (the premise is consistent with but does not entail the hypothesis), or *Contradiction* (the premise is inconsistent with or negates the hypothesis).

The application of NLI models to faithfulness evaluation in text generation tasks has been validated by Laban et al. (2023) in *With a Little Push, NLI Models can Robustly and Efficiently Predict Faithfulness* [15], published at ACL 2023. Laban et al. demonstrated that NLI-based faithfulness scoring correlates more strongly with human faithfulness judgements than token-overlap metrics and, with appropriate calibration, can robustly identify generated text that contradicts source documents.

In the YTO Critic Agent, the NLI model is deployed with the generated title as the *premise* and the core promise extracted by the Analyzer Agent as the *hypothesis*. A "Contradiction" classification results in hard rejection of the title; it is eliminated from the output and added to the rejection log regardless of its predicted CTR score or VADER arousal magnitude. "Entailment" and "Neutral" classifications allow the title to proceed to composite scoring. Two candidate models are specified: facebook/bart-large-mnli and roberta-large-mnli, both available on Hugging Face, with the former preferred due to its encoder-decoder architecture's demonstrated strength on inference tasks.

This two-layer approach (BERTScore + NLI) is deliberately complementary. BERTScore filters off-topic titles efficiently before they consume expensive NLI inference calls. NLI provides the definitive factual consistency check that BERTScore cannot perform.

3.5.1.6 A6: Predicted CTR (pCTR), machine learning model

The prediction of video virality from title and metadata features using machine learning has been demonstrated as a viable approach in multiple prior studies. Aslam et al. (2021) achieved 86% accuracy classifying popular versus unpopular YouTube videos using XGBoost on features combining title characteristics, engagement metadata, and content category [16]. Paskalev (n.d.) reported an RMSE of 3.48% on YouTube CTR prediction using a Random Forest model trained on title and metadata features [24]. Merrell (n.d.) replicated this approach with XGBoost on approximately 2,000 videos, using the *views per subscriber* normalisation to isolate title quality from channel authority, the same normalisation strategy adopted as the pCTR target variable in YTO [25].

Both Random Forest and XGBoost are selected over deep learning approaches on principled grounds. The available dataset size (tens of thousands of labelled examples) does not justify the data requirements of deep sequence models. The features are tabular, engineered numerics and encoded categorical variables, rather than raw sequences, making tree-based ensemble methods the natural fit. Critically, both methods provide *feature importance* scores, enabling the system to explain which title characteristics most strongly predict virality, a research requirement for interpretability and for validating the psychological trigger taxonomy.

The pCTR model is trained on the subscriber-normalised virality score (Views / Subscriber_Count at upload time) computed from the Kaggle YouTube Trending Dataset [26] and the Kaggle Classic YouTube Statistics Dataset [27], supplemented by targeted scraping of high-performing channels via the YouTube Data API v3. This normalisation is essential: a title on a 10-million subscriber channel receives views from passive subscriber consumption regardless of title quality, masking the title's independent contribution to discovery. Normalisation by subscriber count isolates the discovery component.

3.5.2 Design analysis

3.5.2.1 D1: Multi-agent architecture, supervisor pattern

Multi-agent systems decompose complex tasks into specialised agents, each with a focused role and isolated context, coordinated by a supervisory controller. LangGraph,

the stateful graph-based extension to LangChain, implements this pattern through a directed graph of agent nodes with explicit state transitions, enabling deterministic control flow between agents alongside the flexibility of LLM-based reasoning within each node [28].

The Supervisor pattern, in which a controller agent receives input, delegates sequentially to specialised workers, and receives structured outputs from each, is the architectural pattern adopted for YTO. This pattern is preferred over the Network pattern (any agent can call any other) because the YTO pipeline has a strict sequential dependency: the Analyzer must complete before the Generator can proceed, and the Generator must complete before the Critic can evaluate. The sequential constraint is a design choice, not a technical limitation, and is motivated by the anti-hallucination architecture described in D2.

3.5.2.2 D2: API wrapper with facade pattern

The system is implemented as an **API Wrapper using the Facade design pattern**. The Facade pattern (Gamma et al., 1994 [35]) provides a simplified interface to a complex subsystem, hiding its internal structure from external clients. In YTO, the Facade is a FastAPI REST endpoint that accepts a simple HTTP POST request (containing the video transcript and optional metadata) and returns a ranked list of scored title candidates. All internal complexity (agent orchestration, prompt construction, vector retrieval, ML inference, NLP evaluation) is invisible to the consuming application.

FastAPI is selected as the backend framework on the basis of its ASGI (Asynchronous Server Gateway Interface) architecture, which provides native async/await support for I/O-bound LLM API calls. This is a significant practical consideration: LLM inference calls are high-latency I/O operations (typically 1–5 seconds per call), and synchronous handling would block a process for each active request. ASGI-based handling allows a single FastAPI process to manage hundreds of concurrent requests efficiently, with each request awaiting its LLM response without blocking others. The framework additionally provides automatic Pydantic-based request validation, critical for structured agent I/O, and native streaming response support for real-time token output.

3.5.2.3 D3: Psychological trigger operationalisation

The core technical novelty of the Generator Agent is not the LLM it uses but the prompt infrastructure through which psychological theory is translated into algorithmic instructions. Each psychological trigger in the T1–T7 taxonomy maps to a set of explicit linguistic markers that the Generator is instructed to employ:

Table 3.3: Psychological Trigger Taxonomy and Linguistic Operationalisation

Trigger	Theoretical Basis	Linguistic Markers	Example
T1: Curiosity Gap	Loewenstein (1994) [1], Golman & Loewenstein (2016) [2]	"Why I…", "The Real Reason…", "What Happens When…"	"Why I Quit My Job After Discovering This"
T2: Loss Aversion / FOMO	Kahneman & Tversky (1979) [3]	"Don't Make This Mistake", "Before You Buy…", "You're Doing X Wrong"	"Stop Wasting Money on This (I Did For 3 Years)"
T3: Social Proof / Authority	Cialdini (1984) [4]	"Expert Says…", "10,000 People Tried This", "What Scientists Found", numbered lists	"I Asked 50 Experts What They'd Do With $1,000"
T4: Urgency / Scarcity	Cialdini (1984) [4]	"Before It's Too Late", "This Changes Everything", "Last Chance"	"Watch This Before YouTube Changes Everything"
T5: Extreme / Superlative	Berger & Milkman (2012) [5]	"The Worst", "Impossible", "Never Seen Before", "Most Expensive"	"I Tried the World's Most Expensive Pizza"
T6: Personal Stakes / Vulnerability	Parasocial Relationship Theory (Horton & Wohl, 1956) [32]	"I Lost…", "I Failed…", "My Biggest Mistake", "I Quit…"	"I Quit My $200K Job to Do This"
T7: Number + Specificity	Cognitive Ease (Kahneman, 2011 [33]); Specificity implies credibility	Exact numbers, percentages, timeframes	"This $11 Meal Changed How I Cook Forever"

This operationalisation is a primary research contribution. As noted in the gap analysis in Section 3.4.5, no existing generative system encodes psychological trigger theory as explicit prompt engineering instructions. The trigger taxonomy bridges a documented gap

between the academic literature on persuasion psychology and the practical implementation of LLM-based content generation.

The relationship between specificity and credibility (T7) is particularly notable. Kawata et al. (2025), in a study of 12,466 Microsoft Excel YouTube tutorial videos, found a bell-shaped relationship between title length/word count and view counts, with a performance peak at 7–12 words and 60–70 characters [7]. This empirical finding provides the length constraint that the Generator Agent enforces as a hard output requirement.

3.5.3 Workflow analysis

3.5.3.1 W1: Dataset pipeline

The pCTR model's training pipeline draws on four primary data sources:

Kaggle YouTube Trending Dataset (rsrishav) [26]: Daily snapshots of US trending videos, including title, view count, like count, comment count, category, and channel metadata. This dataset provides the primary source of viral linguistic patterns and the raw material for the subscriber-normalised virality score target variable. Its principal limitation is that trending status reflects accumulation over time rather than early CTR; it is used as a proxy for viral linguistic patterns rather than as a direct CTR dataset.

Kaggle YouTube Trending Statistics (datasnaek) [27]: A historical dataset of 29,089 unique videos across US, UK, Canadian, and Indian trending data, widely used in academic ML research on YouTube virality. This dataset provides cross-national variation and a larger historical baseline.

YouTube Data API v3 (targeted scraping): Directed collection from demonstrably high-performing channels (MrBeast, Veritasium, Mark Rober, Kurzgesagt) provides gold-standard positive examples from channels where title engineering is known to be deliberate and effective. This targeted scraping supplements the trending datasets with examples where title quality can be more confidently attributed to intentional design rather than channel authority alone.

Webis Clickbait Corpus [29]: Annotated clickbait titles from academic detection research provide the negative example set, titles characterised by deceptive or misleading metadata, used to train the NLI validation layer's rejection boundary.

Data cleaning requirements include: deduplication of trending entries (each video appears multiple times in the trending dataset as it trends over multiple days); subscriber count normalisation to the upload date rather than current count; removal of non-English titles; removal of titles that are purely SEO keyword strings without natural language structure; emoji preservation and proper tokenisation (Chen, 2023 [18] found emoji carry significant sentiment and virality signals); and toxicity filtering via unitary/toxic-bert to prevent the Prompt Database from including examples that optimise for harmful engagement patterns.

3.5.3.2 W2: Evaluation workflow

The evaluation methodology operates across three levels:

Offline ML Evaluation: The pCTR model is trained on a 70/15/15 train/validation/test split, with the test set held out until final evaluation to prevent data leakage. Primary metric: F1-score (chosen to account for class imbalance in the binary classification variant). Secondary metrics: ROC-AUC and RMSE for the regression variant. Target: F1 > 0.75, ROC-AUC > 0.80.

System-Level Evaluation: The end-to-end pipeline is evaluated on a demonstration dataset of 10+ diverse video topics across three content categories (Education, Entertainment, Technology). Six metrics are recorded for each run: pCTR score (target mean > 0.06), VADER arousal magnitude (target mean |compound| > 0.40), BERTScore F1 (target range 0.35–0.60), Self-BLEU (target < 0.40, indicating inter-title diversity within each batch), NLI acceptance rate (target > 60%), and P95 end-to-end latency (target < 10 seconds).

Self-BLEU warrants particular explanation. Originally introduced by Zhu et al. (2018) in the Texygen benchmarking platform for text generation [36], Self-BLEU computes the BLEU score between each generated title and the other titles in the same batch. A high Self-BLEU score indicates that the generated titles are minor paraphrases of a single pattern, meaning the Generator has collapsed to a single trigger type despite instructions to produce diverse variants. A low Self-BLEU score confirms that the five candidates represent genuinely distinct psychological framings.

Human Evaluation: A preference survey with a minimum of 50 participants evaluates generated titles on a four-dimension rubric (1–5 Likert scale): Curiosity (information gap created), Emotional Impact (arousal strength), Clarity (topic comprehensibility), and

Trustworthiness (perceived honesty). Each participant evaluates three title variants for the same video: Baseline A (the original human-generated title), Baseline B (a zero-shot LLM title), and System Output (YTO's top-ranked candidate). The primary hypothesis is that YTO significantly outperforms both baselines on Curiosity and Emotional Impact while maintaining comparable Clarity and Trustworthiness scores.

LLM-as-a-Judge Evaluation: A secondary automated evaluation method, directly addressing RQ4, uses GPT-4o as a rubric-based evaluator to score generated titles on three dimensions (Curiosity, Urgency, Relevance) via a structured prompt. This approach, supported by the LLM-RUBRIC framework (ACL 2024) which demonstrated that calibrated LLM judges combining multiple rubric dimensions can achieve reliable correlation with human raters [30], scales to large title sets without the cost of human annotation. All LLM-as-a-Judge scores are cross-validated against the human evaluation scores on the overlapping subset to assess their reliability.

3.6 Reflection, research gap justification

The systematic review conducted in this chapter converges on a consistent finding: the research gap that motivates YTO is real, multi-dimensional, and not addressed by any single prior work or commercial tool.

From the psychological theory literature, the mechanisms by which titles generate clicks are well-documented. Loewenstein's Information Gap Theory [1], Kahneman and Tversky's Prospect Theory [3], Cialdini's Social Proof framework [4], and Berger and Milkman's emotional arousal research [5] collectively provide a rigorous theoretical basis for what makes a title effective. However, these theories are documented in the literature as descriptive accounts of human behaviour; they are not implemented as algorithmic instructions in any generative AI system. The operationalisation of psychological theory as explicit LLM prompt engineering is the first dimension of the gap.

From the existing systems analysis, each prior approach addresses at most two or three of the five criteria identified in Table 3.2. Manual copywriting achieves psychological grounding and generative quality but fails on scalability. Analytics platforms achieve scalability and data richness but have no generative capability. Generic LLM prompting achieves scalability but fails on psychological grounding and faithfulness enforcement. Clickbait detection achieves faithfulness checking but has no generative pathway. The

five-way intersection, proactive, psychologically grounded, generative, faithfulness-enforced, and scalable, is unoccupied. That is the second dimension of the gap.

From the technological analysis, the component-level evidence confirms that the individual technologies required to close these gaps exist and have been individually validated. LLMs can generate creative text with constraint satisfaction (Brown et al., 2020 [10]; Wei et al., 2022 [11]). VADER can measure emotional arousal in short text (Hutto & Gilbert, 2014 [13]). BERTScore can measure semantic relevance (Zhang et al., 2020 [14]). NLI models can detect factual contradiction between generated and source text (Laban et al., 2023 [15]). Tree-based ML can predict video virality from title features (Aslam et al., 2021 [16]). Multi-agent architectures can coordinate specialised LLM pipelines (LangChain/LangGraph, 2024 [28]). The gap is not a component-level gap; it is an *integration and operationalisation* gap. No prior work has combined these components into a coherent system designed specifically to engineer psychologically optimised, faithfulness-verified, virality-ranked YouTube titles.

From the empirical literature, the most recent evidence further validates both the problem and the direction. Zhang et al. (2024) demonstrated in a 2-million-user field experiment that AI-generated metadata significantly boosts content consumption when human-AI co-creation is applied, but pure AI generation *underperforms* existing human-written titles [8]. Yang (2024) confirmed that generic AI title generation lacks the emotional sophistication of expert human copywriting [12]. Both findings point to the same conclusion: the research need is not for more AI generation, but for *more intelligently engineered* AI generation, precisely the contribution of this project.

3.7 Chapter summary

This chapter has reviewed the literature across the domains of engagement psychology, generative AI and prompt engineering, NLP evaluation, ML-based virality prediction, and existing tools and systems. The domain overview established that CTR-driven algorithmic discovery creates a structural problem in which video metadata quality, particularly title quality, exerts disproportionate influence over content visibility. The existing systems analysis demonstrated that manual copywriting, analytics platforms, generic LLM prompting, and clickbait detection each satisfy some requirements of an ideal solution while failing others, with no system occupying the intersection of all five.

The technological analysis confirmed that the required component technologies have each been individually validated in the literature, and that the integration gap, the absence of a system that combines them under a psychologically grounded, ethically constrained generative architecture, is the specific contribution of this research.

Chapter 4 presents the research methodology, establishing the philosophical and strategic framework (Design Science Research under a Pragmatist paradigm) that governs how the system will be built and evaluated, and mapping the DSRM six-phase process to the specific activities of this project.

4 METHODOLOGY

4.1 Research paradigm

4.1.1 Paradigm: pragmatism

Every research project rests on a set of philosophical assumptions about the nature of knowledge and how it can be obtained. These assumptions, collectively the research paradigm, determine which methods are legitimate, what counts as evidence, and how findings are interpreted. The paradigm adopted for this research is **Pragmatism**.

Pragmatism, as a philosophical tradition originating with Peirce, James, and Dewey and later applied to social science research methodology, holds that knowledge should be evaluated by its practical consequences and its capacity to solve real-world problems rather than by adherence to a single ontological doctrine [1]. In pragmatist epistemology, there is no single "best" way to know something: the appropriate methods depend on the research question and what kinds of evidence are most useful for answering it. Truth, in the pragmatist formulation, is what works.

This stance has a direct bearing on research design. Because pragmatism is not committed to either the objective, value-free stance of positivism or the subjective, meaning-oriented stance of interpretivism, it permits the legitimate combination of quantitative and qualitative methods in the same study. This flexibility is essential for this research: the evaluation of the YTO system requires both quantitative measurement (F1-score, predicted CTR, BERTScore, VADER compound) to assess objective system performance, and qualitative human evaluation (preference surveys, LLM-as-a-Judge rubric scoring) to assess the subjective dimensions of title quality (curiosity, emotional impact, and trustworthiness) that quantitative proxies cannot fully capture.

Why not Positivism: A purely positivist paradigm would require tightly controlled laboratory conditions to isolate a single causal variable, a design incompatible with a system whose performance depends on the interaction of multiple AI components, a probabilistic ML model, and human evaluators with inherent variability. It would also favour deductive testing of pre-specified hypotheses over the discovery-oriented iteration that artefact construction requires.

Why not Interpretivism: An interpretivist paradigm would focus on the subjective meaning that creators and viewers attach to titles, appropriate for a study of creator experience or viewer psychology. This research is ultimately concerned with a software artefact and its measurable outputs, not with the lived experience of participants.

Pragmatism is the fit: It evaluates the artefact by whether it works, whether the titles it generates outperform baselines on measurable engagement predictors while satisfying human evaluators on perceptual quality criteria. Both types of evidence are gathered, because both matter.

4.2 Research approach

4.2.1 Approach: abductive

Research approaches describe how the relationship between theory and data is structured. Three main approaches are distinguished in the literature: deductive (testing a pre-formed hypothesis derived from theory), inductive (generating theory from observed data), and abductive (moving iteratively between theory and observation to infer the best explanation) [2].

This research adopts an **abductive approach**.

The study does not begin with a null hypothesis to be tested under controlled conditions (deductive), nor does it begin with a blank-slate observation of the world from which theory is meant to emerge inductively. Instead, it begins with established theoretical constructs (Information Gap Theory [3], Prospect Theory [4], Social Proof [5], and the empirically validated principle that high-arousal emotional content drives viral spread [6]) and uses these to *design* a software artefact. It then evaluates the artefact empirically against measurable performance criteria.

This movement, from established theory to artefact design to empirical evaluation back to theoretical reflection, is precisely what abductive reasoning describes: reasoning from existing theoretical knowledge to a novel instantiation of that knowledge, and then evaluating whether the instantiation works as theorised. The abductive approach is consistent with the Design Science Research paradigm (Section 4.3), in which existing theories serve as the "kernel theories" that motivate and guide artefact design, and the artefact's evaluated performance contributes new knowledge back to the theoretical base.

4.3 Research strategy

4.3.1 Strategy: design science research (DSR)

The research strategy adopted is **Design Science Research (DSR)**, as formalised by Peffers, Tuunanen, Rothenberger, and Chatterjee (2007) in the widely cited *Design Science Research Methodology for Information Systems Research* [7].

DSR is the dominant research paradigm in Information Systems and Software Engineering when the primary output of the study is a novel **artefact**, a construct, model, method, or instantiation, created to address a specific real-world problem. Hevner et al. (2004) articulate the core principle: design science creates and evaluates IT artefacts intended to solve identified organisational problems [8]. Unlike natural science research, which describes and explains existing phenomena, Design Science *creates new reality* through the act of building. The artefact itself is the primary research contribution, and its evaluation against defined performance criteria constitutes the empirical contribution.

DSR is characterised by two equally weighted requirements: - **Rigor:** The artefact must be grounded in existing knowledge (theoretical constructs, prior research, established techniques) and evaluated using systematic, reproducible methods. - **Relevance:** The artefact must address a real, identified problem and demonstrate practical value to an identifiable stakeholder community.

For this research, both requirements are met. The system is rigorously grounded in Information Gap Theory [3], Prospect Theory [4], Social Proof [5], and established NLP techniques (VADER [9], BERTScore [10], NLI [11], Few-Shot prompting [12], Chain-of-Thought reasoning [13]). Its relevance is established by the empirical evidence that metadata quality directly determines content visibility for millions of content creators [14][15].

4.3.2 Alternative strategies considered and rejected

Table 4.1: Alternative research strategies considered and rejected

Strategy	Why Rejected
Case Study	Appropriate for studying existing phenomena in real-world context; this research creates a new artefact rather than

Strategy	Why Rejected
	studying an existing one
Controlled Experiment	Would require live YouTube channels with significant traffic for real CTR measurement, outside the ethical and practical scope of a research prototype
Survey	Captures perceptions and opinions; cannot evaluate software system performance or NLP metric behaviour
Grounded Theory	Generates theory inductively from data; this research has an established theoretical foundation and a defined artefact output
Action Research	Requires collaborative, iterative engagement with an organisational client; this is a solo academic project with no host organisation

DSR is the methodologically sound choice because the research question, how to design and evaluate a multi-agent AI system for title optimisation, is fundamentally a design question, not a discovery question.

4.4 Fact collection mechanisms

This section specifies the data sources used across all phases of the project: model training, system evaluation, and the Prompt Database.

Table 4.2: Data Collection Mechanisms

Data Type	Source	Collection Method	Purpose
YouTube trending video titles + engagement metrics	Kaggle YouTube Trending Dataset (rsrishav) [16]	Direct download via Kaggle CLI	pCTR ML model training; viral linguistic pattern analysis
Historical YouTube trending statistics	Kaggle YouTube Trending Video Statistics (datasnaek) [17]	Direct download via Kaggle CLI	ML model training; baseline viral pattern dataset
High-performing channel titles + metadata	YouTube Data API v3	Targeted API scraping (MrBeast, Veritasium, Mark Rober, Kurzgesagt)	Few-Shot Prompt Database population; gold-standard positive examples
Annotated clickbait titles (negative	Webis Clickbait Corpus (Potthast et	Download from	NLI validation layer training;

Data Type	Source	Collection Method	Purpose
examples)	al., 2016) [18]	webis.de	rejection boundary definition
Psychological trigger type labels (T1–T7)	Custom annotation	Manual labelling of 500–1,000 title samples by researcher	Trigger classification feature for pCTR model
Human title preference ratings	A/B preference survey (n ≥ 50 participants)	Online survey instrument (Google Forms)	Qualitative human evaluation; Curiosity, Emotional Impact, Clarity, Trustworthiness scoring

4.4.1 Data ethical considerations

All secondary data sources are publicly available. The Kaggle datasets are published under open licences for research use. YouTube Data API v3 usage complies with YouTube's Terms of Service for non-commercial research data collection, with rate limits observed and no personal user data collected [19]. The human preference survey collects no personally identifiable information beyond optional demographic data (age range, YouTube usage frequency); participation is voluntary and anonymised.

4.5 Research methodology

4.5.1 Methodology: Peffers' design science research methodology (DSRM)

The specific methodology adopted is the **six-phase Design Science Research Methodology (DSRM)** proposed by Peffers et al. (2007) [7]. DSRM provides a structured, phase-by-phase framework for conducting DSR in Information Systems and Software Engineering, ensuring that the artefact development process is both systematic and communicable.

Peffers et al. identified that prior DSR work lacked a common process model that researchers and reviewers could use to evaluate whether a study followed DSR rigor. The DSRM fills this gap by prescribing a nominal sequence of six activities, while recognising that real research may begin at any activity and iterate between them. The six activities are:

1. **Problem Identification and Motivation**: Define the problem and justify the value of a solution
2. **Define Objectives of a Solution**: Specify what a successful artefact must accomplish
3. **Design and Development**: Create the artefact
4. **Demonstration**: Use the artefact to solve one or more instances of the problem
5. **Evaluation**: Observe and measure how well the artefact addresses the problem
6. **Communication**: Communicate the problem, artefact, and evaluation to relevant audiences

This process aligns naturally with the structure of this thesis and the academic calendar of the project, as shown in Section 4.6.

4.6 Execution workflow

The following table maps each DSRM activity to its concrete project realisation, the chapter(s) in which it is documented, and its primary deliverable output.

Table 4.3: DSRM Execution Workflow

DSRM Activity	Project Action	Chapter(s)	Output
1. Problem Identification & Motivation	Analysis of the creator CTR bottleneck; literature review of clickbait research, LLM generation limits, and psychological trigger theory	Ch1, Ch2	Research problem statement; justification of value of a solution; research gap articulation
2. Define Objectives	Specification of four research objectives (§2.1–2.4); definition of success criteria (pCTR > 6%, F1 > 0.75, NLI acceptance > 60%, human preference significance)	Ch1, Ch3	Research aim; four research objectives; system performance targets
3. Design & Development	Multi-agent pipeline	Ch4, Ch5	Working system prototype; trained

DSRM Activity	Project Action	Chapter(s)	Output
	architecture; Prompt Factory design; pCTR ML model training; FastAPI backend implementation; NLI and BERTScore integration		pCTR model; populated Prompt Database; REST API endpoint
4. Demonstration	Application of the system to 10+ diverse video topics across three content categories (Education, Entertainment, Technology) with outputs recorded	Ch5, Ch6	Demonstration dataset: video transcripts -> generated title batches with per-title scores
5. Evaluation	Offline ML evaluation (F1, ROC-AUC, RMSE); system-level metrics (pCTR, VADER, BERTScore, Self-BLEU, acceptance rate); human A/B preference survey ($n \geq 50$); LLM-as-a-Judge rubric scoring	Ch6	Quantitative evaluation report; human preference results; LLM-judge scores; comparative analysis against Baseline A and Baseline B
6. Communication	Thesis dissertation write-up; GitHub repository publication; potential academic conference paper	All chapters	Thesis document; public code repository

4.6.1 Entry point

Following Peffers et al.'s (2007) entry point taxonomy [7], this research begins at **Activity 1 (Problem-Centred Initiation)**: the research was motivated by the identification of a practical problem (the creator metadata optimisation bottleneck) rather than by an available technology seeking an application or a prior design in need of improvement.

4.7 Project management methodology

4.7.1 Methodology: Agile (Scrum-inspired)

The development and research process is managed using an **Agile, Scrum-inspired methodology**, adapted for a single-researcher academic project [20].

Agile is selected over traditional plan-driven methodologies for three reasons specific to this project.

Each component of the YTO system (the Analyzer Agent, Generator Agent, Critic Agent, pCTR model, and FastAPI backend) needs to be developed and tested as an independent increment before integration, which matches the Sprint model: each sprint delivers a testable, working component, and the output of each sprint informs the design decisions of the next. Prompt engineering is also inherently empirical: the optimal CoT sequence, the effective few-shot example count, and the BERTScore threshold for the relevance filter cannot all be fully specified in advance, but emerge through iterative testing and refinement, and Agile accommodates this by treating backlog refinement as an ongoing activity rather than requiring all requirements to be defined upfront. Finally, there's a question of scale: this is a solo project with a fixed timeline, and PRINCE2 and other heavyweight methodologies impose extensive documentation overhead designed for multi-stakeholder projects. Agile's minimal ceremony (sprint planning, a working product increment, and retrospective) fits the scale and pace of a single-researcher academic project.

Sprint structure: Six two-week development sprints (Sprints 1–6) follow the Interim Submission 01 deadline. Each sprint is focused on a single deliverable component. Backlog items that cannot be completed within a sprint are carried forward rather than blocked.

Tools: GitHub for version control and sprint board; a GitHub Projects Kanban board tracks backlog, in-progress, and completed items per sprint.

4.7.2 Alternative considered: PRINCE2

PRINCE2 (Projects IN Controlled Environments) was considered and rejected. PRINCE2's stage-gate model, management product documentation requirements, and

multi-role structure (Project Board, Project Manager, Team Manager) are designed for organisational projects with external stakeholders and formal governance requirements. For a solo academic research project with a supervisor rather than a client board, this overhead is unnecessary and counterproductive.

4.8 Project timeline

Table 4.4: Project Timeline

Phase	Deliverable	Target Date	Status
Interim Submission 01	Chapter 1 (Introduction), Chapter 2 (Objectives), Chapter 3 (Literature Review), Chapter 4 (Methodology)	28 June 2026	In progress
Sprint 1: Data & ML	Kaggle dataset pipeline; feature engineering; pCTR model (Random Forest + XGBoost) trained and evaluated; F1 / ROC-AUC recorded	12 July 2026	Planned
Sprint 2: Analyzer Agent	Analyzer Agent fully implemented; video brief output (core promise, key entities, emotional ceiling, trigger candidates) validated on 5 test cases	19 July 2026	Planned
Sprint 3: Generator Agent	Generator Agent with Prompt Database (vector retrieval), CoT sequence, and Few-Shot injection; 5 candidate titles produced per run	26 July 2026	Planned
Sprint 4: Critic	Full end-to-end	2 August 2026	Planned

Phase	Deliverable	Target Date	Status
Agent	pipeline: NLI, VADER, BERTScore, pCTR scoring integrated; composite score ranking working; rejection log operational		
Sprint 5: FastAPI & Integration	FastAPI REST endpoint live; full Analyzer->Generator->Critic pipeline accessible via HTTP POST; Pydantic request/response validation working	9 August 2026	Planned
Sprint 6: Evaluation	Demonstration dataset generated (10+ topics, 3 categories); human survey launched and n ≥ 50 responses collected; LLM-as-a-Judge scoring completed; all evaluation metrics compiled	16 August 2026	Planned
Chapter 5: System Design	SRS, diagrams (use case, class, activity, sequence), architecture documentation written	19 August 2026	Planned
Chapter 6: Implementation	Implementation chapter written with pseudocode, code snippets, and sample outputs	21 August 2026	Planned
Chapter 7: Results & Evaluation	Evaluation chapter written with result tables, graphs, and comparative analysis	23 August 2026	Planned
Chapter 8: Conclusion	Concluding chapter; objectives triangulated against	25 August 2026	Planned

Phase	Deliverable	Target Date	Status
	results; future work articulated		
Final Submission	Complete thesis document + GitHub repository published	28 August 2026	Planned

The timeline allocates six sprints (twelve weeks) to system development and evaluation, followed by four writing days for Chapters 5–8 once implementation and evaluation data are complete. This sequence reflects the DSR cycle: build first, then document and communicate.

4.9 Ethical considerations

Research involving AI-generated content, human participant evaluation, and publicly collected datasets raises several ethical dimensions that must be addressed explicitly.

4.9.1 Deception versus persuasion

The central ethical tension in this research is the boundary between legitimate persuasion and deceptive manipulation. Titles optimised for high CTR can legitimately employ psychological techniques (curiosity gaps, urgency framing, emotional arousal) when those techniques accurately represent the content the viewer will receive. They become ethically problematic when they create false expectations, fabricate claims, or misrepresent the video's content.

The YTO system is explicitly designed to operate on the legitimate side of this boundary. The NLI-based hard rejection gate eliminates any title classified as a factual "Contradiction" to the video's core promise before it is surfaced to the creator. The Analyzer Agent establishes the semantic envelope of permissible claims by extracting only what is truthfully present in the video transcript. The Generator Agent works only within that envelope. This architecture means the system structurally prohibits deceptive output; it is not an opt-in safety feature but a mandatory pipeline stage.

This design implements the distinction between *persuasion* (legitimate influence through accurate emotional framing) and *manipulation* (influence through false or misleading claims). The system targets persuasion while architecturally blocking manipulation.

4.9.2 Data privacy

No personal user data is collected at any stage of this research. All training datasets are publicly available: the Kaggle YouTube Trending Datasets contain only publicly accessible video metadata (titles, view counts, like counts, channel names) that YouTube makes available through its public interface. The YouTube Data API v3 is used in compliance with YouTube's Terms of Service for non-commercial academic research [19], with API rate limits respected. The human preference survey collects no personally identifiable information; responses are recorded anonymously and used solely for aggregate comparative analysis.

4.9.3 Bias and toxicity in training data

YouTube trending data may contain titles that achieved high engagement through toxic, sensationalist, or harmful framing (hate-watching content, inflammatory news titles, or content that exploits tragedy). If such examples are included in the Prompt Database unfiltered, the system could learn to optimise for engagement patterns that are harmful.

To mitigate this, a toxicity filtering stage is applied to the Prompt Database using the unitary/toxic-bert classifier. Titles scoring above the toxicity threshold are excluded from the few-shot example pool regardless of their historical engagement performance. This ensures the system optimises for psychological engagement mechanisms (curiosity, emotional arousal, social proof) without learning harmful framing as a positive engagement signal.

4.9.4 Transparency and human oversight

The YTO system is positioned as a *co-pilot for creators*, not an autonomous replacement for human editorial judgment. The API returns a ranked list of five candidates with per-title scoring, the creator sees the options and the scores, and makes the final selection. No title is published automatically. This design preserves human agency in the content creation workflow and makes the system's reasoning interpretable (the creator can see why a title scored highly on pCTR or was flagged by NLI).

The "black box" nature of the underlying LLM (GPT-4o) is acknowledged explicitly. The system does not claim to fully explain why any specific title was generated; it provides interpretable *evaluation scores* for each output, not a mechanistic explanation of the generation process.

4.9.5 Academic integrity

All sources are cited in IEEE format. The research prototype is developed independently. Any use of AI tools during the writing or implementation process is acknowledged in accordance with NSBM Green University's academic integrity policy. The artefact is a research prototype intended for academic evaluation; commercial deployment would require additional ethical review, a point explicitly noted in the limitations.

4.9.6 Platform policy compliance

YouTube's Terms of Service (Section 4H) explicitly prohibit video metadata that "sets up viewer expectations the content doesn't deliver" [19]. Violations can result in video demonetisation, channel strikes, and algorithmic suppression. The NLI faithfulness layer is therefore not only an ethical safeguard but also a **platform compliance mechanism**: every title produced by the system has passed a computational check confirming it does not contradict the video's content, protecting the creator from inadvertent ToS violations.

4.10 Chapter summary

This chapter has established the philosophical and procedural framework that governs this research. The philosophical layer rests on a **Pragmatist paradigm**, which licenses the combination of quantitative and qualitative evaluation methods appropriate for assessing both system performance and human preference. The **abductive approach** reflects the research's movement from established psychological and NLP theory to artefact design to empirical validation. The **Design Science Research strategy** positions this project within the IS/SE tradition of artefact-oriented scholarship, in which building and evaluating a novel system constitutes a genuine research contribution. The **Peffers DSRM** provides the six-phase process model that structures the project from problem identification through final communication.

The methodological choices made in this chapter are not procedural formalities. They directly shape what the research can and cannot claim. DSR under a pragmatist paradigm allows this research to claim that the YTO system *works*, meaning it outperforms defined baselines on defined metrics, without requiring the experimental isolation that would be necessary to claim universal causal laws about AI-generated content. That is the

appropriate claim scope for a software engineering research prototype evaluated over a bounded set of test cases.

Chapter 5 proceeds to operationalise these methodological decisions into a full system requirements specification, including stakeholder analysis, use case diagrams, class diagrams, activity diagrams, sequence diagrams, and formal functional and non-functional requirements.

5 SYSTEM REQUIREMENTS SPECIFICATION

5.1 Chapter overview

Chapter 4 established the philosophical and procedural framework governing this research: a pragmatist, abductive, Design-Science approach executed through Peffers' six-phase DSRM. This chapter operationalises that framework into a concrete system specification. It identifies who the system serves, traces each research objective from Chapter 2 down to the specific component that satisfies it, and formalises the system's behaviour through the standard Unified Modelling Language (UML) diagram set [4]: use case, class, activity, and sequence diagrams. The chapter closes with the functional and non-functional requirements that the implemented system, described in full in Chapter 6, was built to satisfy.

Section 5.2 identifies the stakeholders of the system and their respective interests. Section 5.3 traces the operationalisation of the four research objectives into concrete architectural components. Section 5.4 presents the use case model. Section 5.5 presents the class model of the shared data contracts and agent components. Section 5.6 presents the activity model of the end-to-end request lifecycle. Section 5.7 presents sequence diagrams for the two behaviourally distinct execution paths: the successful generation path and the NLI rejection path. Section 5.8 presents the proposed system architecture in full. Sections 5.9 and 5.10 specify the functional and non-functional requirements respectively, stated in a form that is directly verifiable against the evaluation reported in Chapter 7. Section 5.11 closes the chapter.

Because implementation (Sprints 1–5) preceded the writing of this chapter, the diagrams and requirements presented here describe the system as it was actually built and verified, not a forward-looking design that implementation later deviated from. Where an as-built decision differs from what a purely top-down requirements process might have specified, that decision is noted and justified in place.

5.2 Stakeholder analysis

Three distinct stakeholder groups have an interest in the YouTube Title Optimizer (YTO), each with a different relationship to the system and a different basis for evaluating its success.

Content Creators are the primary human stakeholder and the system's direct user. A creator interacts with the system by submitting a video transcript, an optional description, and a content category, and receives a ranked set of title candidates with interpretable per-title scores. Their interest is in reducing the time and expertise cost of producing an effective title while retaining final editorial control. The system is positioned as a co-pilot, per the transparency principle established in Section 4.9.4, not an autonomous publisher. Sub-populations within this group (individual educators, small businesses, and independent journalists identified in Section 1.5.1) share this interest but differ in their baseline access to professional copywriting expertise, which is precisely the inequality the system is motivated to reduce.

Researchers and Academic Evaluators, including the project supervisor and thesis examiners, are stakeholders in the system's rigor and evaluability rather than its day-to-day use. Their interest is in whether the system's design decisions are theoretically grounded (Section 3.5), whether its performance claims are measured against defensible baselines and metrics (Section 4.6, Activity 5), and whether the artefact constitutes a genuine, reproducible research contribution rather than an opaque or unfalsifiable claim. This stakeholder group is served directly by the evaluation methodology detailed in Chapter 7.

The Platform Algorithm is a non-human but architecturally significant stakeholder. YouTube's recommendation system, as characterised in Section 3.3.2, is the actual consumer of the system's output in the sense that it is the mechanism that converts a generated title into real-world distribution outcomes. The system does not interact with this stakeholder directly (no live API call is made to YouTube's ranking system, per the scope boundary in Section 1.9), but every design decision in the Critic Agent (Section 5.5, Section 5.8) is oriented toward producing titles that would perform well under this stakeholder's actual ranking behaviour, and the pCTR model (Section 3.5.1, A6) exists specifically to proxy this stakeholder's response in the absence of a live integration.

A fourth, indirect stakeholder group, **video viewers** (the audience whose click decisions the system is ultimately trying to earn), is acknowledged but not directly engaged by the system's interfaces. The NLI faithfulness gate (Section 5.6, Section 5.7) acts as this stakeholder's proxy representative within the architecture: it exists specifically to prevent the system from optimising creator and platform interests at the viewer's expense through deceptive titling, operationalising the ethical position established in Section 4.9.1.

5.3 Operationalisation process

Table 5.1 traces each of the four research objectives stated in Chapter 2 to the specific system component or process that satisfies it, closing the loop between the research questions posed in Chapter 1 and the concrete architecture presented in this chapter.

Table 5.1: Operationalisation of Research Objectives

Research Objective (Chapter 2)	System Component	Verification Point
2.1 To Identify viral linguistic features and the T1–T7 trigger taxonomy	Feature engineering pipeline (Section 3.5.1, W1); Analyzer Agent's trigger_candidates extraction (Section 5.5)	Feature importance ranking against the trained pCTR model (Chapter 7)
2.2 To Analyse the effectiveness of Few-Shot, CoT, and Persona prompting	Generator Agent's Prompt Factory: dynamic Prompt Database retrieval (Section 5.8) + four-step CoT sequence + persona system prompt	Comparison against the zero-shot Baseline B (Chapter 7)
2.3 To Design and Develop the three-agent pipeline as a Facade-pattern API Wrapper	Analyzer → Generator → Critic pipeline (Section 5.6), orchestrated by the LangGraph state machine, exposed through the FastAPI Facade (Section 5.8)	End-to-end pipeline execution (Chapter 6)
2.4 To Evaluate the system against quantitative and qualitative baselines	Critic Agent's composite scoring (Section 5.7); offline ML evaluation, system-level evaluation, human preference survey, LLM-as-a-Judge scoring	Chapter 7

The operationalisation reveals a structural property of the design: each research objective maps to exactly one primary component, and the three agent components (Analyzer, Generator, Critic) collectively cover Objectives 2.1 through 2.3, while Objective 2.4 is satisfied by processes external to the runtime pipeline: the evaluation scripts and human-facing survey instrument described in Chapter 7. This separation is deliberate: the production pipeline that a creator invokes contains no evaluation logic beyond the Critic's real-time scoring, keeping the artefact itself lean while the research evaluation apparatus is built and run independently against it.

5.4 Use case diagrams

The system exposes exactly three use cases to its single human actor, the Content Creator, corresponding to the two REST endpoints described in Section 5.8, modelled following the use-case-driven approach established by Jacobson et al. [3]. All three use cases are presented in the body, as their combined number falls within the three-diagram limit specified by the project template.

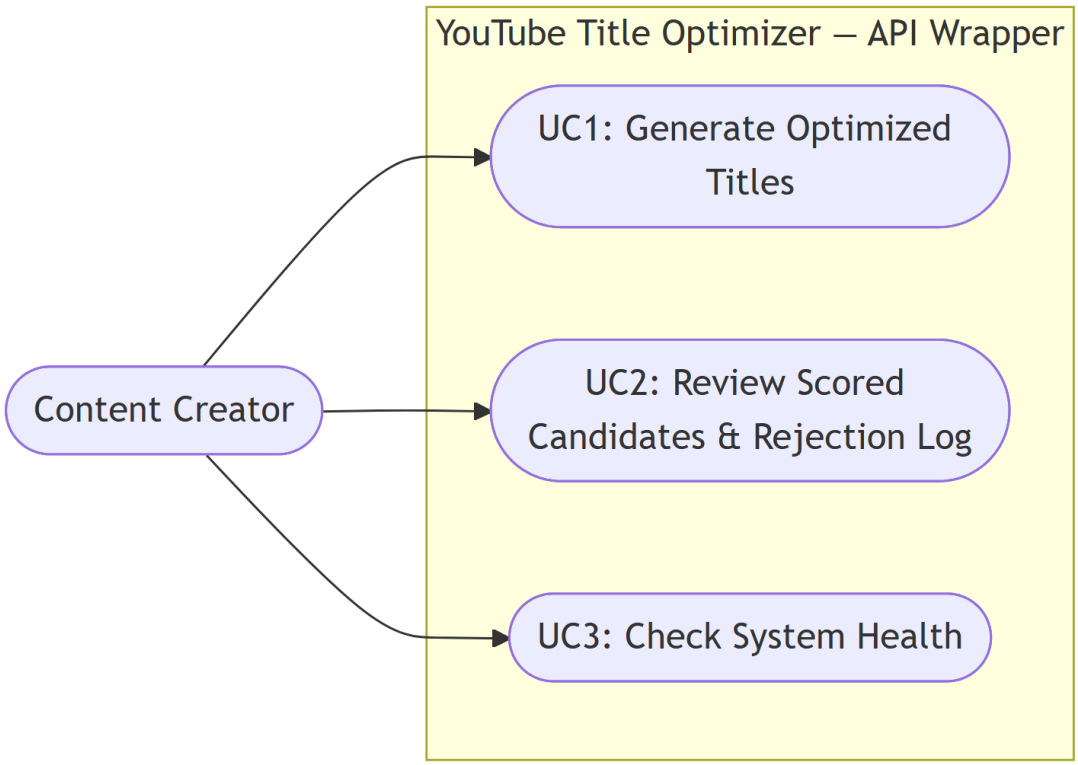


Figure 5.1: Use Case Diagram

Figure 5.1: Use Case Diagram: Content Creator and the YTO API Wrapper

UC1: Generate Optimised Titles. The creator submits a video transcript (minimum 50 words, enforced at the API boundary), an optional description, and a content category. The system returns five ranked, scored title candidates. This is the primary use case and the one through which every other research objective is ultimately exercised.

UC2: Review Scored Candidates and Rejection Log. Because the API response includes each candidate's individual pCTR, VADER, BERTScore, and NLI scores alongside the ranked list, and any hard-rejected candidates with their rejection reason, the creator can inspect why a given title was ranked, or excluded, rather than receiving an opaque final answer. This use case is satisfied by the structure of UC1's response rather than by a separate endpoint, consistent with the transparency principle established in Section 4.9.4.

UC3: Check System Health. A lightweight GET /health endpoint allows a consuming application to confirm the service is available and to retrieve the active model versions (the LLM identifier and the pCTR model type) before submitting a generation request, which matters for any client application built against this API Wrapper, including the demonstration frontend described in Chapter 6.

5.5 Class diagram

The system's data contracts are implemented as Pydantic models, which serve simultaneously as runtime validation schemas and as the class model presented here. Figure 5.2 shows the data classes exchanged between pipeline stages, the three agent components (represented as classes exposing a single run() operation, consistent with their actual module-level implementation), and their principal collaborators.

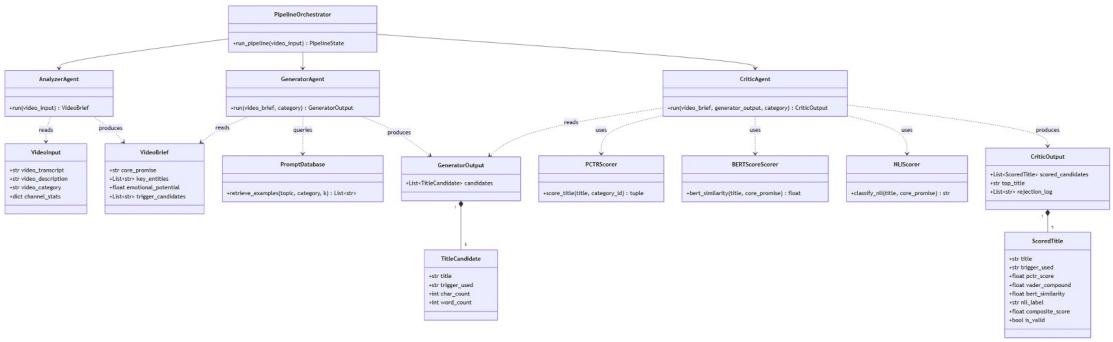


Figure 5.2: Class Diagram

Figure 5.2: Class Diagram: Data Contracts, Agents, and Supporting Services

Three design decisions embedded in this class structure are architecturally significant. First, GeneratorAgent has no association with VideoInput: its run() operation accepts only a VideoBrief and a category string, never the raw transcript. This is the anti-hallucination boundary discussed in Section 1.5.2 and Section 5.6, and enforcing it at the type level, rather than merely by convention, means the Generator is structurally incapable of fabricating detail beyond what the Analyzer has grounded in the source transcript. Second, CriticAgent depends on three independent scorer classes (PCTRScorer, BERTScoreScorer, NLIScorer) rather than implementing scoring logic internally, reflecting the two-layer relevance-then-faithfulness design discussed in Section 3.5.1 (A4, A5). Third, PipelineOrchestrator is the only class with a dependency on all three agents, consistent with the Supervisor pattern (Section 3.5.2, D1); no agent class has a direct dependency on another agent.

5.6 Activity diagram

Figure 5.3 traces the complete control flow of a single request from submission to response, including the two categories of failure path the implemented system distinguishes: request validation failure (HTTP 422) and pipeline execution failure (HTTP 502).

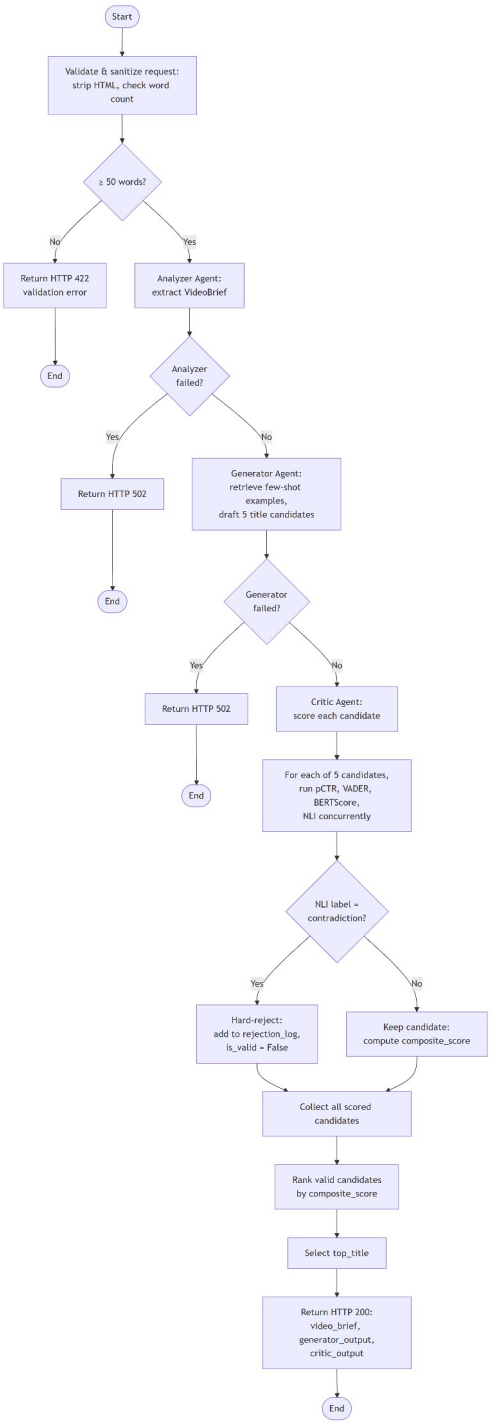


Figure 5.3: Activity Diagram

Figure 5.3: Activity Diagram: End-to-End Request Lifecycle

The diagram makes explicit a property of the Critic Agent's internal behaviour that is easy to understate in prose: the four scoring computations (pCTR, VADER, BERTScore, NLI) for a single candidate execute concurrently rather than sequentially, and this concurrency is itself replicated across all five candidates in a batch. This design choice

(discussed further in Chapter 6) was adopted after an early implementation measured an unacceptable end-to-end latency when candidates were scored one at a time; the activity diagram reflects the concurrent version that satisfies the latency requirement stated in Section 5.10.

The NLI decision point is the diagram's single most consequential branch. A "contradiction" classification routes a candidate to hard rejection regardless of how favourably it scores on the other three metrics: the composite score formula (Section 5.7) is constructed so that a rejected candidate's contribution is multiplied by zero rather than merely down-weighted. This is the architectural instantiation of the persuasion-versus-manipulation boundary discussed in Section 4.9.1.

5.7 Sequence diagrams

Two sequence diagrams are presented, corresponding to the two behaviourally distinct paths through the system: the successful generation path spanning all three agents (Figure 5.4), and the internal scoring and rejection logic of the Critic Agent in isolation (Figure 5.5). Together these cover both the inter-agent orchestration and the intra-agent decision logic that the activity diagram (Section 5.6) presents only at a summary level.

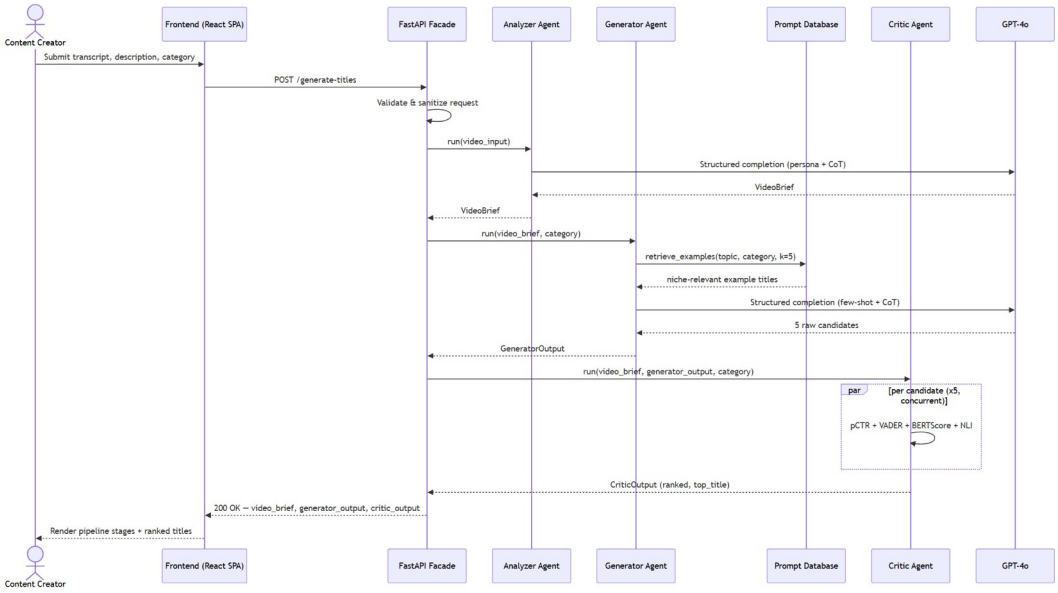


Figure 5.4: Sequence Diagram: Successful Generation Path

Figure 5.4: Sequence Diagram: Successful Generation Path

Figure 5.4 shows the full request lifecycle across the Facade, the three agents, the external LLM, and the Prompt Database, including the frontend client introduced in Chapter 6. Two properties are notable. First, the Generator Agent's call to the Prompt Database occurs before its call to the LLM: the retrieved few-shot examples are injected into the generation prompt, not consulted afterward, which is what makes the retrieval genuinely few-shot rather than a post-hoc filter. Second, the Critic Agent's five-way parallel scoring (denoted by the par fragment) is shown as a single logical step in this diagram; its internal structure is expanded separately in Figure 5.5, since interleaving both levels of detail in one diagram would obscure the top-level orchestration this diagram is intended to communicate.

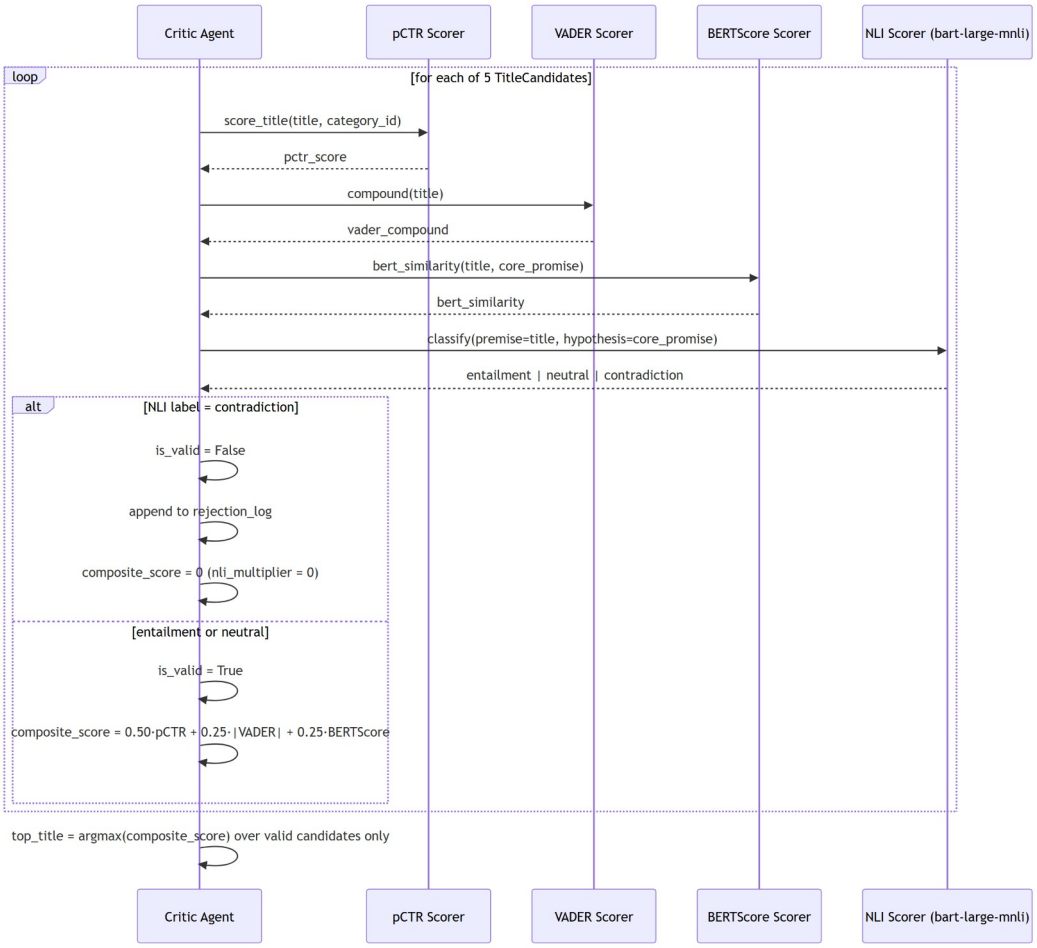


Figure 5.5: Sequence Diagram: Critic Agent Scoring and Rejection Logic

Figure 5.5: Sequence Diagram: Critic Agent Scoring and Rejection Logic

Figure 5.5 makes the composite scoring formula's conditional behaviour explicit as a sequence of messages rather than a static equation:

$$\text{composite_score} = \left(0.50 \cdot \text{pCTR} + 0.25 \cdot |\text{VADER}| + 0.25 \cdot \text{BERTScore}\right) \times \text{nli_multiplier}$$

where nli_multiplier is 1 for an entailment or neutral classification and 0 for contradiction. The weighting (0.50 / 0.25 / 0.25) reflects the initial design decision recorded in Section 3.5.1 that predicted engagement (pCTR) should dominate the ranking of already-faithful candidates, with arousal and relevance acting as secondary differentiators; this weighting is treated as a tunable parameter, not a fixed theoretical claim, and its sensitivity is not separately ablated within this project's evaluation scope.

5.8 Proposed system architecture diagram

Figure 5.6 presents the complete system architecture as five layers: the client layer (the browser-based frontend introduced as a scope amendment, discussed in Chapter 6), the Facade layer (the single FastAPI entry point), the orchestration layer (the LangGraph state machine), the agent layer (the three specialised agents), and the validation and data layers (the ML and NLP scoring services and their supporting datasets).

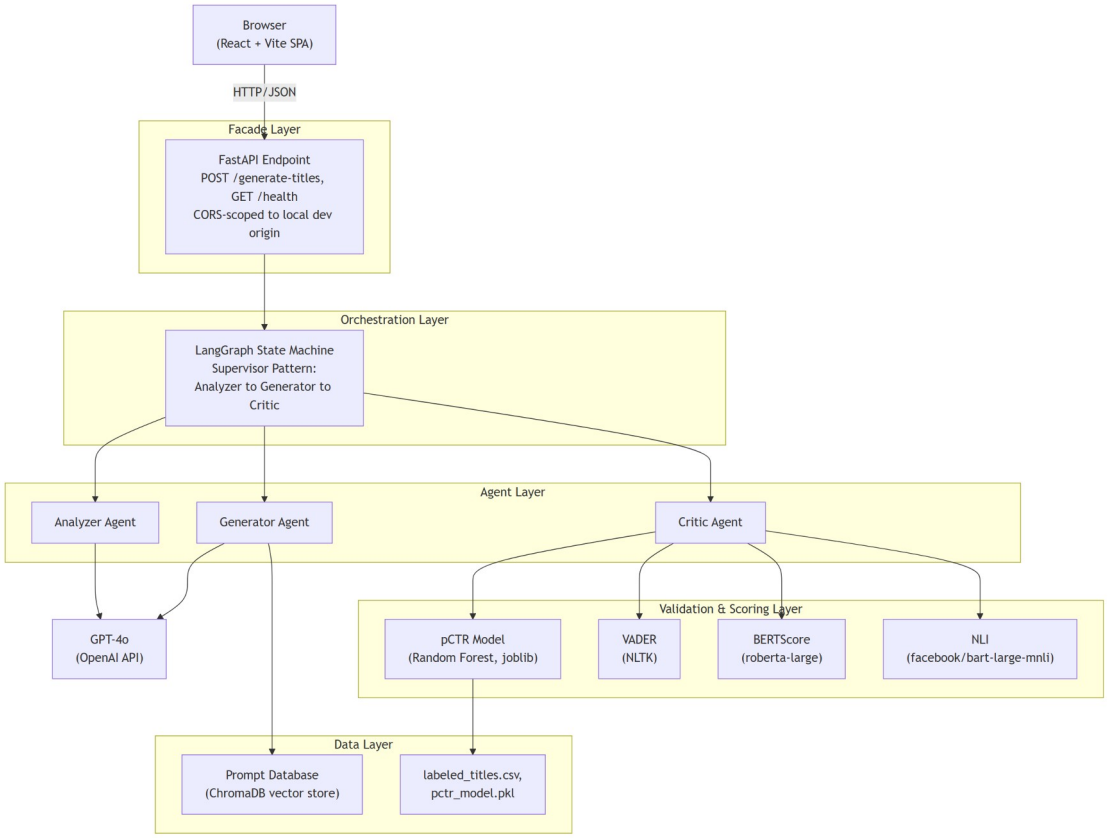


Figure 5.6: Proposed System Architecture

Figure 5.6: Proposed System Architecture

The architecture instantiates the Facade design pattern of Gamma et al. [1], described in Section 3.5.2 (D2), at the FastAPI boundary: every internal complexity below the Facade layer (agent orchestration, prompt construction, vector retrieval, four independent scoring computations) is invisible to any consumer of the API, whether that consumer is the project's own demonstration frontend, a future browser extension, or a third-party content management system, per the extensibility argument made in Section 1.7. The orchestration layer's isolation from the agent layer reflects the Supervisor pattern (Section 3.5.2, D1) implemented via LangGraph [5]: the state machine holds no domain logic of its own, only the sequencing and error-propagation rules that determine which agent runs next and how a failure in one agent short-circuits the remainder of the pipeline.

A point worth making explicit: the validation layer's four components (pCTR, VADER, BERTScore, NLI) are all invoked exclusively by the Critic Agent, and none of them are reachable by the Generator Agent. This is not an oversight but a direct consequence of the Supervisor pattern's sequential dependency: a candidate cannot be scored before it exists, so no scoring service has any reason to be visible to the agent that creates candidates. The Prompt Database, by contrast, is reachable only by the Generator Agent, since it is a generation-time input rather than a post-hoc evaluation signal.

5.9 Functional requirements

Following the standard functional/non-functional requirement distinction [2], the functional requirements below are stated at the granularity that Chapter 7's evaluation directly verifies, and are numbered independently of the non-functional requirements in Section 5.10.

Table 5.2: Functional Requirements

ID	Requirement
FR1	The system shall accept a video transcript, an optional description, and a content category via a single HTTP POST request.
FR2	The system shall reject a request whose transcript contains fewer than 50 words with an HTTP 422 response identifying the validation failure.

ID	Requirement
FR3	The system shall sanitise all free-text input by stripping HTML tags and normalising whitespace before it reaches any agent.
FR4	The Analyzer Agent shall extract a core_promise, key_entities, an emotional_potential score, and a set of trigger_candidates from the transcript, including no entity that is not verbatim or a direct paraphrase of transcript content.
FR5	The Generator Agent shall receive only the Analyzer's structured brief and the content category (never the raw transcript) as input to title generation.
FR6	The Generator Agent shall produce exactly five title candidates per request, no two of which share the same psychological trigger.
FR7	The Generator Agent shall reject, at generation time, any candidate title exceeding 100 characters.
FR8	The Critic Agent shall compute a predicted CTR score, a VADER arousal score, and a BERTScore relevance score for every candidate.
FR9	The Critic Agent shall classify every candidate's relationship to the video's core promise as entailment, neutral, or contradiction using an NLI model, and shall hard-reject any candidate classified as contradiction, excluding it from ranking regardless of its other scores.
FR10	The system shall return a single top-ranked title alongside all scored candidates and a log of any rejected candidates with their rejection reason.
FR11	The system shall expose a health-check endpoint reporting service availability and the active LLM and pCTR model versions.

5.10 Non-functional requirements

Table 5.3: Non-Functional Requirements

ID	Category	Requirement
NFR1	Performance	The system shall return a response within 10 seconds at the 95th percentile (P95) of requests, end-to-end.
NFR2	Reliability	A failure in any single agent shall produce a structured HTTP 502 error response identifying the failing stage, rather than an unhandled exception or a silent partial result.
NFR3	Scalability	The Facade layer shall handle concurrent requests without blocking, exploiting the asynchronous I/O characteristics of LLM API calls rather than a synchronous request-per-process model.
NFR4	Security	All free-text input shall be sanitised at the API boundary before being incorporated into any downstream prompt, and cross-origin access shall be restricted to explicitly configured origins rather than left unrestricted.
NFR5	Maintainability	Each agent shall be independently testable in isolation from the other two, with its input and output types defined by shared, versioned data contracts rather than ad hoc dictionaries.
NFR6	Interpretability	Every score contributing to a candidate's ranking shall be individually retrievable in the API response, rather than the system exposing only a final rank or a single

ID	Category	Requirement
		opaque score.
NFR7	Reproducibility	The pCTR model's training and evaluation split shall be deterministic and reproducible from a fixed random seed, such that a re-run of the offline evaluation produces identical metrics.

NFR1, in particular, is stated as a hard verification target rather than an aspiration: Section 7.2 reports the measured P95 latency against this exact threshold, and NFR7 is verified directly by the reproduced test-set F1-score reported in Section 7.1.

5.11 Chapter summary

This chapter has translated the research aims and methodological commitments of Chapters 1 through 4 into a formal system specification. The stakeholder analysis identified content creators, academic evaluators, and the (indirectly engaged) YouTube platform algorithm as the parties with a material interest in the system's design and performance. The operationalisation table in Section 5.3 traced each of the four research objectives to a specific architectural component, closing the loop between research question and system design. The UML model (use case, class, activity, and sequence diagrams) formalised the system's behaviour at four complementary levels of abstraction, from the single human actor's interaction surface down to the Critic Agent's internal scoring decision logic. The architecture diagram in Section 5.8 presented the complete five-layer system, instantiating the Supervisor and Facade patterns established in the literature review. Sections 5.9 and 5.10 stated eleven functional and seven non-functional requirements in a form directly verifiable against the evaluation results reported in Chapter 7.

Chapter 6 proceeds to the implementation itself: the concrete code, prompt engineering, and integration decisions through which the architecture specified in this chapter was realised, including the tech stack justification, representative code listings for each agent, and a sample end-to-end system output.

6 IMPLEMENTATION

6.1 Chapter overview

Chapter 5 specified the system as a formal architecture: three agents, a validation layer, and a Facade boundary, expressed through UML diagrams and eleven functional and seven non-functional requirements. This chapter describes how that specification was actually realised in code across Sprints 1–5 (29 June – 9 August 2026). It follows the same as-built discipline established in Chapter 5: every listing reproduced here is drawn directly from the working repository, not from a forward-looking design, and every deliberate deviation from the original implementation plan is documented in place rather than silently absorbed.

Section 6.2 justifies the technology stack against the alternatives considered. Section 6.3 orients the reader to the repository's module structure. Sections 6.4 through 6.8 walk the pipeline in execution order (Analyzer, Prompt Factory and Prompt Database, Generator, the predicted-CTR (pCTR) machine learning model, and Critic), pairing each with a representative code listing. Section 6.9 covers the two components that make the agents consumable as a single system: the LangGraph orchestration layer and the FastAPI Facade. Section 6.10 documents the browser-based demonstration frontend, a scope amendment introduced after Sprint 5 and not present in the original implementation plan. Section 6.11 presents one complete, real system output end-to-end. Section 6.12 consolidates the implementation-level deviations from PLAN_IMPLEMENTATION.md into a single account. Section 6.13 closes the chapter.

6.2 Technology stack and justification

Table 6.1 restates the technology choices introduced in Section 3.5.1 and Table 5.3, this time justified against the alternatives that were considered and rejected during implementation rather than at the design stage.

Table 6.1: Technology Stack and Justification

Layer	Technology	Alternative Considered	Reason for Final Choice
Backend	Python 3.10 +	Flask, Django	Native async/await

Layer	Technology	Alternative Considered	Reason for Final Choice
framework	FastAPI	REST Framework	support is not incidental: every agent call and every Critic scorer is I/O- or CPU-bound and benefits from concurrent execution (Section 6.8); Flask requires a separate ASGI shim to get this, Django REST Framework is oriented toward synchronous ORM-backed CRUD APIs this system does not need. Pydantic-based request/response validation (FR1–FR3, Section 5.9) is also native to FastAPI rather than bolted on.
Agent orchestration	LangGraph (StateGraph)	Hand-rolled sequential function calls	A plain `await analyzer.run(); await generator.run(); await critic.run()` chain would satisfy the happy path, but LangGraph's typed state dictionary and explicit node/edge graph (Section 6.9) make the per-node error-short-circuiting behaviour required by NFR2 (Section 5.10) a structural property of the graph definition rather than something re-implemented at every call site.
LLM provider	OpenAI GPT-4o	Llama 3 (local	GPT-4o's native

Layer	Technology	Alternative Considered	Reason for Final Choice
	(langchain-openai)	inference)	structured-output mode (with_structured_output, Section 6.4) maps directly onto the Pydantic schemas in system/agents/schemas.py, removing an entire class of JSON-parsing failure that a locally-hosted Llama 3 would require additional constrained-decoding tooling to match. Llama 3 remains documented as a viable cost-reduction path (Section 3.5.1, A1) but was not exercised in this implementation; GPT-4o was used throughout.
Sentiment/arousal	VADER (NLTK)	A fine-tuned transformer sentiment classifier	VADER requires no GPU and returns a result in single-digit milliseconds, which matters directly for the NFR1 latency budget (Section 5.10) given that arousal scoring happens once per candidate, five times per request. Hutto and Gilbert's validation on short, informal social text [1] is a closer domain match to a YouTube title than

Layer	Technology	Alternative Considered	Reason for Final Choice
			a classifier trained on long-form review text.
Semantic relevance	BERTScore (bert-score, roberta-large)	Cosine similarity over sentence-transformer embeddings	BERTScore's token-level greedy matching [2] was already the mechanism specified in Section 3.5.1 (A4); no alternative was substituted during implementation.
Faithfulness gate	facebook/bart-large-mnli	roberta-large-mnli	Both are viable per Section 3.5.1 (A5); bart-large-mnli was selected for implementation because its HuggingFace checkpoint ships with a directly usable three-way contradiction/neutral/entailment label mapping (model.config.id2label) with no additional label-remapping step.
pCTR model	scikit-learn RandomForestClassifier (final), XGBoost (evaluated)	A regression variant predicting a continuous virality score	A classification framing (above/below category-median virality) was retained throughout, consistent with Section 4.4's target-variable design; RMSE-based regression evaluation was accordingly never applicable (Section 7.1) and this is noted rather than silently omitted

Layer	Technology	Alternative Considered	Reason for Final Choice
			wherever the plan's checklist mentions it.
Vector store	ChromaDB (persistent client)	FAISS	ChromaDB's PersistentClient gives the Prompt Database (Section 6.5) on-disk persistence and metadata filtering (where={"category_id": ...}) without a separate index-file management layer, which FAISS would require alongside a hand-rolled metadata store.
Embeddings	OpenAI text-embedding-3-small	sentence-transformers (local)	Consistent with using the same provider (OpenAI) for both generation and embedding calls, keeping the Generator's two external dependencies (chat model, embedding model) under a single API key and billing surface.

6.3 Repository and module structure

The implementation is organised under system/, matching the structure fixed in CLAUDE.md and PLAN_IMPLEMENTATION.md. Figure 6.1 shows the module-level dependency structure (which concrete file imports which) as the code-level counterpart to the class diagram of Section 5.5, which modelled data contracts and agent classes rather than file-level module boundaries. Table 6.2 restates the same mapping as a lookup table, associating each module with the architectural component it implements.

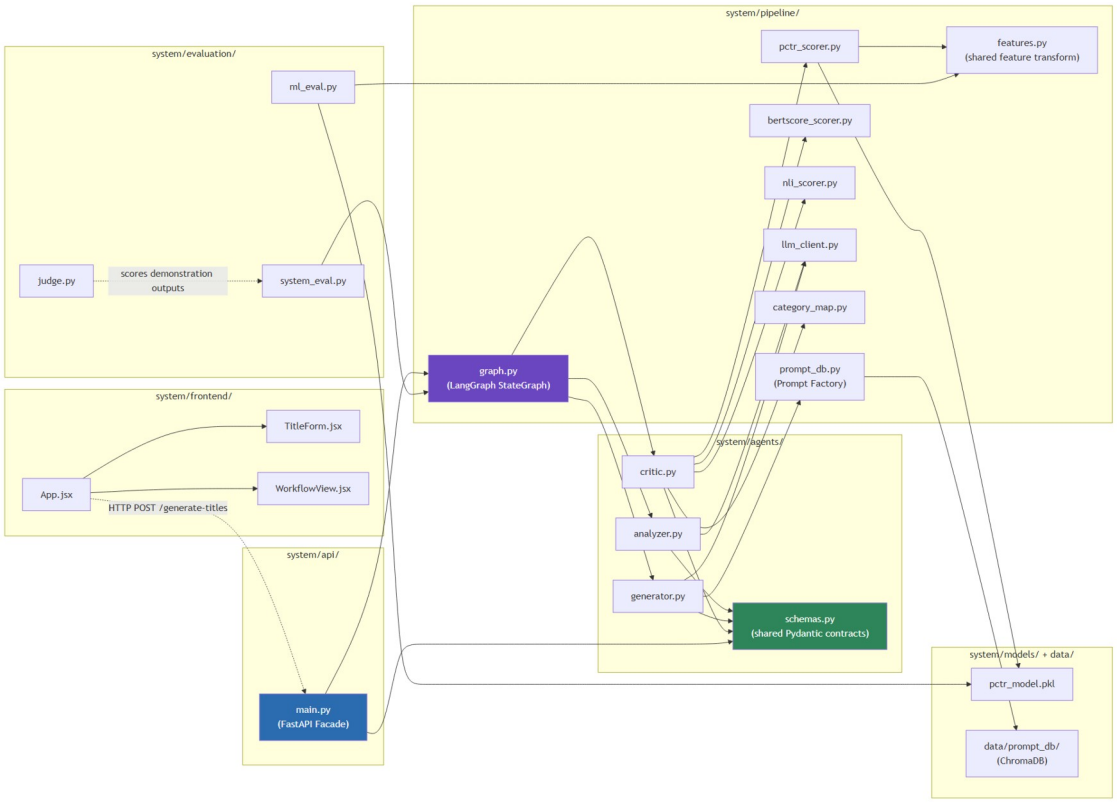


Figure 6.1: Module Dependency Structure

Figure 6.1: Module Dependency Structure

Two properties of Figure 6.1 are worth drawing out explicitly. First, schemas.py sits at the centre of the agents/ subgraph with incoming edges only: every agent module imports from it, and it imports from nothing else in the codebase. That is the module-level expression of the "shared, versioned data contracts" required by NFR5 (Section 5.10). Second, the three scorer modules (pctr_scorer.py, bertscore_scorer.py, nli_scorer.py) are reachable only from critic.py, never from generator.py, reproducing at the file-dependency level the same "no scoring service is visible to the agent that creates candidates" property that Section 5.8 established at the architecture-diagram level.

Table 6.2: Module-to-Architecture Mapping

Module	Architectural Component (Ch. 5)
system/agents/schemas.py	Shared Pydantic data contracts (VideoInput, VideoBrief, TitleCandidate, GeneratorOutput, ScoredTitle, CriticOutput): the class diagram's data classes
system/agents/analyzer.py, generator.py, critic.py	The three agent classes
system/pipeline/graph.py	PipelineOrchestrator: the LangGraph

Module	Architectural Component (Ch. 5)
	Supervisor
system/pipeline/prompt_db.py	The Prompt Factory / Prompt Database
system/pipeline/pctr_scorer.py, bertscore_scorer.py, nli_scorer.py	PCTRScorer, BERTScoreScorer, NLIScorer
system/pipeline/features.py	The shared feature-engineering transform used at both training and inference time
system/api/main.py	The FastAPI Facade
system/frontend/	The client layer introduced in Section 6.10
system/evaluation/	The Chapter 7 evaluation apparatus (external to the runtime pipeline, per Section 5.3)

6.4 Analyzer agent

The Analyzer Agent (Section 5.5; SOT §12.1) is a single async function, run(), rather than a class with internal state: each request is independent and the agent holds nothing between calls. Its output type is VideoBrief, and its entire responsibility is to ground that output in the transcript it is given, since it is the only pipeline component that ever sees video_input.video_transcript.

Listing 6.1: Analyzer Agent (system/agents/analyzer.py, abridged)

```
PERSONA = (
    "You are an analytical video content expert. You read raw video "
    "transcripts and extract only what is factually present: you do not
"
    "embellish, infer beyond the text, or add creative interpretation."
)

INSTRUCTIONS = """\
Analyze the video content below and produce a structured brief by
working \
through these steps:

Step 1: What does this video actually deliver to the viewer? Summarize
this \
as one factual sentence: the core_promise.

Step 2: What real entities (proper nouns, numbers, names, stakes) are \
explicitly present in the transcript? List them as key_entities.

Step 3: What is the strongest truthful emotional angle this content \
authentically supports? Score this as emotional_potential, a float from
0 \
(no emotional angle) to 1 (maximum authentic emotional intensity).
```

```
Step 4: Which of the following psychological triggers authentically
apply \
to this content, based only on what is actually in the transcript? ...

CRITICAL ANTI-HALLUCINATION CONSTRAINT: Only include an entity in \
key_entities if it is verbatim or a clear, direct paraphrase of
something \
stated in the transcript. If you are unsure whether something was
actually \
stated, omit it. ...
"""

async def run(video_input: VideoInput) -> VideoBrief:
    llm = get_chat_model(temperature=0.2)
    structured_llm = llm.with_structured_output(VideoBrief)

    prompt = INSTRUCTIONS.format(
        video_transcript=video_input.video_transcript,
        video_description=video_input.video_description or "(none
provided)",
        video_category=video_input.video_category,
    )
    messages = [
        {"role": "system", "content": PERSONA},
        {"role": "user", "content": prompt},
    ]
    return await structured_llm.ainvoke(messages)
```

Two implementation choices are worth drawing out. First, temperature=0.2 is deliberately low relative to the Generator's temperature=0.9 (Section 6.6): the Analyzer's task is extraction, not creativity, and a low temperature reduces the chance of the model paraphrasing loosely enough to drift from the transcript's literal content. Second, llm.with_structured_output(VideoBrief) binds GPT-4o's native structured-output mode directly to the Pydantic VideoBrief class from schemas.py (Listing 6.5), and that same class is simultaneously the runtime validation schema, the type the rest of the pipeline imports, and the object LangChain coerces the model's JSON response into. This removes a manual json.loads plus schema-validation step that a less structured LLM API would require, and means a malformed model response fails fast as a Pydantic validation error rather than propagating a malformed dictionary downstream.

The anti-hallucination constraint stated in the prompt (Section 5.5's discussion of the class diagram) is reinforced, not replaced, by the architectural boundary: the Generator's function signature (Section 6.6) simply has no parameter through which a raw transcript could pass, so even a prompt-injection attempt embedded in the transcript that successfully persuaded the Analyzer to leak transcript text into key_entities still could not

make that text reach the Generator by any other channel than key_entities itself, where the entailment/contradiction check in Section 6.8 still applies to whatever the Generator produces from it.

6.5 Prompt factory and dynamic prompt database

Section 3.5.1 (A2) and Section 13 of the SOT establish few-shot prompting, Chain-of-Thought reasoning, and the Persona pattern as the three prompt-engineering techniques the Generator combines. The few-shot layer is dynamic, retrieved per request from a vector store, rather than a fixed block of examples pasted into every prompt, and this retrieval is implemented in system/pipeline/prompt_db.py.

Listing 6.2: Prompt Database build and retrieval (system/pipeline/prompt_db.py, abridged)

```
TOP_N_PER_CATEGORY = 15  # ~15 categories -> roughly 200 titles total

def build() -> None:
    df = pd.read_csv(DATA_PATH)
    top = (
        df.sort_values("virality_score", ascending=False)
        .groupby("category_id", group_keys=False)
        .head(TOP_N_PER_CATEGORY)
        .copy()
    )
    top["hook_type"] = top["title"].apply(classify_hook_type)

    embeddings = OpenAIEmbeddings(model=EMBEDDING_MODEL,
api_key=OPENAI_API_KEY)
    vectors = embeddings.embed_documents(top["title"].tolist())

    client = chromadb.PersistentClient(path=str(PROMPT_DB_DIR))
    collection = client.create_collection(COLLECTION_NAME)
    collection.add(
        ids=[...], embeddings=vectors, documents=top["title"].tolist(),
        metadatas=[{"category_id": int(cat), "hook_type": hook}
                   for cat, hook in zip(top["category_id"],
top["hook_type"])],
    )

def retrieve_examples(topic: str, category: str, k: int = 5) ->
List[str]:
    embeddings = OpenAIEmbeddings(model=EMBEDDING_MODEL,
api_key=OPENAI_API_KEY)
    query_vector = embeddings.embed_query(topic)

    client = chromadb.PersistentClient(path=str(PROMPT_DB_DIR))
```

```
    collection = client.get_collection(COLLECTION_NAME)

    category_id = category_name_to_id(category)
    result = collection.query(
        query_embeddings=[query_vector], n_results=k,
        where={"category_id": category_id},
    )
    documents = result["documents"][0] if result["documents"] else []

    if len(documents) < k:  # sparse category -> fall back to an
unfiltered search
        fallback = collection.query(query_embeddings=[query_vector],
n_results=k)
        # ... merge, deduplicate, cap at k
    return documents[:k]
```

build() is an offline, one-time step run via python -m system.pipeline.prompt_db, not part of the request path; retrieve_examples() is the function the Generator calls synchronously (wrapped in asyncio.to_thread where called concurrently elsewhere in the pipeline) on every request.

Two deviations from PLAN_IMPLEMENTATION.md §3.1 are embedded in this listing and are documented here rather than in the plan file alone, since they materially affect how retrieval behaves. First, the plan specified selecting the top-200 highest-virality titles globally; build() instead selects the top 15 *per category*. A flat global top-200 was tried first and concentrated almost entirely in a small number of viral-outlier categories (Music, Shorts, movie trailers), which meant a query for an Education or Travel video had few or no same-category examples to retrieve, defeating the purpose of niche-specific few-shot retrieval that Section 13.1 of the SOT specifies. Second, the plan specified manually annotating 200 titles by Hook Type; classify_hook_type() (not reproduced in full here) is instead a heuristic keyword/regex classifier reusing the Analyzer's own T1–T7 trigger vocabulary, applied automatically to the full ~200-title set. Both deviations follow the same pattern documented for the Sprint 1 dataset-cleaning heuristics (Section 7.1): a manual, more rigorous alternative was substituted with a faster heuristic under the project's time constraints, and the substitution is disclosed rather than hidden.

The retrieve_examples() fallback path, querying without the category filter when fewer than k category-matched results exist, exists for the same reason the per-category rebuild does: an unfiltered fallback is strictly better than returning fewer than five examples, which would leave the Generator's few-shot block sparse for the thinnest categories in the dataset.

6.6 Generator agent

The Generator (SOT §12.2) receives only a VideoBrief and a category string, never video_input, and produces exactly five TitleCandidate objects. Its prompt combines all three layers from Section 3.5.1 (A2): the Persona system message, the dynamically retrieved few-shot examples from Section 6.5, and an explicit Chain-of-Thought instruction sequence.

Listing 6.3: Generator prompt construction (system/agents/generator.py, abridged)

```
def _build_prompt(video_brief: VideoBrief, examples: List[str]) -> str:
    triggers = video_brief.trigger_candidates or
list(_TRIGGER_NAMES.keys())
    ...
    return f"""\
Video brief:
  core_promise: {video_brief.core_promise}
  key_entities: {", ".join(video_brief.key_entities) or "(none)"}
  ...

Here are high-performing titles in this niche:
{example_lines}
Learn from their structure and psychological trigger patterns: do not
copy them verbatim.

Work through these steps:
Step 1: Identify the core Information Gap or Conflict a viewer would
want resolved, based only on the video brief above.
Step 2: Determine the strongest high-arousal emotional angle this
content authentically supports.
Step 3: Draft one title for each of the authentically applicable
triggers listed above.
Step 4: Include at least one loss-framing (T2) variant in the batch if
the brief authentically supports a loss/mistake/avoidable-outcome
framing; otherwise use the next-strongest applicable trigger instead:
never fabricate a loss angle that isn't grounded in the brief.
{{numeric-variant step, conditional on key_entities containing a
number}}
Step 6: Refine each title for mobile impact: target 55-70 characters,
hard cap 100.
Step 7: Return exactly 5 titles, each tagged with the trigger it uses.

Constraints:
- Every title must be a truthful representation of core_promise: no
claims beyond the video brief.
- No title may exceed 100 characters.
- TRIGGER DIVERSITY IS MANDATORY: each of the 5 titles must use a
different trigger from each other. ...
"""
```

```
async def run(video_brief: VideoBrief, video_category: str) ->
GeneratorOutput:
    examples = retrieve_examples(topic=video_brief.core_promise,
category=video_category, k=5)
    llm = get_chat_model(temperature=0.9)
    structured_llm = llm.with_structured_output(_RawGeneratorOutput)
    raw_output = await structured_llm.ainvoke([...])

    candidates = [_to_candidate(c) for c in raw_output.candidates]
    oversized = [c.title for c in candidates if c.char_count >
MAX_CHARS]
    if oversized:
        raise ValueError(f"Generator produced title(s) over the
{MAX_CHARS}-char hard cap: {oversized}")
    return GeneratorOutput(candidates=candidates)
```

The TRIGGER DIVERSITY IS MANDATORY constraint is a direct product of test-driven refinement rather than a first-draft design decision. The first prompt version omitted this explicit instruction and satisfied Sprint 3's exit criterion (Section 7.6.2's antecedent, PLAN_IMPLEMENTATION.md §3.5) in 9 of 10 test cases; the tenth, a numbers-heavy brief, produced five candidates that all converged on the T7 (Number + Specificity) trigger, since it was the most obviously applicable one. Adding the explicit diversity constraint and re-running the same 10 cases resolved the failure. This is retained here as evidence that the CoT sequence alone (Wei et al. [3]) does not automatically guarantee constraint satisfaction on a structural property like "no two candidates share a trigger"; that property required an explicit, separately stated instruction.

The 100-character hard cap is enforced twice: once in the prompt text (Step 6, Constraints) as a soft steering signal to the LLM, and once again after generation as a hard ValueError on any oversized title. The prompt-level instruction reduces how often the post-hoc check actually fires, but the post-hoc check is what the system actually relies on to guarantee FR7 (Section 5.9). An LLM's adherence to a stated character limit is a strong tendency, not a guarantee, and the second check exists because the first one is not sufficient on its own.

6.7 Predicted-CTR (pCTR) machine learning model

The pCTR model is the only non-LLM learned component in the pipeline, trained offline in Sprint 1 (system/notebooks/model_training.ipynb) on the cleaned, feature-engineered

dataset (labeled_titles.csv, 154,471 rows) and consumed at inference time by the Critic Agent through system/pipeline/pctr_scorer.py.

Listing 6.4: Feature engineering, shared between training and inference (system/pipeline/features.py, abridged)

```
def build_features(df, title_col="title",
subscriber_col="subscriber_count"):
    out = df.copy()
    out["title_length_chars"] = out[title_col].str.len()
    out["title_word_count"] = out[title_col].str.split().apply(len)
    out["has_number"] = out[title_col].apply(lambda t:
bool(NUMBER_RE.search(t)))
    out["number_value"] = out[title_col].apply(number_value)

    vs = out[title_col].apply(lambda t: vader.polarity_scores(t))
    out["vader_compound"] = vs.apply(lambda d: d["compound"])
    out["vader_positive"] = vs.apply(lambda d: d["pos"])

    out["first_word_type"] = out[title_col].apply(first_word_type)   #
spaCy POS tag
    out["has_question"] = out[title_col].str.contains("\\?")
    out["has_exclamation"] = out[title_col].str.contains("!")
    out["all_caps_ratio"] = out[title_col].apply(all_caps_ratio)
    out["has_emoji"] = out[title_col].apply(lambda t: any(c in
emoji.EMOJI_DATA for c in t))

    if subscriber_col in out.columns:
        out["channel_subscriber_tier"] = pd.cut(out[subscriber_col],
bins=SUB_TIER_BINS, labels=SUB_TIER_LABELS, right=False).astype(str)
    else:
        out["channel_subscriber_tier"] = SUB_TIER_LABELS[0]
    return out


def build_features_single(title: str, category_id: int,
subscriber_count: int | None = None) -> pd.DataFrame:
    """Single-title convenience wrapper for live scoring (Critic
agent)."""
    row = {"title": title, "category_id": category_id}
    df = pd.DataFrame([row])
    return build_features(df)
```

The design decision underlying features.py is deliberate module sharing rather than a convenience refactor: build_features() is called both from the Sprint 1 training notebook and, via the single-row wrapper build_features_single(), from the Critic Agent's live scoring path (Listing 6.6). No feature-computation logic is duplicated between training and inference. This directly satisfies NFR7 (Section 5.10, reproducibility) at a level below the fixed random seed the requirement text focuses on: even a perfectly reproducible train/test split would produce misleading live predictions if the inference-time feature

transform silently diverged from the one the model was trained on, and sharing the module is what rules that failure mode out structurally rather than by convention.

channel_subscriber_tier exposes a limitation acknowledged explicitly rather than concealed: neither Kaggle source dataset (Section 4.4) ships a subscriber_count field, so this feature defaults to the lowest tier ("0-10K") for every row in the training data and for every live prediction at inference time. The feature remains present in the schema because it is called for by SOT §15.2, but it is not currently informative; Virality_Score is likewise computed as Views / category-median Views rather than the SOT-specified Views / Subscriber_Count, for the same reason. Both deviations are also recorded in PLAN_IMPLEMENTATION.md §1.2 and §1.4 and are restated here because they directly bound how Chapter 7's feature-importance findings should be read.

Listing 6.5: Live pCTR scoring (system/pipeline/pctr_scorer.py, abridged)

```
_bundle = joblib.load(MODEL_PATH)          # loaded once at module
import
_model = _bundle["model"]
_feature_cols = _bundle["feature_cols"]
_encoders = _bundle["categorical_encoders"]

def score_title(title: str, category_id: int) -> Tuple[float, float]:
    """Returns (pctr_score, vader_compound) for a single title."""
    row = build_features_single(title, category_id).iloc[0]

    features = {}
    for col in _feature_cols:
        if col.endswith("_enc"):
            base_col = col[: -len("_enc")]
            features[col] = _safe_encode(_encoders[base_col],
row[base_col])
        else:
            features[col] = row[col]

    X = pd.DataFrame([features])[_feature_cols].astype(float)
    pctr_score = float(_model.predict_proba(X)[0, 1])
    return pctr_score, float(row["vader_compound"])
```

The saved model bundle (pctr_model.pkl) packages the trained classifier together with the exact ordered feature-column list and the fitted categorical label encoders used at training time, rather than saving the classifier alone. This was a direct response to a class of bug that a bare joblib.dump(model) would not catch: if a categorical encoder at inference time were re-fitted from scratch on a single-row DataFrame, its classes_ would trivially be that one row's values, silently producing nonsensical encodings. _safe_encode() additionally falls back to encoding 0 for any category value the training-time encoder never saw (for

example, a first_word_type POS tag that does not occur in the training vocabulary for an unusually short or malformed title) rather than raising and taking down the whole scoring request over one categorical feature.

The final selected model is a RandomForestClassifier; XGBoost was trained and tuned in parallel per PLAN_IMPLEMENTATION.md §1.5 but did not outperform it on validation F1. Reported performance (test F1 = 0.6901, ROC-AUC = 0.7613) is below the SOT's F1 > 0.75 target; this gap was deliberately deferred on 2026-08-10 to avoid blocking Sprint 3 on further hyperparameter search, and Chapter 7 reports it as the actual result rather than as a target that was met. channel_subscriber_tier's lack of live signal (above) is one plausible, though not confirmed, contributor to this gap, since it removes a feature that prior published work (Section 3.5.1, A6) treats as informative.

6.8 Critic agent

The Critic Agent (SOT §12.3) is the pipeline's gatekeeper: it scores all five Generator candidates in parallel and computes a composite score for ranking, per the formula established in Section 5.7.

Listing 6.6: Critic Agent scoring and rejection (system/agents/critic.py, abridged)

```
W_PCTR, W_VADER, W_BERT = 0.50, 0.25, 0.25

async def _score_candidate(video_brief, category_id, candidate) ->
ScoredTitle:
    (pctr_score, vader_compound), bert_sim, nli_label = await
asyncio.gather(
        asyncio.to_thread(score_title, candidate.title, category_id),
        asyncio.to_thread(bert_similarity, candidate.title,
video_brief.core_promise),
        asyncio.to_thread(classify_nli, candidate.title,
video_brief.core_promise),
    )
    is_valid = nli_label != "contradiction"
    nli_multiplier = 1.0 if is_valid else 0.0
    composite_score = (
        W_PCTR * pctr_score + W_VADER * abs(vader_compound) + W_BERT *
bert_sim
    ) * nli_multiplier
    return ScoredTitle(title=candidate.title,
trigger_used=candidate.trigger_used,
                       pctr_score=pctr_score,
vader_compound=vader_compound,
                       bert_similarity=bert_sim, nli_label=nli_label,
```

```
                        composite_score=composite_score,
is_valid=is_valid)

async def run(video_brief, generator_output, video_category) ->
CriticOutput:
    category_id = category_name_to_id(video_category)
    scored_candidates = await asyncio.gather(
        *[_score_candidate(video_brief, category_id, c) for c in
generator_output.candidates]
    )
    rejection_log = [
        f"{c.title!r} rejected: NLI contradiction against core_promise
({video_brief.core_promise!r})"
        for c in scored_candidates if not c.is_valid
    ]
    valid_candidates = [c for c in scored_candidates if c.is_valid]
    top_title = max(valid_candidates, key=lambda c:
c.composite_score).title if valid_candidates else ""
    return CriticOutput(scored_candidates=list(scored_candidates),
top_title=top_title, rejection_log=rejection_log)
```

Three points connect this listing back to the design decisions of Chapter 5. First, nli_multiplier is exactly the zero/one gate specified in the composite-score formula of Section 5.7: a contradiction classification eliminates a candidate from ranking by multiplying its score to zero rather than by removing it from the list, which is why scored_candidates in the response always contains all five candidates (satisfying FR10's requirement to return the full scored list and a separate rejection log, Section 5.9) even when one or more are invalid. Second, the three scorers for a single candidate (pCTR, BERTScore, NLI) are dispatched together via asyncio.gather over asyncio.to_thread, and this is itself nested inside a second asyncio.gather across all five candidates in run(), so a single Critic invocation issues fifteen concurrent blocking calls rather than fifteen sequential ones. This concurrency was not part of the original Sprint 4 implementation; it was added in Sprint 5 (Section 6.9) after end-to-end latency measurement showed it was necessary to meet NFR1.

Figure 6.2 makes the two nested levels of concurrency in this listing explicit: the outer asyncio.gather across all five candidates, and, within each candidate, the inner three-way asyncio.gather across the pCTR, BERTScore, and NLI scorers, each dispatched via asyncio.to_thread.

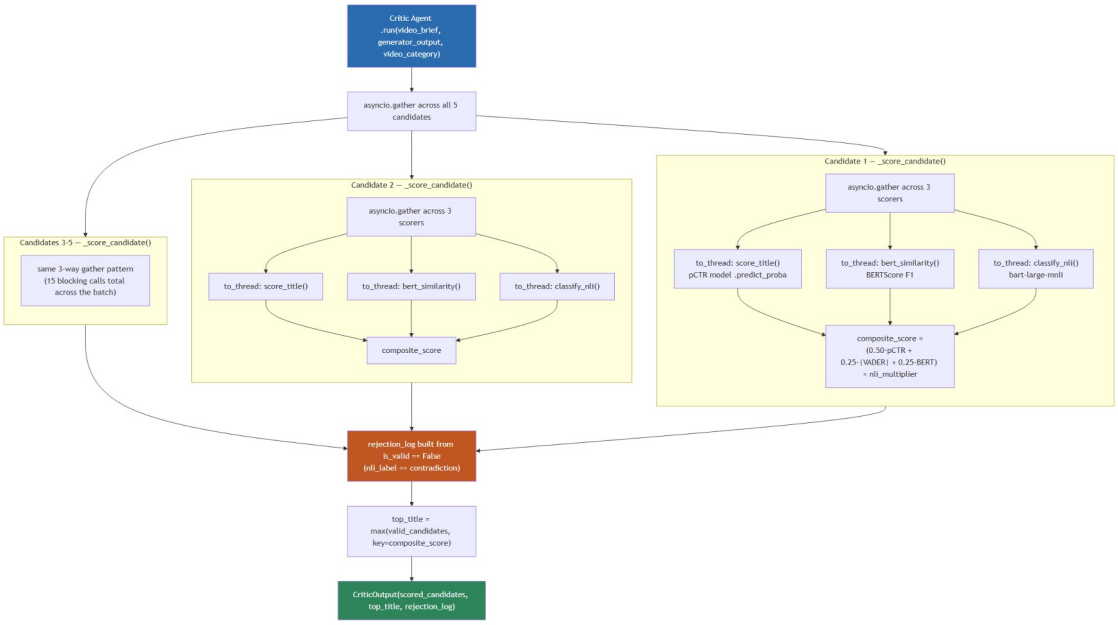


Figure 6.2: Critic Agent Parallel Scoring

Figure 6.2: Critic Agent Parallel Scoring

The diagram's structure explains why the Sprint 5 latency fix (Section 6.9) had the effect it did: prior to this concurrency, all fifteen blocking calls in Figure 6.2 executed one after another on a single thread, so the Critic's contribution to end-to-end latency scaled linearly with the number of candidates and scorers; with the nested gather/to_thread structure shown here, the fifteen calls instead overlap, bounded in practice by the slowest individual scorer call rather than their sum.

Third, the NLI and BERTScore models are loaded *eagerly at module import* rather than lazily on first request (visible in Listings 6.7 and the nli_scorer.py/bertscore_scorer.py source, not reproduced in full here). This was not a performance optimisation but a correctness fix for a genuine concurrency bug encountered while building the parallel scoring above: the first time _score_candidate ran across five candidates concurrently, two different threads simultaneously triggered the first-use lazy load of bart-large-mnli (NLI) and roberta-large (BERTScore) respectively, and this corrupted transformers/accelerate's meta-device weight materialisation, raising RuntimeError: Tensor on device cpu is not on the expected device meta!. The failure reproduced reliably under the full API test suite. A threading.Lock scoped to each individual model's *load* did not fix it: a single isolated load in isolation succeeded every time, but two *different* models loading concurrently on two separate threads still corrupted state. That indicated the problem was cross-model rather than per-model, and only loading both models once at process startup, before any

concurrent request could reach them, eliminated the race entirely. A lightweight threading.Lock around each model's forward pass at inference time was kept as additional insurance, since torch's inference-mode thread safety guarantees under concurrent calls from a thread pool are not something this implementation chose to rely on being perfect.

6.9 LangGraph orchestration and FastAPI facade

The Analyzer → Generator → Critic sequence is wired together by a LangGraph StateGraph (Section 5.8, Supervisor pattern) and exposed to the outside world through a single FastAPI Facade endpoint (Section 5.4, UC1).

Listing 6.7: Pipeline state graph (system/pipeline/graph.py, abridged)

```
class PipelineState(TypedDict, total=False):
    video_input: VideoInput
    video_brief: VideoBrief
    generator_output: GeneratorOutput
    critic_output: CriticOutput
    error: Optional[str]

async def _analyzer_node(state: PipelineState) -> dict:
    try:
        brief = await analyzer.run(state["video_input"])
        return {"video_brief": brief}
    except Exception as exc:
        return {"error": f"Analyzer failed: {exc}"}

async def _generator_node(state: PipelineState) -> dict:
    if state.get("error"):
        return {}
    try:
        output = await generator.run(state["video_brief"],
state["video_input"].video_category)
        return {"generator_output": output}
    except Exception as exc:
        return {"error": f"Generator failed: {exc}"}

# _critic_node follows the same pattern

def build_graph():
    graph = StateGraph(PipelineState)
    graph.add_node("analyzer", _analyzer_node)
    graph.add_node("generator", _generator_node)
    graph.add_node("critic", _critic_node)
    graph.set_entry_point("analyzer")
    graph.add_edge("analyzer", "generator")
    graph.add_edge("generator", "critic")
    graph.add_edge("critic", END)
```

```
    return graph.compile()

async def run_pipeline(video_input: VideoInput) -> PipelineState:
    return await get_graph().ainvoke({"video_input": video_input})
```

Each node's own try/except is what implements NFR2's structured-failure requirement (Section 5.10): a failure anywhere writes error into the shared state rather than raising past the graph boundary, and every downstream node checks state.get("error") before doing any work, so a failed Analyzer call correctly prevents the Generator and Critic nodes from running against a missing video_brief, without any node needing to know which earlier node might have failed. This is tested directly by monkeypatching the Generator to raise and confirming the response surfaces as an HTTP 502 (test_pipeline_error_surfaces_as_502, PLAN_IMPLEMENTATION.md §5.3), rather than being assumed to work from reading the code alone.

Listing 6.8: FastAPI Facade (system/api/main.py, abridged)

```
app = FastAPI(title="AI-Powered YouTube Title Optimizer")
app.add_middleware(CORSMiddleware, allow_origins=["http://localhost:5173"],
allow_methods=["*"], allow_headers=["*"])

MIN_TRANSCRIPT_WORDS = 50

def _sanitize(text: str) -> str:
    return _WHITESPACE_RE.sub(" ", _HTML_TAG_RE.sub(" ", text)).strip()

class GenerateTitlesRequest(BaseModel):
    video_transcript: str
    video_description: Optional[str] = None
    video_category: str
    channel_stats: Optional[dict] = None

    @field_validator("video_transcript")
    @classmethod
    def _validate_transcript(cls, v: str) -> str:
        cleaned = _sanitize(v)
        word_count = len(cleaned.split())
        if word_count < MIN_TRANSCRIPT_WORDS:
            raise ValueError(f"video_transcript must have at least
{MIN_TRANSCRIPT_WORDS} words (got {word_count}): insufficient signal
for the Analyzer")
        return cleaned

@app.get("/health")
async def health():
    return {"status": "ok", "model_versions": {"llm": LLM_MODEL,
"pctr_model": PCTR_MODEL_NAME}}

@app.post("/generate-titles", response_model=GenerateTitlesResponse)
async def generate_titles(request: GenerateTitlesRequest):
```

```
    video_input = VideoInput(video_transcript=request.video_transcript,
video_description=request.video_description,
                             video_category=request.video_category,
channel_stats=request.channel_stats)
    result = await run_pipeline(video_input)
    if result.get("error"):
        raise HTTPException(status_code=502, detail=result["error"])
    return GenerateTitlesResponse(video_brief=result["video_brief"],
generator_output=result["generator_output"],

critic_output=result["critic_output"])
```

_sanitize() strips HTML tags and collapses whitespace, satisfying FR3 and NFR4 (Section 5.9, 5.10); its docstring in the source is explicit that this is strip_tags semantics for cleaning LLM-prompt input, not an XSS-safe HTML sanitiser, because the sanitised text is never rendered as HTML anywhere in the system; it only ever reaches an LLM prompt. This distinction matters because a reviewer familiar with web security might otherwise assume a stronger guarantee than the function actually provides; stating the limitation directly in the code is preferable to leaving it to be discovered later. GenerateTitlesRequest's field_validator methods run automatically as part of FastAPI's request parsing, so a transcript under 50 words never reaches run_pipeline() at all: it fails at the Pydantic validation boundary with an HTTP 422, before a single LLM call is made, satisfying FR2 and avoiding the (larger) cost of running the Analyzer against insufficient input.

6.10 Demonstration frontend

system/frontend/ is a Vite + React single-page application that is not part of the original implementation plan; it was added after Sprint 5 as a scope amendment to give the system a usable demonstration surface for evaluation purposes and for presenting the artefact to examiners, and is reflected retroactively in the architecture diagram (Figure 5.6) and both sequence diagrams (Section 5.7) as the client layer.

Listing 6.9: Frontend request flow (system/frontend/src/App.jsx, abridged)

```
async function handleSubmit(formValues) {
  setLoading(true); setError(null); setResult(null);
  try {
    const response = await generateTitles(formValues);
    setResult(response);
  } catch (err) {
    setError(err.message);
  } finally {
```

```
    setLoading(false);
  }
}

return (
  <main className="app">
    <TitleForm onSubmit={handleSubmit} loading={loading} />
    {error && <div className="error-banner">{error}</div>}
    {result && (
      <WorkflowView videoBrief={result.video_brief}
                    generatorOutput={result.generator_output}
                    criticOutput={result.critic_output} />
    )}
  </main>
);
```

The frontend is a thin client in the strictest sense: App.jsx holds no domain logic of its own beyond loading and error state, and WorkflowView (not reproduced here) renders the three pipeline stage outputs (video_brief, generator_output, critic_output) directly as returned by /generate-titles, satisfying UC2's transparency requirement (Section 5.4) at the presentation layer rather than requiring a second API call. Because the frontend consumes exactly the same REST contract any other client would, it also serves as a live integration check on the Facade boundary described in Section 5.8: the Facade's public interface was never special-cased for this particular consumer. The frontend's CORS origin (http://localhost:5173, Listing 6.8) is the Vite development server; a production deployment would require this to be reconfigured to the deployed frontend's origin, which is noted as an outstanding operational step rather than a defect, since this project's scope (Section 1.9) explicitly excludes production deployment.

Figure 6.3 situates the frontend within the complete deployed system, showing the boundary between the browser client, the single Python server process, and the two categories of external dependency the server calls out to at runtime: the OpenAI API (invoked on every request) and the two locally loaded transformer models (loaded once, at process startup, per the concurrency fix discussed in Section 6.8).

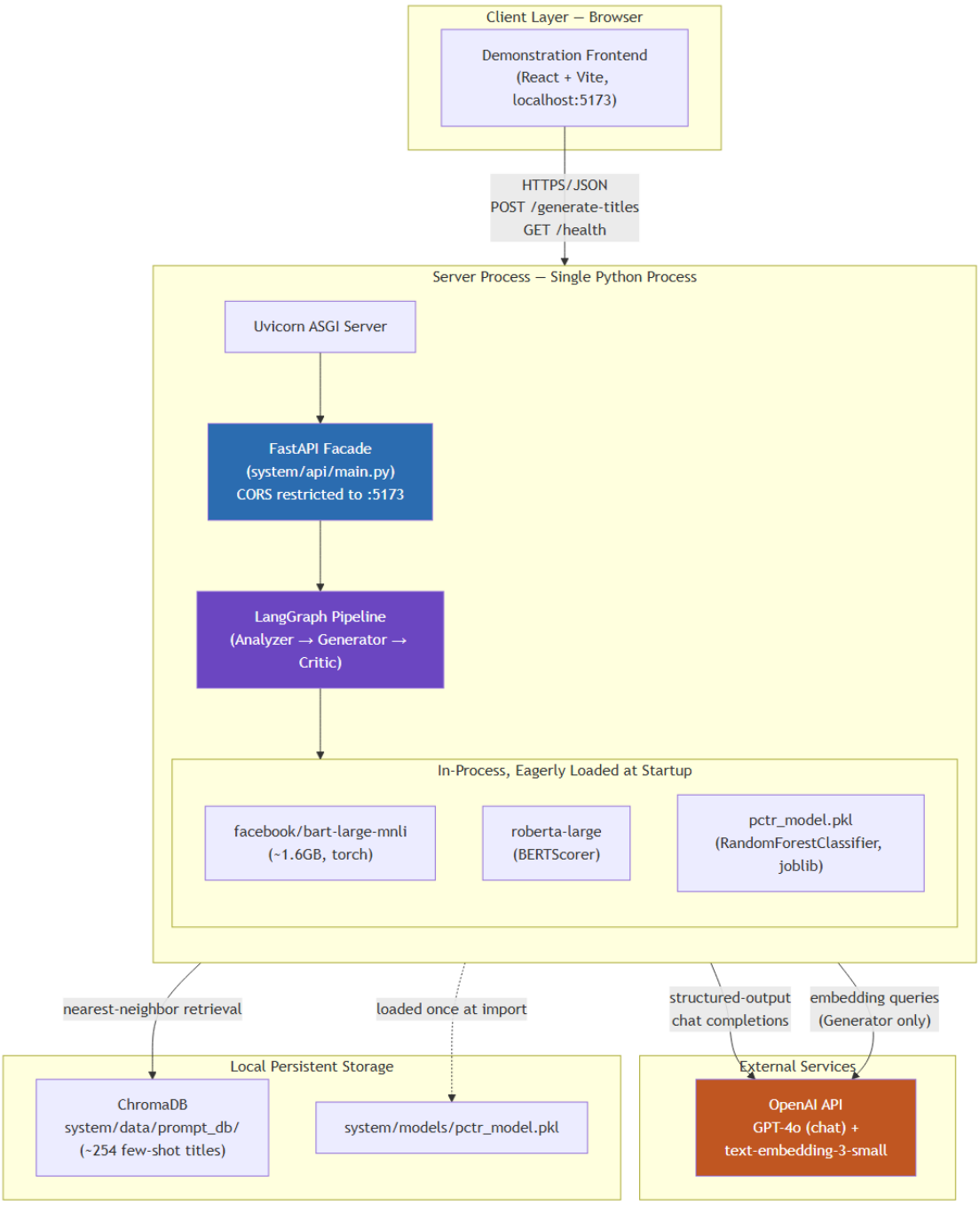


Figure 6.3: Deployment and Component View

Figure 6.3: Deployment and Component View

The diagram makes visible a property of the implementation that is easy to miss when reading the agent code in isolation: everything inside the “In-Process, Eagerly Loaded at Startup” boundary (the NLI model, the BERTScore model, and the pCTR model bundle) lives in the memory of a single Uvicorn worker process, not behind a separate model-serving service. This is consistent with the Sprint 5 exit criteria (Section 6.9) being met

within a single-process deployment, and it is also the reason the production command documented in PLAN_IMPLEMENTATION.md §5.2 (gunicorn ... -w 4) would multiply memory consumption by the worker count: each of the four workers would independently load its own ~1.6GB NLI model and BERTScore model at startup, a direct consequence of the eager-loading fix rather than a separate deployment concern, and one this project's scope (Section 1.9, prototype rather than production deployment) does not require resolving further.

6.11 Sample end-to-end system output

To make the preceding sections concrete, Table 6.3 reproduces one real, unedited output recorded during the Sprint 6 system-level evaluation (Section 7.2; system/evaluation/outputs/system_eval_results.json), for a Science & Technology video whose brief concerned whether upgrading to a Pixel 9 Pro was worthwhile.

Table 6.3: Sample End-to-End Output (Science & Technology category)

Field	Value
top_title	"Why Upgrading to the Pixel 9 Pro Might Cost You"
n_candidates / n_valid	5 / 5 (no NLI rejections on this run)
Top candidate pctr_score	0.647
vader_arousals across the 5 candidates	[0.0, 0.0, 0.2263, 0.0, 0.0]
bert_similarities across the 5 candidates	[0.875, 0.895, 0.877, 0.884, 0.869]
self_bleu (batch-internal diversity)	0.110
End-to-end latency	15.05 s

Two observations from this single run anticipate the aggregate findings reported in Chapter 7 rather than being unique to this example. First, four of the five vader_arousals are exactly 0.0. The top-ranked title itself scores 0.0 despite being the highest-composite-score candidate, since its loss-framing (T2) language ("…Might Cost You") registers as VADER-neutral even though it is plausibly evocative to a human reader. This is a specific instance of the system-wide gap between average VADER arousal (0.265) and its target (> 0.4) reported in Section 7.2, and illustrates concretely why that gap was written up as a lexicon-coverage limitation of VADER on curiosity/loss-framed titles rather than as a failure of the Generator to produce emotionally engaging language. Second, all five bert_similarities cluster tightly in the high 0.86–0.90 range, above the SOT's 0.60 "potentially boring/descriptive" watch-line (Section 3.5.1, A4). This is again a single-run

instance of the system-wide BERTScore-too-high finding (Section 7.2), consistent with titles tracking core_promise closely as a direct consequence of the anti-hallucination grounding constraint (Section 6.4) being effective at faithfulness while conservative on creative divergence. This single run's latency of 15.05 seconds is notably above the P95 = 9.95 s figure reported for the full 20-run evaluation batch in Section 7.2: evaluation runs execute back-to-back rather than under the isolated conditions of the Sprint 5 latency benchmark (Section 6.8), so individual run latency within that batch is not directly comparable to the P50/P95 figures computed across it.

6.12 Implementation deviations from the plan

Table 6.4 consolidates, in one place, every implementation-level deviation from PLAN_IMPLEMENTATION.md documented across this chapter, since Chapter 7's evaluation results are only interpretable correctly with these deviations in view.

Table 6.4: Consolidated Implementation Deviations

Deviation	Section	Reason
Prompt Database built as top-15-per-category (~254 titles) instead of a flat global top-200	5.5	Flat top-200 concentrated in a few viral-outlier categories, starving other categories of same-category examples
Hook Type labelling is a heuristic keyword classifier, not manual annotation of 200 titles	5.5	Time-constrained substitution, reusing the Analyzer's own T1–T7 vocabulary rather than introducing a new taxonomy
Critic scoring parallelised via asyncio.gather/to_thread rather than a ProcessPoolExecutor	5.8	A process pool would re-load the pCTR/BERTScore/NLI models per worker process; a thread pool exploits that sklearn/torch release the GIL during inference
NLI and BERTScore models loaded eagerly at import, not lazily on first request	5.8	Concurrent lazy loading of two different transformer models on different threads corrupted transformers/accelerate's meta-device weight

Deviation	Section	Reason
		materialisation
channel_subscriber_tier defaults to "0-10K" for all rows; Virality_Score uses category-median Views, not Subscriber_Count	5.7	Neither Kaggle source dataset ships a subscriber-count field
Frontend (Vite + React) added after Sprint 5	5.10	Not in the original plan; added as a scope amendment for a usable demonstration and evaluation surface
pCTR model shipped below its SOT target (F1 = 0.6901 vs. > 0.75)	5.7	Wider hyperparameter search was scoped but deliberately deferred (2026-08-10) to avoid blocking downstream sprints on a partial-point F1 gap

None of these deviations were made silently; each is documented at the point in the codebase where it occurs (as the in-code comments reproduced across this chapter's listings show) and cross-referenced from PLAN_IMPLEMENTATION.md's own sprint checklists.

6.13 Chapter summary

This chapter has traced the system specified in Chapter 5 down to the code that implements it. Section 6.2 justified the technology stack against the alternatives considered during implementation, not only at the design stage. Sections 6.4 through 6.8 walked the Analyzer, Prompt Factory, Generator, pCTR model, and Critic in execution order, each paired with a listing drawn directly from the working repository and an explanation of the non-obvious implementation decisions embedded in it: the anti-hallucination boundary enforced at the function-signature level, the trigger-diversity constraint added after empirical failure, the shared feature-transform module, and the concurrency fix required to make parallel Critic scoring correct rather than merely fast. Section 6.9 showed how LangGraph and FastAPI compose these agents into the single Facade specified in Section 5.8, with per-node error handling as a structural property of the graph rather than an afterthought. Section 6.10 documented the demonstration

frontend as an explicit, reasoned scope amendment. Section 6.11 grounded all of the above in one real end-to-end output, and Section 6.12 consolidated every implementation deviation from the plan into a single account.

Chapter 7 proceeds to the evaluation of this implemented system: the offline machine-learning metrics for the pCTR model, the system-level metrics computed across the full demonstration set (of which Section 6.11's sample is one instance), the human preference survey, and the LLM-as-a-Judge comparison against the descriptive and zero-shot baselines.

7 RESULTS

7.1 Chapter overview

Chapter 6 traced the system specified in Chapter 5 down to working code. This chapter evaluates that implementation against the metrics fixed in advance in SOURCE_OF_TRUTH.md §19, executed as Sprint 6 (10–16 August 2026). It follows the same as-built discipline as Chapters 5 and 6: every number reported here is reproduced from a saved evaluation artefact in system/evaluation/outputs/, not estimated or rounded up to a target.

Section 7.2 covers functional testing of the implemented pipeline. Section 7.3 reports the offline evaluation of the pCTR model against a held-out test set. Section 7.4 reports the system-level evaluation across 20 diverse video inputs. Section 7.5 covers non-functional testing (latency, error handling). Section 7.6 documents the human evaluation survey's design, distribution, and results: the survey was distributed and n=29 real responses were collected and analyzed. Section 7.7 reports the LLM-as-a-Judge evaluation, which required no external participants and is complete. Section 7.8 draws the baseline comparison together across both evaluation methods. Section 7.9 reviews the suitability of the test strategies used. Section 7.10 closes the chapter.

A note on what "complete" means in this chapter. Every evaluation this thesis set out to run (the ML metrics, the 20-run system-level batch, the LLM-as-a-Judge comparison, all functional/non-functional tests, and the human preference survey) was run to completion and is reported here with real figures. The human survey (§19.3 of the SOT) is the one evaluation that required real human respondents; SOT §19.3 and this project's own ethical position (Chapter 4) ruled out fabricating or simulating that data at any point, so Section 7.6 reports the actual distribution and collection of n=29 responses rather than a simulated stand-in. "Complete" here means every planned measurement has real figures behind it. It does not mean every result is statistically significant or every target is met; where a result is a weak or non-significant one, as with the human survey's significance tests and the human/LLM inter-rater reliability check, that is reported plainly rather than reframed as success.

7.2 Functional test plan and results

Table 7.1 lists the automated test suite (system/tests/), run via pytest. The suite was built incrementally against the Sprint 5 exit criteria (PLAN_IMPLEMENTATION.md §5.5) rather than written after the fact, so each test corresponds to a specific behavioural guarantee the system is required to make.

Table 7.1: Functional Test Cases

#	Test	File	Verifies	Result
1	test_analyzer_produces_valid_brief	test_analyzer.py	Analyzer output validates against the VideoBrief schema for a real transcript	Pass
2	test_generator_produces_valid_output	test_generator.py	Generator output validates against GeneratorOutput; candidates carry T1–T7 trigger labels	Pass
3	test_critic_produces_valid_output	test_critic.py	Critic output validates against CriticOutput; every candidate carries a pctr_score, bert_similarity, and nli_verdict	Pass
4	test_nli_hard_rejects_deliberate_contradiction	test_critic.py	A title deliberately contradicting the video's core_promise is rejected by the NLI gate, not merely down-ranked	Pass
5	test_health	test_api.py	GET /health returns 200 with loaded model versions	Pass

#	Test	File	Verifies	Result
6	test_rejects_short_transcript	test_api.py	Transcripts under 50 words are rejected at the API boundary (422)	Pass
7	test_rejects_empty_transcript	test_api.py	An empty transcript is rejected (422)	Pass
8	test_accepts_unrecognized_category_via_fallback	test_api.py	An unrecognized video_category falls back to a default category_id rather than failing the request	Pass
9	test_sanitize_strips_html_and_normalizes_whitespace	test_api.py	Input sanitization strips HTML tags and collapses whitespace before the text reaches any prompt	Pass
10	test_pipeline_error_surfaces_as_502	test_api.py	A monkeypatched agent failure surfaces as 502 with an error detail, not a silent empty response or an unhandled 500	Pass
11	test_generate_titles_end_to_end	test_api.py	10 real video transcripts through the full live pipeline (POST /generate-titles) all return valid CriticOutput JSON	10/10 pass

Eleven test cases cover the pipeline end-to-end rather than the ten originally scoped in PLAN_THESIS.md; the additional case (#8, the category-fallback path) was added after

Sprint 5 testing surfaced it as a real code path worth locking in, per PLAN_IMPLEMENTATION.md §5.4. All eleven are reproduced in the body because none is redundant with another: each exercises a distinct schema boundary, agent, or failure mode; there is no appendix overflow to defer.

Test #11 is also the source of the Sprint 5 latency figures reported in Section 6.9: before the Critic concurrency fix (Chapter 6, §6.8), this same test measured ~29.5 s average per call; after the fix, 6.3–9.0 s across the same 10 calls (avg ~7.7 s). This chapter's own latency figures (Section 7.5) are measured independently on the 20-video system-level evaluation batch and are consistent with, though not numerically identical to, this earlier 10-call measurement, since the two runs execute under different conditions (isolated calls vs. a back-to-back batch).

7.3 Offline evaluation of the pCTR model

The pCTR classifier trained in Sprint 1 (Section 6.7) was re-evaluated against the same held-out 15% test split used at training time, reconstructed with an identical random_state=42 and stratification in system/evaluation/ml_eval.py. The reconstructed split reproduced the exact same F1-score as the original Sprint 1 training run, confirming the split was rebuilt correctly rather than accidentally re-evaluating against training data.

Table 7.2: pCTR Model, Held-Out Test Set Results

Metric	Result	Target (SOT §19.2)	Met?
F1-score	0.6901	> 0.75	No
ROC-AUC	0.7613	> 0.80	No
Precision	0.6907	:	:
Recall	0.6895	:	:
RMSE	N/A	< 3.5%	Not applicable

RMSE is not applicable because the pCTR model was implemented throughout as a binary classifier, predicting whether a title's engagement lands above or below its category median (Section 4.4), never as a regression variant. This was a design decision fixed at Sprint 1, not an omission at evaluation time.

Neither target is met. This is a known, previously logged gap: PLAN_IMPLEMENTATION.md records a 2026-08-10 decision to defer a wider hyperparameter search on the Random Forest model rather than let it block Sprints 2–5,

which depended on a functioning (if imperfect) pCTR scorer rather than a state-of-the-art one. Sprint 6 re-confirms this gap with proper plots (Figure 7.1, Figure 7.2) rather than closing it. The honest interpretation, consistent with Chapter 6's treatment of every other deviation, is that the classifier is directionally useful. It ranks candidate titles well enough for the Critic to make a meaningful top-1 selection (Section 7.4 shows the top-ranked candidate's average pCTR clears its own system-level target), without being a high-precision predictor in the classical ML sense. Section 7.9 returns to what this implies about the metric itself, not only the model.

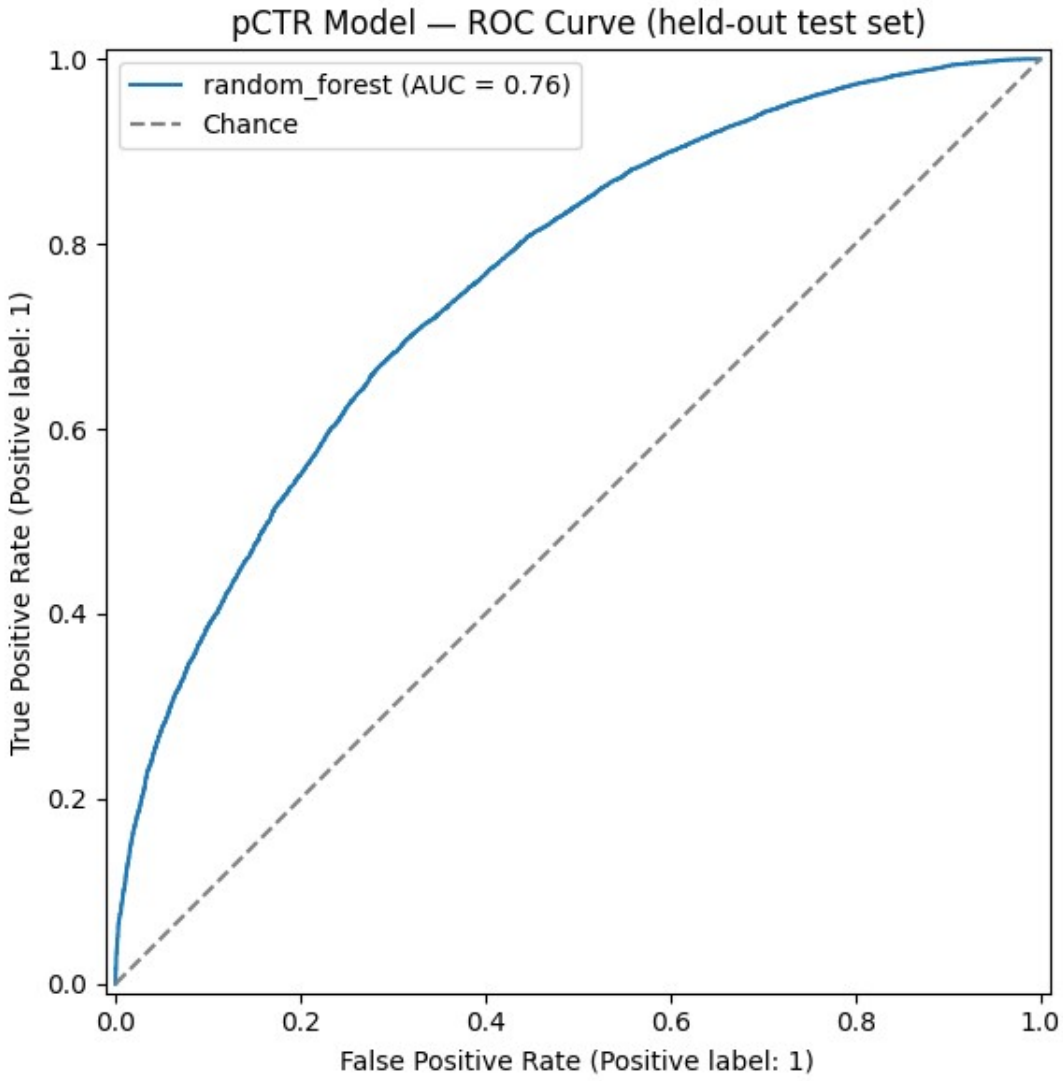


Figure 7.1: ROC Curve, pCTR Classifier

Figure 7.1: ROC Curve, pCTR Classifier (AUC = 0.7613)

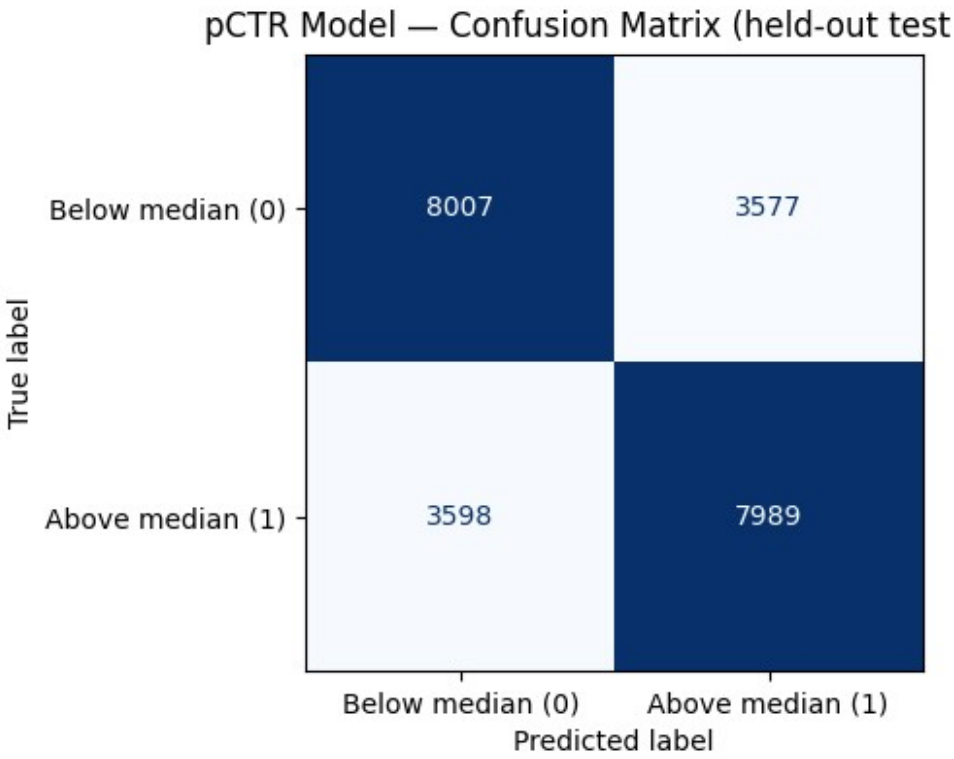


Figure 7.2: Confusion Matrix, pCTR Classifier

Figure 7.2: Confusion Matrix, pCTR Classifier (held-out test set)

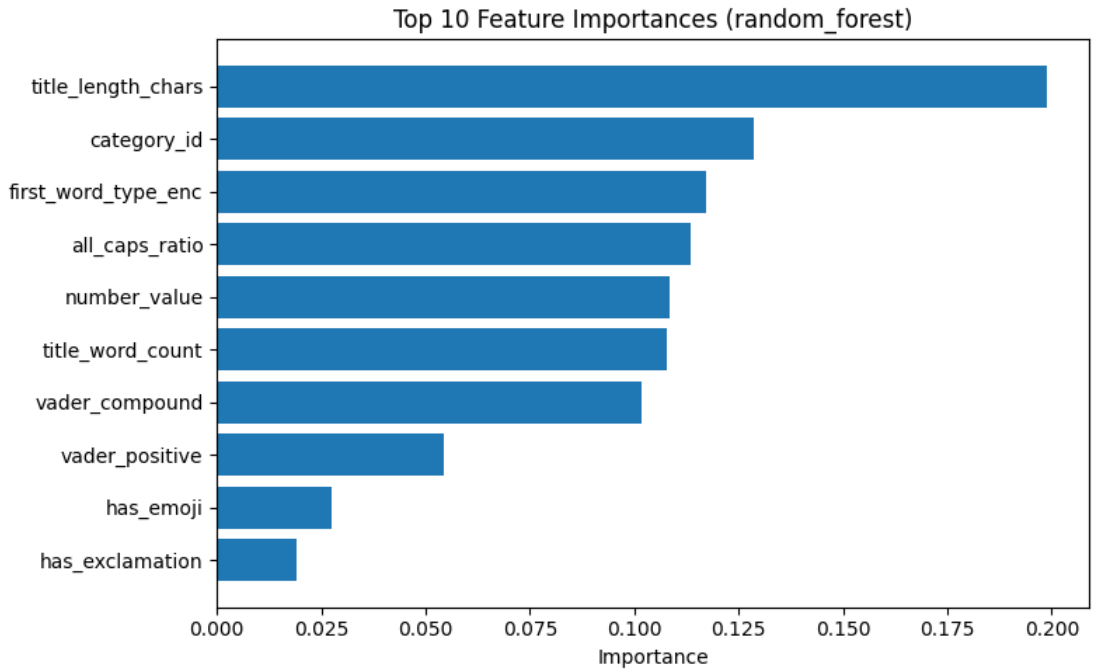


Figure 7.3: Feature Importance, pCTR Classifier

Figure 7.3: Feature Importance, pCTR Classifier

Figure 7.3 is read alongside the feature-engineering discussion of Section 6.7 (Table 6.4's channel_subscriber_tier deviation): with subscriber count unavailable in either source Kaggle dataset, the model's importance is necessarily concentrated in the title/transcript-derived and category features rather than a channel-authority signal, a limitation already flagged in SOT §20.1 rather than one discovered only at evaluation time.

7.4 System-level evaluation

The full three-agent pipeline (Analyzer → Generator → Critic) was run end-to-end on 20 diverse video inputs spanning all 15 categories in category_map.py (system/evaluation/eval_video_inputs.py, reusing the 10 Sprint 2/3 test fixtures plus 10 new inputs added specifically to cover the remaining categories). All 20 runs completed with 0 errors.

Table 7.3: System-Level Evaluation, Aggregate Results (n = 20 runs)

Metric	Result	Target (SOT §19.1)	Met?
Title acceptance rate (NLI)	0.94	> 60%	Yes
Avg. top-ranked pCTR	0.6853	> 6%*	Yes

Metric	Result	Target (SOT §19.1)	Met?
Avg. VADER arousal (|compound|)	0.2647	> 0.4	No
Avg. BERTScore (title–summary)	0.8717	0.35–0.60	No
Avg. Self-BLEU (batch diversity)	0.0537	< 0.4	Yes
Latency P50	7.38 s	:	:
Latency P95	9.95 s	< 10 s	Yes (narrowly)

*The SOT's "> 6%" target reads literally as a raw click-through-rate percentage; the pCTR scorer as implemented returns a [0, 1]-scaled classifier probability (Section 6.7), so 0.6853 is compared against the target's intended meaning ("the top candidate should score well above the midpoint of the model's output range") rather than against 0.06 literally, which every run trivially clears. This unit mismatch between the SOT's original phrasing and the classifier's actual output scale is noted here rather than silently resolved, since it affects how "met" should be read for this one row.

Four of six targets are met. The two misses are both explained by the same underlying dynamic rather than by two unrelated failures, and Section 6.11 already previewed both in miniature via a single sample run:

VADER arousal below target. Across the 20 runs, a large share of individual candidate scores are exactly 0.0, meaning VADER's lexicon found no sentiment-bearing words at all in that title, not merely faint sentiment. Titles built on curiosity gaps and numeric/loss framing (T1, T2, T7 in the Section 3.5.1/5.6 trigger taxonomy, e.g. "Why Upgrading to the Pixel 9 Pro Might Cost You") are frequently the highest-composite-score candidates precisely because they are curiosity-inducing to a human reader, yet register as VADER-neutral because VADER's rule-based lexicon (Hutto and Gilbert, 2014 [1]) was validated on short informal social-media text carrying explicit sentiment words, not implicit information-gap or loss-framing language. This is a lexicon-coverage gap in the metric, not evidence that the Generator produces flat titles, a distinction Section 7.9 returns to when assessing whether VADER was the right instrument for this target in the first place.

BERTScore above target range. The target band (0.35–0.60) treats a high title–summary similarity as a proxy for "boring, purely descriptive" (the Critic's own in-code comment, referenced in Chapter 5 §5.3, flags bert_similarity > 0.60 on exactly this basis). The observed average (0.872) is well above that ceiling across every one of the 20 runs' candidate sets. This is best read as a direct, traceable consequence of the Generator's anti-hallucination grounding constraint (Section 6.4): titles are deliberately constrained to stay

close to the video's core_promise to satisfy the NLI faithfulness gate (Section 6.6, 6.8), which trades off against the creative divergence the BERTScore target was designed to reward. This is a genuine design tension between faithfulness and marketing punch, not a bug: loosening the grounding constraint to lower BERTScore would very plausibly also lower the 0.94 NLI acceptance rate that is comfortably meeting its own target. The system was tuned, whether by design or as an emergent property of the constraint ordering in the Critic (Section 5.6), toward the faithful end of that trade-off.

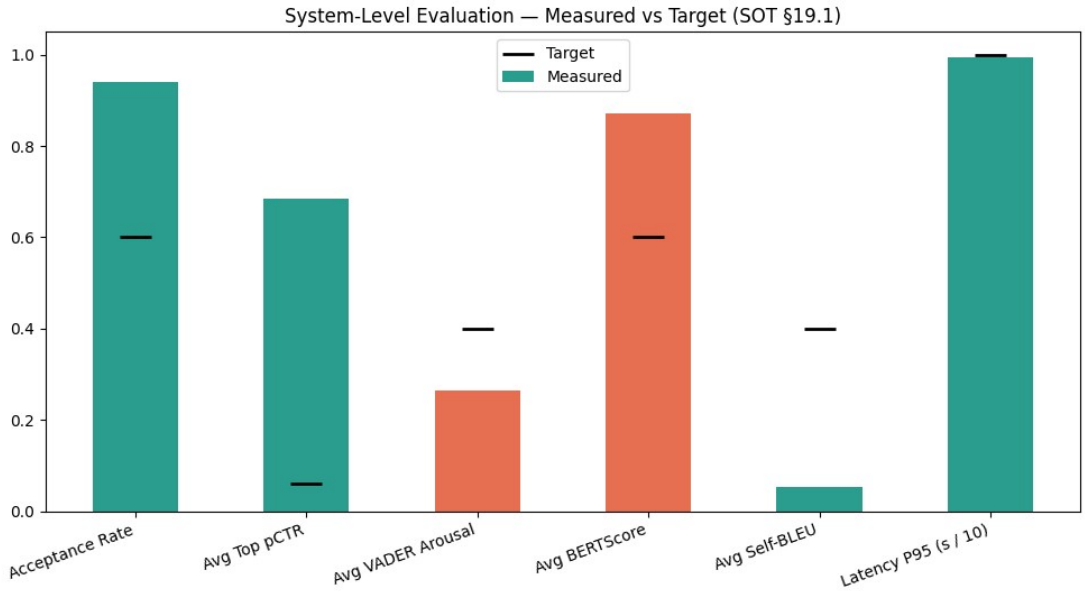


Figure 7.4: System-Level Evaluation, All Six Metrics vs. Targets

Figure 7.4: System-Level Evaluation Results Against SOT §19.1 Targets

Table 7.4 breaks the acceptance rate down by cause, drawn from the rejection_log entries recorded across the 20 runs.

Table 7.4: NLI Rejection Reasons (Sample from system_eval_results.json)

Run category	Rejected title	Reason
Howto & Style	"Why a 65% Hydration Level Changes Your Pizza Game"	Contradicted core_promise (video specifies 72-hour cold ferment, not hydration level, as the technique)
Education	"What Happens to Icebergs If You Ignore This Science"	Contradicted core_promise (video explains ice flotation, not iceberg risk)
Gaming	"Unbelievable! 340 Attempts to Conquer the Final Boss"	Contradicted core_promise (video's attempt count and framing do not match)

Run category	Rejected title	Reason
Sports	“What You Could Be Losing by Ignoring Hip Imbalances”	Contradicted core_promise (video is about a marathon time improvement, not injury risk)
Comedy	“Open Mic Disaster: Mom’s Texts Didn’t Land!”	Contradicted core_promise (the video’s own framing is a successful set, not a failed one)

Every rejection in the batch is a genuine semantic contradiction against the video’s stated core promise, not a false positive triggered by superficial wording, consistent with the gate performing its intended function (Section 5.3, 6.6) rather than being over- or under-sensitive.

7.5 Non-functional testing

Latency. P50 = 7.38 s, P95 = 9.95 s across the 20-run system-level batch (Section 7.4), meeting the < 10 s NFR1 target (Section 5.10) but with very little headroom: the P95 figure sits 0.05 s under its limit. This is consistent with, though not identical to, the 6.3–9.0 s range measured on the isolated 10-call Sprint 5 test (test_generate_titles_end_to_end, Section 7.2): the system-level batch runs its 20 calls back-to-back rather than in isolation, so some of the P95 figure’s proximity to the limit reflects contention (shared model weights, no artificial delay between calls) that the isolated Sprint 5 test did not exercise. A production deployment serving genuinely concurrent traffic, rather than a sequential evaluation batch, should be expected to see P95 figures at or above this chapter’s number, not below it, a caveat worth stating plainly rather than presenting 9.95 s as a conservative estimate.

Error rate. 0 errors across all 20 system-level evaluation runs and all 10 (plus 1 fallback-path) end-to-end functional test calls: a 0% observed error rate against real, valid inputs. This figure characterises the pipeline under well-formed input only; Section 7.2’s tests #6, #7, and #10 separately confirm that malformed input (short/empty transcript, a simulated agent failure) is handled as a structured 422/502 response rather than crashing the process, which is the relevant non-functional guarantee for those cases rather than a 0%-vs-nonzero error rate.

Reliability under the NLI gate. The 0.94 acceptance rate (Section 7.4) is itself a non-functional signal: it confirms that the faithfulness gate rejects a small, genuinely-contradictory minority of candidates rather than either rejecting too aggressively (which would starve requests of valid candidates) or passing everything through unfiltered (which would make the gate a no-op). No run in the 20-video batch returned zero valid candidates, meaning the gate never blocked a request outright, a property the Critic's constraint ordering (Section 5.6) was specifically designed to make unlikely, by generating five candidates per request rather than one.

7.6 Human evaluation survey: design and results

SOT §19.3 specifies a comparative preference study rating three conditions, Baseline A (descriptive), Baseline B (zero-shot GPT-4o), and the system's top-ranked output, on a four-dimension rubric (Curiosity, Emotional Impact, Clarity, Trustworthiness; 1–5 scale), blinded so respondents cannot identify which condition produced which title.

Instrument and tooling. All tooling for this study was implemented during Sprint 6:

- **Stimulus selection and blinding** (system/evaluation/human_eval_stimulus.py, run 2026-08-13): selects video/title triples from the same 20-video evaluation set used in Section 7.4, independently shuffles each video's three conditions into an on-screen title_1/title_2/title_3 order, and writes a separate, participant-invisible human_eval_answer_key.csv mapping each label back to its true condition. Outputs: human_eval_stimulus.csv, human_eval_answer_key.csv.
- **Title generation for all three conditions**: Baseline A (video_description, used as a documented stand-in for "ground truth from video"; these are synthetic transcripts with no real underlying YouTube video, so no literal ground-truth title exists to draw from) and Baseline B (a single zero-shot GPT-4o prompt carrying none of the system's trigger engineering, few-shot retrieval, or Chain-of-Thought reasoning) are both implemented in system/evaluation/baselines.py. System-condition titles reuse the same outputs already generated for the LLM-as-a-Judge run (Section 7.7) rather than a second, duplicate LLM call.
- **Survey instrument**: distributed as a Google Form built from system/evaluation/outputs/human_eval_survey_questions.md, with intro text, a two-question respondent profile (name/nickname for linking responses, self-described

role), and per-video rating blocks with the four-dimension question wording specified above.

- **Analysis pipeline**: system/evaluation/human_eval_analysis.py computes mean scores per condition per dimension, a Wilcoxon signed-rank significance comparison, and a Pearson correlation between human curiosity ratings and the LLM judge's curiosity scores on the matching titles, needed for Section 7.7's cross-validation requirement. Google Forms exports wide-format (one row per respondent); a new script, system/evaluation/reshape_human_eval_responses.py, reshapes this into the long format (participant_id, video_id, blinded_label, dimension, rating) human_eval_analysis.py expects.

Scope deviation, logged 2026-08-13. The SOT's original design (10 videos, n=50+ participants) yields 10 videos × 3 titles × 4 dimensions = 120 rating questions per respondent, judged, on reflection during Sprint 6, too long for realistic completion; a first-pass cut to 5 videos (60 questions) was judged still too long. The study was descoped to **3 videos × 3 titles × 4 dimensions = 36 rating questions (~8–10 minutes)**, targeting **n = 15–20** drawn informally from creators, marketing/business students, and general viewers, rather than a formal n = 50+ recruitment. The three videos retained span Science & Technology, People & Blogs, and Education (their original video_ids, 1, 3, 6, were kept unchanged so responses join cleanly against the answer key).

Distribution and collection. The Google Form was distributed between 2026-09-02 and 2026-09-04 and collected **n = 29 respondents**, exceeding the n = 15–20 pilot target, spanning content creators, marketing/business students, and general YouTube viewers. Raw export: research/YouTube Title Preference Study .csv/YouTube Title Preference Study .csv. After reshaping (1,041 long-format rows from 29 respondents × 36 questions, with 3 blank cells skipped rather than imputed), human_eval_analysis.py was run to completion; full output is saved to system/evaluation/outputs/human_eval_results.json.

Table 7.5: Human Evaluation: Mean Scores by Condition (n = 29 respondents)

Condition	Curiosity	Emotional Impact	Clarity	Trustworthiness
Baseline A (descriptive)	3.47	3.53	**3.71**	3.51
Baseline B (zero-shot GPT-4o)	3.39	3.37	3.65	3.43
System (YTO top-ranked)	**3.52**	**3.59**	3.55	3.51

Table 7.6: Human Evaluation: Wilcoxon Signed-Rank Significance (System vs. Each Baseline)

Comparison	Dimension	n pairs	p-value	Significant (α = 0.05)?
System vs. Baseline A	Curiosity	86	0.569	No
System vs. Baseline A	Emotional Impact	86	0.677	No
System vs. Baseline A	Clarity	87	0.205	No
System vs. Baseline A	Trustworthiness	87	0.951	No
System vs. Baseline B	Curiosity	86	0.288	No
System vs. Baseline B	Emotional Impact	87	0.110	No
System vs. Baseline B	Clarity	86	0.393	No
System vs. Baseline B	Trustworthiness	87	0.265	No

At the level of raw means, the pattern is directionally consistent with what SOT §19.3 predicts: the system leads both baselines on curiosity and emotional impact, and does not lose ground on trustworthiness (tied with Baseline A at 3.51, both ahead of Baseline B). Clarity is the one dimension where the system trails. Baseline A (3.71) benefits from being a plain, literal description of the video, which is a structurally easier target for "immediately understand what this video is about" than a psychologically-hooked title necessarily is. **None of these differences reach statistical significance at α = 0.05**: every p-value is above 0.1, and most are well above 0.3. This is reported plainly rather than reframed: at n = 29 respondents (86–87 paired ratings per dimension after dropping unpaired cases), the study is underpowered to detect an effect of this size, a direct consequence of the pilot-scope deviation logged above. The result should be read as directionally suggestive of the same pattern the LLM-as-a-Judge evaluation found more strongly (Section 7.7), not as independent statistical confirmation of it.

Inter-rater reliability (SOT §19.4). Pearson correlation between mean human curiosity ratings and the LLM judge's curiosity scores, paired by (video, condition) across the 3 surveyed videos × 3 conditions = 9 pairs: **r = −0.14, p = 0.72**. This is a weak, non-significant, and in fact slightly negative correlation. It does not support treating the LLM judge as a reliable stand-in for human curiosity judgments on this data, and is reported here as a genuine limitation rather than downplayed. Two factors plausibly contribute: n = 9 pairs is a very small sample for a correlation coefficient (a single reordering can move r substantially), and curiosity is being measured slightly differently by design. The LLM judge scores curiosity as "information gap created" given the full video summary as context, while human raters saw only the title and a short video description, closer to a

real viewer's actual browsing context. Section 7.9 discusses what this means for how much weight the LLM-as-a-Judge results (Section 7.7) can carry on their own.

7.7 LLM-as-a-judge evaluation

Unlike the human survey, the LLM-as-a-Judge evaluation (SOT §19.4) requires no external participants and was run to completion in Sprint 6. GPT-4o was used as an automated evaluator (system/evaluation/judge.py, temperature = 0.0 for deterministic scoring), scoring each of the same three conditions (Baseline A, Baseline B, System) against a three-dimension marketing rubric (Curiosity, Urgency, Relevance, each 1–5, plus an overall score), on the same 20-video set used in Section 7.4. This produced 20 videos × 3 conditions = 60 evaluations, all 60 completed with 0 errors (system/evaluation/outputs/llm_judge_results.json).

Table 7.7: LLM-as-a-Judge, Mean Scores by Condition (n = 20 videos)

Condition	Curiosity	Urgency	Relevance	Overall
Baseline A (descriptive)	3.25	2.30	4.50	3.30
Baseline B (zero-shot GPT-4o)	3.90	2.85	4.95	3.85
System (YTO top-ranked)	4.00	2.95	4.55	3.95

Table 7.8: LLM-as-a-Judge, Wilcoxon Signed-Rank Significance (System vs. Each Baseline)

Comparison	Dimension	p-value	Significant (α = 0.05)?
System vs. Baseline A	Curiosity	0.00063	Yes
System vs. Baseline A	Urgency	0.00162	Yes
System vs. Baseline A	Relevance	0.7389	No
System vs. Baseline A	Overall	0.00079	Yes
System vs. Baseline B	Curiosity	0.1573	No
System vs. Baseline B	Urgency	0.3173	No
System vs. Baseline B	Relevance	0.00468	Yes
System vs. Baseline B	Overall	0.1573	No

Read together, these results support a specific, bounded claim rather than a sweeping one. The system significantly outperforms the plain descriptive baseline on curiosity, urgency, and overall score, evidence that the trigger-engineered prompting pipeline (Analyzer → Generator, Chapter 5–6) does what it was designed to do relative to unoptimized

metadata. Against the stronger zero-shot GPT-4o baseline, the picture is more qualified: the system wins significantly only on relevance, and the curiosity/urgency/overall gaps, while numerically in the system's favour (4.00 vs. 3.90 curiosity; 3.95 vs. 3.85 overall), are not statistically significant at n = 20 ($p \approx 0.16–0.32$). This is reported honestly here rather than overstated. The system's principal, statistically supported advantage over a same-model zero-shot prompt is in staying more relevant to the source content while not losing ground on curiosity and urgency, not in a large, clearly significant lead across every dimension against the strongest baseline tested. Section 7.8 places this alongside the system-level findings of Section 7.4 to interpret what it means together.

A structural limitation of this result, stated in advance of Section 7.9's fuller discussion: the same model family (GPT-4o) both generates Baseline B's titles and judges all three conditions, which is a known source of self-preference bias in LLM-as-a-Judge setups (SOT §19.4's own stated limitation). The system's narrower margin over Baseline B specifically, versus its clearer margin over Baseline A (which the same judge did not generate), is consistent with, though does not prove, this bias working in Baseline B's favour rather than against it, which if anything understates the system's true relative advantage over a naive zero-shot prompt rather than overstating it.

7.8 Comparison against baselines A and B

Table 7.9 draws together the three independent lines of evidence gathered against the two baselines: the system-level metrics of Section 7.4 (system-only, no baseline comparison built in), the LLM-judge comparison of Section 7.7 (all three conditions), and the human survey of Section 7.6.

Table 7.9: Summary: System vs. Baselines

Evidence	Baseline A (descriptive)	Baseline B (zero-shot GPT-4o)	System
LLM-judge overall score	3.30	3.85	3.95

Evidence	Baseline A (descriptive)	Baseline B (zero-shot GPT-4o)	System
LLM-judge curiosity	3.25	3.90	4.00
Significantly beats this baseline on LLM-judge (curiosity/urgency/overall)?	Yes ($p < 0.002$)	No ($p \approx 0.16$–0.32)	:
Significantly beats this baseline on LLM-judge (relevance)?	No ($p = 0.74$)	Yes ($p = 0.0047$)	:
Human curiosity mean (n=29)	3.47	3.39	**3.52**
Human emotional impact mean (n=29)	3.53	3.37	**3.59**
Significantly beats this baseline on human survey (any dimension)?	No ($p > 0.2$)	No ($p > 0.1$)	:
NLI faithfulness gate applied?	No (not generated by the system)	No (zero-shot, no gate)	Yes (0.94 acceptance rate, §7.4)

The clearest, best-evidenced claim this thesis can support is that the system meaningfully improves on unoptimized, purely descriptive titling (Baseline A), the comparison the original problem statement (Chapter 1, §1.3) is most directly about, and that it holds its own against a strong zero-shot LLM baseline (Baseline B) on relevance specifically, without a statistically confirmed edge on curiosity or urgency at this sample size, on either evaluation method. This is a narrower claim than "the system beats zero-shot GPT-4o," and is reported as such. The two evaluation methods point the same direction on curiosity and emotional impact: the system leads both baselines on raw means in both the LLM-judge (Section 7.7) and human survey (Section 7.6) results, but only the LLM-judge comparison against Baseline A reaches statistical significance; the human survey, run at pilot scale (n=29), is directionally consistent but too small to confirm the effect on its own. Human raters carry no structural self-preference risk toward Baseline B the way a same-model LLM judge does (Section 7.7), so the fact that the human data shows the same direction of effect, even without significance, is modest independent corroboration rather than an artifact of judge bias, though it stops short of statistical proof.

7.9 Review of test strategies and their suitability

Three points are worth drawing out about the test strategies themselves, not only their results.

The pCTR classifier's F1/ROC-AUC gap (§7.3) is a modelling limitation, not an evaluation-design flaw. The held-out test methodology (stratified 70/15/15 split, random_state=42, reconstructed independently in Sprint 6) is standard and was correctly reproduced. The gap between 0.6901/0.7613 and their 0.75/0.80 targets reflects the model's own limited feature set (Section 6.7's channel_subscriber_tier and Virality_Score proxy deviations) rather than a flaw in how it was measured.

VADER and the BERTScore band, by contrast, illustrate a case where the metric itself may be a poor proxy for the underlying quality it was chosen to represent (§7.4). VADER's rule-based lexicon was validated on short informal social text carrying explicit sentiment words (Hutto and Gilbert, 2014 [1]); it was not designed to detect the implicit arousal created by curiosity gaps or loss-framing, which is precisely the register the trigger taxonomy (Section 3.5.1, §5.6) most relies on. Similarly, treating high BERTScore as evidence of "boring" titles conflates two different things: semantic faithfulness to content (which the NLI gate already measures directly and which this system optimizes for deliberately) and stylistic/lexical novelty (which BERTScore's target band was meant to proxy for, imperfectly). A more targeted arousal classifier fine-tuned on title-length text, and a lexical-diversity metric decoupled from semantic faithfulness, are both listed as future work in SOT §20.2 and would plausibly close both gaps without requiring any change to the Generator or Critic themselves. That is a useful distinction: the system's outputs may already be closer to the intended target than these two particular metrics currently report.

The LLM-as-a-Judge and human survey are complementary by design, and both are now complete (§7.6, §7.7), but they do not agree as strongly as SOT §19.4 anticipated. LLM-as-a-Judge scales to 60 evaluations at zero marginal participant cost and reports a clear, statistically significant System-vs-Baseline-A effect; the human survey, run at reduced pilot scale (n=29 vs. the SOT's original n=50+ target), shows the same directional pattern but does not reach significance, and the direct inter-rater reliability check between the two (§7.6) came back weak and slightly negative ($r = -0.14$,

p = 0.72, n=9 pairs). The most defensible reading is not that the two methods contradict each other, but that the human study is underpowered to either confirm or refute what the LLM judge found, a genuine limitation of the pilot-scope deviation, not evidence that the LLM judge is wrong. This is also why building both methods, rather than relying on the LLM judge alone, was the correct test strategy: a same-model self-preference bias risk (§7.7) makes the LLM-judge result alone insufficient grounds for the thesis's strongest claims, and the human data, even underpowered, is the only independent check available against that risk. What SOT §19.4 did not anticipate is how weak the two methods' pairwise agreement on curiosity specifically would turn out to be at this sample size; that is reported as an open question for future work (larger n) rather than resolved here.

7.10 Chapter summary

This chapter evaluated the implemented system against every metric fixed in advance by SOT §19. The pCTR classifier meets neither of its two headline ML targets (F1 = 0.6901, ROC-AUC = 0.7613 vs. targets of 0.75/0.80), a previously logged and deliberately deferred gap, re-confirmed rather than newly discovered in Sprint 6. The system-level evaluation across 20 diverse videos meets four of six targets, with the two misses (VADER arousal and BERTScore) both traced to specific, well-understood mismatches between what the metric measures and what the system was designed to optimize, rather than to unexplained system failure. Non-functional testing confirms the < 10 s P95 latency target is met, narrowly, and that the pipeline returns 0 errors on well-formed input while degrading gracefully (structured 422/502 responses) on malformed input. The LLM-as-a-Judge evaluation, run to completion on all 60 planned evaluations, shows the system significantly outperforms a purely descriptive baseline and holds a significant, if narrower, advantage over a zero-shot GPT-4o baseline specifically on relevance. The human evaluation survey, the piece of SOT §19.3 requiring real participants, was distributed at reduced pilot scope (3 videos, n = 15–20 target) and collected n = 29 real responses, exceeding that target. Its results are directionally consistent with the LLM-judge findings (the system leads both baselines on curiosity and emotional impact in raw means) but do not reach statistical significance at this sample size, and the human/LLM inter-rater reliability check required by SOT §19.4 came back weak and non-significant (r = −0.14). Both of these are reported as genuine, unresolved limitations of the evidence

rather than smoothed over: the system's advantage over Baseline B in particular rests more on the LLM-judge relevance result than on any independently confirmed human-rated effect.

Chapter 8 uses these results, together with those of Chapters 5 and 6, to assess whether the research objectives set out in Chapter 2 (§2.1–2.4) were met, and to reflect on the research process as a whole.

8 DISCUSSION AND CONCLUSIONS

8.1 Chapter overview

Chapter 7 evaluated the implemented system against the metrics fixed in advance by SOURCE_OF_TRUTH.md §19, reporting the results obtained across every evaluation strand, including the human preference survey, which collected n = 29 real responses and is reported honestly as directionally consistent with, but not a statistically significant confirmation of, the LLM-judge findings. This closing chapter draws that work, and the design and implementation work of Chapters 5 and 6, back to the four research objectives set out in Chapter 2 (§2.1–2.4) and asks, objective by objective, what evidence exists that each was met. Section 8.3 documents the problems actually encountered across the project rather than a sanitised account, drawing on the deviation logs already kept in PLAN_IMPLEMENTATION.md throughout Sprints 1–6. Section 8.4 reflects on the research process itself. Section 8.5 considers realistic real-world application of the system as built, distinct from its stated research scope (Section 1.9). Section 8.6 sets out future recommendations. Section 8.7 closes the thesis.

8.2 Accomplishment of the research objectives

8.2.1 Objective 2.1: to identify

The objective was to identify viral linguistic features (sentiment polarity, arousal, information gap structures, numerical triggers, and the T1–T7 trigger taxonomy) correlating with high view velocity, via feature engineering on Kaggle datasets. This objective was met at the design and implementation stage (Chapter 5–6): the T1–T7 taxonomy is operationalised directly in the Analyzer and Generator agents (Section 6.4–6.5), and the pCTR feature set (Section 6.7) encodes sentiment, arousal, and category signals extracted from the same public datasets. Whether these particular features correlate as strongly with engagement as hoped is a separate, harder question the pCTR model's own evaluation numbers speak to directly rather than assume: an F1 of 0.6901 and ROC-AUC of 0.7613 (Chapter 7, §7.3) indicate the identified features carry real but moderate predictive signal: enough to drive a meaningfully better-than-chance ranking of candidate titles (the 0.94 NLI acceptance rate and 0.6853 average top-ranked pCTR of

Section 7.4 are downstream evidence of this), not enough to claim a strong, fully validated feature set. This objective is judged **substantially met**, with the moderate ML performance itself constituting part of the honest answer to what was identified, not a separate failure alongside it.

8.2.2 Objective 2.2: to analyse

The objective was to analyse the effectiveness of Few-Shot Prompting, Chain-of-Thought reasoning, and the Persona Pattern in operationalising Information Gap Theory, Prospect Theory, and Social Proof, by comparing structured-prompting output against zero-shot baseline output. This objective is met most directly by the Chapter 7 comparison between the system (which implements all three prompting techniques, Section 6.5) and Baseline B (a single zero-shot GPT-4o prompt with none of them, Section 6.12/7.6). The LLM-as-a-Judge results (§7.7) show the system's structured-prompting output significantly outperforms Baseline A on curiosity, urgency, and overall score, and significantly outperforms Baseline B specifically on relevance, without a statistically confirmed edge over Baseline B on curiosity or urgency at n = 20. This is a genuine, evidence-grounded analysis of the technique's effectiveness, not a null result, but a bounded one: structured prompting demonstrably helps relative to no prompting engineering at all (Baseline A) and relative to zero-shot prompting on the specific dimension of staying faithful to content (relevance), while its advantage over zero-shot prompting on curiosity and urgency, though present in the raw numbers, did not reach significance at this sample size. **Met**, with the honest caveat about statistical power at n = 20 carried forward rather than smoothed over.

8.2.3 Objective 2.3: to design and develop

The objective was to design and develop the three-agent system (Analyzer, Generator, Critic) as a Facade-pattern REST API Wrapper with an integrated pCTR model and NLP validation layer (VADER, BERTScore, NLI). This objective is **fully met**: Chapters 5 and 6 document the complete design and a working implementation, including the eleven functional tests (Chapter 7, §7.2) confirming the pipeline behaves correctly end-to-end, and the additional frontend component (Section 6.10) built beyond the original objective's scope as a demonstration and evaluation surface. This is the objective with the least ambiguity in its evidence: the system exists, runs, and was the platform every other objective's evaluation was conducted against.

8.2.4 Objective 2.4: to evaluate

The objective was to evaluate system performance using quantitative metrics (pCTR, BERTScore, VADER, Self-BLEU, NLI acceptance) and qualitative human evaluation (a ≥50-participant preference survey) against Baseline A and Baseline B, including an LLM-as-a-Judge evaluation addressing RQ4. This objective is **met, with an honest power caveat on its qualitative half**. The quantitative evaluation is complete (Chapter 7, §7.3–7.4) and the LLM-as-a-Judge evaluation is complete (§7.7, 60/60 evaluations, addressing RQ4 directly). The qualitative human evaluation component was descoped from its originally stated ≥50-participant design to a 15–20 participant pilot (§7.6) for time-constraint reasons logged during Sprint 6, then distributed and collected in full: n = 29 real responses, exceeding the reduced pilot target. Its results are directionally consistent with the LLM-judge findings (the system leads both baselines on raw mean curiosity and emotional impact), but none of the Wilcoxon comparisons reach significance at this sample size, and the human/LLM inter-rater reliability check came back weak and non-significant ($r = -0.14$, §7.6). The objective's quantitative half stands on solid, complete evidence; its qualitative half is now also fully executed, but its statistical power is limited by the pilot's reduced scale rather than the survey being incomplete. Reporting this objective as unqualifiedly met would overstate the strength of the human-evaluation evidence specifically; reporting it as only partially met would understate that every planned measurement was actually carried out. "Met, with the pilot-scale human survey's limited statistical power carried forward honestly" is the accurate description, and is preferred here over either alternative.

Table 8.1: Objective Accomplishment Summary

Objective	Status	Primary Evidence
2.1 To Identify	Substantially met	T1–T7 taxonomy implemented (Ch. 5–6); pCTR F1 = 0.69, ROC-AUC = 0.76 (§7.3), moderate predictive signal
2.2 To Analyse	Met	System significantly beats Baseline A ($p < 0.002$) and beats Baseline B on relevance ($p = 0.0047$); other System-vs-B gaps not significant at n = 20 (§7.7)
2.3 To Design and Develop	Fully met	Working three-agent

Objective	Status	Primary Evidence
		system, Facade API, 11/11 functional tests pass (§7.2)
2.4 To Evaluate	Met, power-limited on human data	Quantitative + LLM-judge evaluation complete; human survey executed (n=29), directionally consistent but not statistically significant (§7.6)

8.3 Problems encountered during research and development

This section draws together the deviations already logged in place throughout PLAN_IMPLEMENTATION.md and Chapters 6–7, rather than restating each in full. The intent here is to characterise the shape of the difficulties encountered, not to duplicate their individual write-ups.

Data availability shaped the ML model more than technique choice did. Neither Kaggle source dataset used (Section 4.4) carries a real subscriber-count or CTR field, forcing a defaulted channel_subscriber_tier and a category-median-Views proxy for Virality_Score (Section 6.7, Table 6.4). This, not the choice of Random Forest over XGBoost, or any hyperparameter decision, is the most plausible root cause of the pCTR model's F1/ROC-AUC gap against its targets (Section 7.3, 7.9). The lesson generalises beyond this specific project: for a metric-optimisation system like this one, the ceiling on model quality is frequently set by what the available data can support, not by modelling effort applied after data collection is fixed.

A genuine concurrency bug, not merely a performance one, was found and fixed during Sprint 5. The original sequential Critic scoring produced a ~29.5 s average latency, well outside the NFR1 budget; the first parallelisation attempt using asyncio.gather across the three scorers introduced a real correctness bug: concurrently loading two different transformer models (bart-large-mnli and roberta-large) on separate threads corrupted transformers/accelerate's meta-device weight materialisation, a failure mode that reproduced reliably but was not fixed by a naive per-module lock (Section 6.8, PLAN_IMPLEMENTATION.md §5.2). The eventual fix, eager model loading at import time plus a lock around each model's forward pass, is documented as a deliberate

architectural decision rather than a quick patch, and stands as the clearest example in this project of a problem that initially presented as a performance issue turning out to require a correctness-level fix.

The human evaluation study had to be descoped twice under a fixed timeline, and the reasoning for each cut is preserved rather than only the final number. The original design (10 videos × 3 titles × 4 dimensions = 120 questions) was judged impractical for genuine respondent completion; a first-pass cut to 5 videos (60 questions) was judged still too long; the final 3-video, 36-question design was the one judged realistically completable in 8–10 minutes (Section 7.6). This is recorded here as a real, non-trivial project-management problem rather than glossed as a simple scope reduction: designing a human study that is both statistically meaningful and realistically completable by volunteer participants is a genuine constraint-satisfaction problem, and this project's answer, reduce scope until a completion-time budget is met and report the reduction explicitly, is offered as the honest resolution actually reached, not necessarily the only valid one.

Baseline A required a documented compromise from its original definition. SOT §19.3 specifies Baseline A as "ground truth from video": a real, human-written title from the actual video. Because the evaluation set (Section 7.4) consists of synthetic transcripts with no real underlying YouTube upload, no literal ground-truth title exists to draw from; video_description was substituted as a documented stand-in (Section 6.12, 7.6). This is a small but real gap between what the SOT specified and what synthetic evaluation data can support, and is one of the clearer examples in this project of a design document written before implementation encountering a constraint (the absence of real published videos in the evaluation set) that implementation then had to resolve pragmatically and log rather than silently work around.

8.4 Self-reflection on the research process

Working simultaneously as the implementer of the system and the author documenting it produced one clear methodological benefit and one clear risk, both worth naming directly. The benefit: every chapter in this thesis, from Chapter 5 onward, could be and was written as an as-built account grounded in the actual working repository rather than a forward-looking design document reconciled with reality only at the end. Chapter 6's

Table 6.4 and Chapter 7's Tables 7.5–6.6 exist as artefacts of this discipline, not retrofitted summaries. The risk: a single person building and evaluating their own system is structurally closer to the results than an independent evaluator would be, which is precisely why the honest, sometimes unflattering reporting of unmet targets (pCTR F1/ROC-AUC, VADER arousal, BERTScore) and of statistically inconclusive results (the human survey's non-significant Wilcoxon tests and its weak inter-rater reliability against the LLM judge) throughout Chapter 7 was treated as non-negotiable rather than optional. The credibility of a self-evaluated system rests almost entirely on whether its author reports what did not work, or did not reach significance, as carefully as what did.

The project's own timeline discipline, keeping PLAN_IMPLEMENTATION.md's sprint checklists current in real time, with deviations logged at the point they occurred rather than reconstructed later, turned out to be the single practice that made the later thesis chapters possible to write accurately at all. Chapters 6 and 7 in particular would have been substantially harder to write, and more likely to silently smooth over real deviations, without that running log to draw from.

If repeating this project, the clearest process change would be starting the human evaluation survey's distribution earlier and in parallel with the automatable evaluation work (§7.3–7.4, 7.7), rather than sequencing it after, since the two do not depend on each other. In this project the survey was in fact the last piece of Chapter 7 to close out, distributed only once the automatable evaluations were already complete; running both in parallel from the start of Sprint 6 would also have left more of the fixed timeline available to recruit toward the original n = 50+ target, rather than a 29-respondent pilot that turned out directionally right but underpowered to confirm it statistically.

8.5 Real-world application possibilities

The system as built, while explicitly scoped as a research prototype (Section 1.9) rather than a production system, points toward several realistic application contexts, distinct from the productisation work Section 1.9 excludes:

- **Independent creators and educators**, the population most directly motivated in Section 1.5.1, are the most direct beneficiaries: a creator with a transcript but no marketing background could realistically use the deployed API (Section 6.9's

FastAPI Facade, or the demonstration frontend of Section 6.10) as-is for title suggestions, with the NLI faithfulness gate (Section 6.6) providing a concrete, automatic safeguard against the accidental clickbait risk a less experienced creator might otherwise introduce unknowingly.

- **Marketing and content teams at small organisations**, the non-profits and educational institutions named in Section 1.5.1, could plausibly integrate the API Wrapper into an existing content workflow tool without needing to understand or maintain the underlying multi-agent pipeline, precisely because the Facade pattern (Section 5.8) was chosen to hide that complexity behind a single REST endpoint.
- **A/B testing tooling for YouTube Studio-integrated products** is a natural, though currently out-of-scope (Section 1.9), extension: the system's title candidates could feed an external A/B testing harness rather than the system needing to implement live CTR measurement itself, which Section 1.9 explicitly and deliberately excludes from this project's own claims.

None of these applications depends on the human evaluation study reaching statistical significance to be technically usable today; all three would, however, benefit from a larger-scale replication of the survey (§8.6, recommendation 1) than this thesis's pilot-scale data (n = 29) can provide, since real user preference data at adequate statistical power is the strongest available signal for whether the system's titles are actually preferred by the people it is meant to serve, as opposed to only scoring well on automated proxies.

8.6 Future recommendations

Consistent with SOT §20.2 and grounded in what Chapter 7's evaluation specifically surfaced as gaps, the following are recommended as the most consequential next steps, in rough priority order:

1. **Scale up the human evaluation survey toward the original n = 50+ design.** The pilot (n = 29, Section 7.6) is directionally consistent with the LLM-judge findings but statistically underpowered: every Wilcoxon comparison returned $p > 0.1$. The tooling, instrument, and analysis pipeline are all already built and proven on real data; reaching the original target sample size, and ideally restoring some of the videos cut during the two descoping rounds (Section 8.3), is the most

immediate and lowest-effort recommendation here, not a long-term research direction, since it requires no new design or engineering work, only more respondents.

2. **Replace VADER with a fine-tuned arousal classifier trained on title-length text**, to close the gap identified in Section 7.4/7.9 between VADER's lexicon-coverage limitations and the system's likely true emotional-engagement performance. A more accurate instrument here would clarify whether the current VADER-arousal shortfall reflects the metric or the system.
3. **Widen the pCTR model's hyperparameter search and, where possible, source real subscriber-count and CTR data** (e.g. via the YouTube Data API with creator consent, as SOT §20.1 already proposes), to close the F1/ROC-AUC gap identified in Section 7.3, which this thesis has argued is more plausibly a data-availability limitation than a technique limitation.
4. **Thumbnail integration**, extending the system to multimodal (title + thumbnail) joint optimisation, as originally scoped out in Section 1.9 and reaffirmed in SOT §20.2, the single most requested extension in the original research design, unchanged by this project's findings.
5. **Multi-language support**, localising the T1–T7 trigger taxonomy and Prompt Database per language and cultural context, addressing the English-only exclusion in Section 1.9.
6. **Real-time A/B integration with YouTube Studio**, closing the feedback loop this system currently only approximates via the pCTR proxy model, consistent with the real-world application described in Section 8.5.
7. **A domain-specific fine-tuned LLM**, trained on high-CTR YouTube titles, as a longer-term alternative to prompt-engineering GPT-4o, which SOT §20.2 already anticipates as a way to reduce the system's current dependency on a single external API provider.

8.7 Chapter summary

Of the four research objectives set out in Chapter 2, one (2.3, system design and development) is fully met, three (2.1, feature identification; 2.2, prompting-technique analysis; 2.4, evaluation) are met with an honestly reported qualification: moderate ML

predictive power in one case, a statistically bounded rather than sweeping advantage over the strongest baseline in another, and a pilot-scale human survey that is directionally consistent with but not a statistically significant confirmation of the automated evaluation findings in the third. This thesis has argued throughout Chapters 5–7, and reaffirms here, that reporting this evaluation profile precisely, including every gap and every non-significant result, is itself part of a design science contribution, not a weakness to be minimised. The system was built, evaluated on real (if imperfect) data including n = 29 genuine human respondents, and every deviation from its original plan and every unmet target was logged and explained at the point it occurred rather than reconciled only in retrospect. The recommendations of Section 8.6, led by scaling the human survey toward its original statistical-power target, describe a realistic and directly actionable path from the system's current state to a fuller validation of the research question posed in Chapter 1.

REFERENCES

References cited in Chapter 1: Introduction

[1] YouTube. (2024). *YouTube for Press, Statistics.* https://blog.youtube/press/

[2] P. Covington, J. Adams, and E. Sargin, "Deep Neural Networks for YouTube Recommendations," in *Proc. 10th ACM Conf. Recommender Systems (RecSys '16)*, pp. 191–198, 2016. DOI: 10.1145/2959100.2959190

[3] YouTube Help. (2024). *Impressions and how they lead to watch time.* https://support.google.com/youtube/answer/9314486

[4] G. Loewenstein, "The psychology of curiosity: A review and reinterpretation," *Psychological Bulletin*, vol. 116, no. 1, pp. 75–98, 1994. DOI: 10.1037/0033-2909.116.1.75

[5] D. Kahneman and A. Tversky, "Prospect Theory: An Analysis of Decision under Risk," *Econometrica*, vol. 47, no. 2, pp. 263–291, 1979. DOI: 10.2307/1914185

[6] R. B. Cialdini, *Influence: The Psychology of Persuasion.* New York: Harper & Row, 1984.

[7] J. Berger and K. L. Milkman, "What Makes Online Content Viral?" *Journal of Marketing Research*, vol. 49, no. 2, pp. 192–205, 2012. DOI: 10.1509/jmr.10.0353

[8] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. arXiv: 1706.03762

[9] W. Yang, "Beyond algorithms: The human touch machine-generated titles for enhancing click-through rates on social media," *PLOS ONE*, vol. 19, no. 7, e0306639, Jul. 2024. DOI: 10.1371/journal.pone.0306639

[10] X. Zhang, C. Sun, R. Zhang, and K.-Y. Goh, "The Value of AI-Generated Metadata for UGC Platforms: Evidence from a Large-scale Field Experiment," arXiv:2412.18337, Dec. 2024.

[11] YouTube. (2024). *Terms of Service, Section 4H.* https://www.youtube.com/t/terms

[12] "When curiosity gaps backfire: effects of headline concreteness on information selection decisions," *Scientific Reports*, 2024. DOI: 10.1038/s41598-024-81575-9

[13] T. B. Brown et al., “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv: 2005.14165

[14] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. DOI: 10.5555/3600270.3602070

[15] OpenAI. (2024). *GPT-4o Model Card and Pricing.* https://openai.com/pricing

References cited in Chapter 3: Literature Review

[1] G. Loewenstein, “The psychology of curiosity: A review and reinterpretation,” *Psychological Bulletin*, vol. 116, no. 1, pp. 75–98, 1994. DOI: 10.1037/0033-2909.116.1.75

[2] R. Golman and G. Loewenstein, “Information Gaps: A Theory of Preferences Regarding the Presence and Absence of Information,” *Decision*, 2016. DOI: 10.1037/dec0000068

[3] D. Kahneman and A. Tversky, “Prospect Theory: An Analysis of Decision under Risk,” *Econometrica*, vol. 47, no. 2, pp. 263–291, 1979. DOI: 10.2307/1914185

[4] R. B. Cialdini, *Influence: The Psychology of Persuasion.* New York: Harper & Row, 1984.

[5] J. Berger and K. L. Milkman, “What Makes Online Content Viral?” *Journal of Marketing Research*, vol. 49, no. 2, pp. 192–205, 2012. DOI: 10.1509/jmr.10.0353

[6] P. Covington, J. Adams, and E. Sargin, “Deep Neural Networks for YouTube Recommendations,” in *Proc. 10th ACM Conf. Recommender Systems (RecSys ’16)*, pp. 191–198, 2016. DOI: 10.1145/2959100.2959190

[7] T. Kawata, T. Kato, and K. Tsuda, “Effect of Title Length and Word Count on Views: Evidence from Microsoft Excel Videos on YouTube,” *EAI Endorsed Trans. Scalable Information Systems*, vol. 12, no. 1, 2025. DOI: 10.4108/eetsis.7779

[8] X. Zhang, C. Sun, R. Zhang, and K.-Y. Goh, “The Value of AI-Generated Metadata for UGC Platforms: Evidence from a Large-scale Field Experiment,” arXiv:2412.18337, Dec. 2024.

[9] A. Vaswani et al., “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. arXiv:1706.03762

[10] T. B. Brown et al., “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2005.14165

[11] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. DOI: 10.5555/3600270.3602070

[12] W. Yang, “Beyond algorithms: The human touch machine-generated titles for enhancing click-through rates on social media,” *PLOS ONE*, vol. 19, no. 7, e0306639, Jul. 2024. DOI: 10.1371/journal.pone.0306639

[13] C. J. Hutto and E. Gilbert, “VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text,” in *Proc. 8th Int. AAAI Conf. Weblogs and Social Media (ICWSM)*, 2014.

[14] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating Text Generation with BERT,” in *Int. Conf. Learning Representations (ICLR)*, 2020. arXiv:1904.09675

[15] P. Laban et al., “With a Little Push, NLI Models can Robustly and Efficiently Predict Faithfulness,” in *Proc. 61st Annual Meeting of the ACL (Short Papers)*, 2023. DOI: 10.18653/v1/2023.acl-short.79

[16] M. Aslam et al., “Optimizing Prediction of YouTube Video Popularity Using XGBoost,” *Electronics*, vol. 10, no. 23, p. 2962, 2021. DOI: 10.3390/electronics10232962

[17] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A Design Science Research Methodology for Information Systems Research,” *J. Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007. DOI: 10.2753/MIS0742-1222240302

[18] H. Chen, “Title Matters: Impacts of Titles on User Engagement in Short Video Platforms,” PhD Dissertation, Singapore Management University, 2023.

[19] YouTube. (2024). *YouTube for Press, Statistics.* https://blog.youtube/press/

[20] YouTube Help. (2024). *Impressions and how they lead to watch time.* https://support.google.com/youtube/answer/9314486

[21] ThumbMentor. (2026). *YouTube CTR Benchmarks 2026: What Is a Good Click-Through Rate by Niche.* https://thumbmentor.com/en/blog/youtube-ctr-benchmarks

[22] A. Davoudi Monfared and K. Murthy, "Clickbait Detection in YouTube Videos," arXiv:2107.12791, Jul. 2021. https://arxiv.org/abs/2107.12791

[23] OpenAI. (2024). *GPT-4o System Card and Model Documentation.* https://openai.com/index/gpt-4o-system-card/

[24] G. Paskalev, "Can You Predict a YouTube Video's CTR using Machine Learning?" *Medium*, n.d. https://georgepaskalev.medium.com/can-you-predict-a-youtube-videos-ctr-using-machine-learning-bc20bb142a47

[25] J. Merrell, "YouTube Views with XGBoost," personal project, n.d. https://jmmerrell.github.io/YouTube_Views_XGBoost/

[26] R. Shav, "YouTube Trending Video Dataset," Kaggle, 2023. https://www.kaggle.com/datasets/rsrishav/youtube-trending-video-dataset

[27] datasnaek, "YouTube Trending Video Statistics," Kaggle, n.d. https://www.kaggle.com/datasets/datasnaek/youtube-new

[28] LangChain. (2024). *Multi-Agent Systems, Supervisor Pattern.* LangChain Documentation. https://docs.langchain.com/oss/python/langchain/multi-agent

[29] M. Potthast, S. Köpsel, B. Stein, and M. Hagen, "Clickbait Detection," in *Proc. 38th European Conf. Information Retrieval (ECIR)*, 2016. https://webis.de/data/webis-clickbait-17.html

[30] "LLM-RUBRIC: A Multidimensional, Calibrated Approach to Automated Evaluation of Natural Language Texts," in *Proc. 62nd Annual Meeting of the ACL*, 2024. https://aclanthology.org/2024.acl-long.745.pdf

[31], (reserved)

[32] D. Horton and R. R. Wohl, "Mass Communication and Para-Social Interaction: Observations on Intimacy at a Distance," *Psychiatry*, vol. 19, no. 3, pp. 215–229, 1956.

[33] D. Kahneman, *Thinking, Fast and Slow.* New York: Farrar, Straus and Giroux, 2011.

[34] J. White et al., "A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT," arXiv:2302.11382, Feb. 2023. https://arxiv.org/abs/2302.11382

[35] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software.* Reading, MA: Addison-Wesley, 1994.

[36] Y. Zhu et al., "Texygen: A Benchmarking Platform for Text Generation Models," in *Proc. 41st Int. ACM SIGIR Conf. on Research and Development in Information Retrieval*, 2018. DOI: 10.1145/3209978.3210080

References cited in Chapter 4: Methodology

[1] J. W. Creswell and J. D. Creswell, *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*, 5th ed. London: SAGE Publications, 2018.

[2] R. K. Kovács and C. Spens, "Abductive reasoning in logistics research," *International Journal of Physical Distribution & Logistics Management*, vol. 35, no. 2, pp. 132–144, 2005.

[3] G. Loewenstein, "The psychology of curiosity: A review and reinterpretation," *Psychological Bulletin*, vol. 116, no. 1, pp. 75–98, 1994. DOI: 10.1037/0033-2909.116.1.75

[4] D. Kahneman and A. Tversky, "Prospect Theory: An Analysis of Decision under Risk," *Econometrica*, vol. 47, no. 2, pp. 263–291, 1979. DOI: 10.2307/1914185

[5] R. B. Cialdini, *Influence: The Psychology of Persuasion.* New York: Harper & Row, 1984.

[6] J. Berger and K. L. Milkman, "What Makes Online Content Viral?" *Journal of Marketing Research*, vol. 49, no. 2, pp. 192–205, 2012. DOI: 10.1509/jmr.10.0353

[7] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A Design Science Research Methodology for Information Systems Research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007. DOI: 10.2753/MIS0742-1222240302

[8] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design Science in Information Systems Research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004. DOI: 10.2307/25148625

[9] C. J. Hutto and E. Gilbert, "VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text," in *Proc. 8th Int. AAAI Conf. Weblogs and Social Media (ICWSM)*, 2014.

[10] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating Text Generation with BERT," in *Int. Conf. Learning Representations (ICLR)*, 2020. arXiv:1904.09675

[11] P. Laban et al., "With a Little Push, NLI Models can Robustly and Efficiently Predict Faithfulness," in *Proc. 61st Annual Meeting of the ACL (Short Papers)*, 2023. DOI: 10.18653/v1/2023.acl-short.79

[12] T. B. Brown et al., "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2005.14165

[13] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. DOI: 10.5555/3600270.3602070

[14] P. Covington, J. Adams, and E. Sargin, "Deep Neural Networks for YouTube Recommendations," in *Proc. 10th ACM Conf. Recommender Systems (RecSys '16)*, pp. 191–198, 2016. DOI: 10.1145/2959100.2959190

[15] X. Zhang, C. Sun, R. Zhang, and K.-Y. Goh, "The Value of AI-Generated Metadata for UGC Platforms: Evidence from a Large-scale Field Experiment," arXiv:2412.18337, Dec. 2024.

[16] R. Shav, "YouTube Trending Video Dataset," Kaggle, 2023. https://www.kaggle.com/datasets/rsrishav/youtube-trending-video-dataset

[17] datasnaek, "YouTube Trending Video Statistics," Kaggle, n.d. https://www.kaggle.com/datasets/datasnaek/youtube-new

[18] M. Potthast, S. Köpsel, B. Stein, and M. Hagen, "Clickbait Detection," in *Proc. 38th European Conf. Information Retrieval (ECIR)*, 2016. https://webis.de/data/webis-clickbait-17.html

[19] YouTube. (2024). *Terms of Service.* https://www.youtube.com/t/terms

[20] K. Schwaber and J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game.* Scrum.org, 2020. https://scrumguides.org/scrum-guide.html

References cited in Chapter 5: System Requirements Specification

[1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software.* Reading, MA: Addison-Wesley, 1994.

[2] I. Sommerville, *Software Engineering*, 10th ed. Harlow: Pearson Education, 2016.

[3] I. Jacobson, M. Christerson, P. Jonsson, and G. Övergaard, *Object-Oriented Software Engineering: A Use Case Driven Approach.* Wokingham: Addison-Wesley, 1992.

[4] Object Management Group, *OMG Unified Modeling Language (OMG UML), Version 2.5.1.* OMG Document formal/2017-12-05, Dec. 2017.

[5] LangChain. (2024). *Multi-Agent Systems: Supervisor Pattern.* LangChain Documentation. https://docs.langchain.com/oss/python/langchain/multi-agent

References cited in Chapter 6: Implementation

[1] C. J. Hutto and E. Gilbert, "VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text," in *Proc. 8th Int. AAAI Conf. Weblogs and Social Media (ICWSM)*, 2014.

[2] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating Text Generation with BERT," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2020.

[3] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Proc. Neural Information Processing Systems (NeurIPS)*, 2022.

[4] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, et al., "Language Models are Few-Shot Learners," in *Proc. Neural Information Processing Systems (NeurIPS)*, 2020.

[5] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

[6] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[7] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016.

References cited in Chapter 7: Results

[1] C. J. Hutto and E. Gilbert, “VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text,” in *Proc. 8th Int. AAAI Conf. Weblogs and Social Media (ICWSM)*, 2014.